

Universidad
Tecnológica
de Pereira

Estabilidad de las explicaciones de redes neuronales de grafos para la detección de lavado de dinero bajo desbalance extremo de clases

Alejandro Gómez Huertas
Juan Diego Garzón Ovalle

Universidad Tecnológica de Pereira
Facultad de Ingenierías
Maestría en Ingeniería de Sistemas y Computación
Pereira, Colombia
2026

Estabilidad de las explicaciones de redes neuronales de grafos para la detección de lavado de dinero bajo desbalance extremo de clases

Alejandro Gómez Huertas
Juan Diego Garzón Ovalle

Tesis presentada como requisito parcial para optar al título de:

Magíster en Ingeniería de Sistemas y Computación

Director
Ph.D. Cristian Rosero Arias

Universidad Tecnológica de Pereira
Facultad de Ingenierías
Maestría en Ingeniería de Sistemas y Computación
Pereira, Colombia
2026

Resumen

Las redes neuronales de grafos (GNN) se usan cada vez más para detectar lavado de dinero en criptomonedas, y en un entorno regulado sus alertas deben poder explicarse. Para que una explicación sirva como evidencia debe ser estable: el mismo método, aplicado a la misma transacción y al mismo modelo, debe dar el mismo resultado. Esta tesis estudia cómo cambia esa estabilidad cuando se varían el nivel de desbalance de clases, la arquitectura y la técnica que compensa el desbalance. Sobre el conjunto de datos Elliptic de transacciones de Bitcoin se entrenaron 216 modelos: cuatro arquitecturas (GCN, GraphSAGE, GAT y TAGCN), seis niveles de desbalance (de 1:10 a 1:200), tres funciones de pérdida y tres semillas. Sus predicciones se explicaron con GNNExplainer, PGExplainer, valores de Shapley, Integrated Gradients y Expected Gradients. Como la partición temporal habitual deja la prueba después del cierre de un mercado ilegal, donde ningún modelo supera al azar, el estudio usa solo el periodo anterior al cierre; con él, el mejor modelo alcanza un PR-AUC de 0,31 en prueba, frente a 0,02 del azar. La técnica de compensación cambia mucho el rendimiento, y de ella depende el efecto del desbalance: con pesos por clase la estabilidad no cambia entre 1:10 y 1:200, mientras que con *focal loss* o sin ajuste la de GNNExplainer baja junto con el rendimiento del modelo. La arquitectura no cambia la estabilidad. Un control con pesos aleatorios muestra por qué: con la correlación de Spearman sobre todas las variables, la explicación de un modelo entrenado se parece a la de una red sin entrenar casi tanto como dos repeticiones entre sí, porque cerca de la mitad de las variables vale cero y los explicadores siempre la dejan al final. Las diez variables principales sí dependen del modelo, y cambian más entre semillas del entrenamiento que entre niveles de desbalance. La tesis concluye que la estabilidad de una explicación no basta para confiar en ella, y recomienda acompañarla de un control de pesos aleatorios y de una medida sobre las variables principales.

Palabras clave: redes neuronales de grafos, explicabilidad, estabilidad de las explicaciones, lavado de dinero, desbalance de clases, Elliptic.

Abstract

Graph neural networks (GNNs) are increasingly used to detect money laundering in cryptocurrencies, and in a regulated setting their alerts must be explainable. For an explanation to serve as evidence it must be stable: the same method, applied to the same transaction and the same model, should give the same result. This thesis studies how that stability changes when the level of class imbalance, the architecture, and the technique used to compensate for imbalance are varied. On the Elliptic Bitcoin transaction dataset, 216 models were trained: four architectures (GCN, GraphSAGE, GAT, and TAGCN), six imbalance levels (from 1:10 to 1:200), three loss functions, and three seeds. Their predictions were explained with GNNExplainer, PGExplainer, Shapley values, Integrated Gradients, and Expected Gradients. Because the usual temporal split places the test set after the closure of a dark market, where no model beats chance, the study uses only the period before the closure; with it, the best model reaches a test PR-AUC of 0.31, against 0.02 for chance. The compensation technique changes performance substantially, and the effect of imbalance depends on it: with class weights, stability does not change between 1:10 and 1:200, whereas with focal loss or no adjustment the stability of GNNExplainer drops together with model performance. Architecture does not change stability. A randomized-weights control shows why: with the Spearman correlation over all features, the explanation of a trained model resembles that of an untrained network almost as much as two repetitions resemble each other, because about half of the features are zero and the explainers always rank them last. The ten top features do depend on the model, and they vary more across training seeds than across imbalance levels. The thesis concludes that the stability of an explanation is not enough to trust it, and recommends complementing it with a randomized-weights control and a measure on the top features.

Keywords: graph neural networks, explainability, explanation stability, money laundering, class imbalance, Elliptic.

Índice general

Resumen **i**

Abstract **ii**

Índice de figuras **ix**

Índice de tablas **xii**

Lista de acrónimos **xiii**

1. Introducción **1**

1.1. Planteamiento del problema 1

1.2. Una limitación del conjunto de datos que condiciona el diseño 2

1.3. Preguntas de investigación 2

1.4. Hipótesis 3

1.5. Objetivos 3

1.5.1. Objetivo general 3

1.5.2. Objetivos específicos 3

1.6. Justificación 4

1.7. Contribuciones 5

1.8. Alcance y limitaciones 5

1.9. Estructura del documento 5

2. Marco contextual **6**

2.1. El lavado de dinero 6

2.1.1. Las tres etapas 6
2.1.2. La escala del problema 7
2.2. Criptomonedas y lavado de dinero 7
2.2.1. Qué cambia con las criptomonedas 7
2.2.2. Seudonimato y trazabilidad 8
2.3. Sistemas de monitoreo de transacciones 8
2.3.1. Monitoreo basado en reglas 8
2.3.2. Monitoreo con aprendizaje automático 8
2.4. Regulación e inteligencia artificial explicable 9
2.5. El conjunto de datos Elliptic 9

3. Marco teórico **11**
3.1. Del aprendizaje automático al aprendizaje sobre grafos 11
3.2. Redes neuronales de grafos 12
3.3. Propiedades de los grafos que condicionan el aprendizaje 13
3.4. Desbalance de clases 14
3.5. Métricas de rendimiento bajo desbalance 14
3.6. Explicabilidad de modelos 15
3.7. Evaluación de las explicaciones 16
3.8. Medidas de acuerdo entre explicaciones 17
3.9. Pruebas estadísticas 18

4. Estado del arte **19**
4.1. Detección de lavado de dinero con grafos 19
4.2. Explicabilidad en la detección de lavado de dinero 20
4.3. Estabilidad de las explicaciones y desbalance de clases 21
4.4. Síntesis comparativa 21
4.5. Vacío de investigación 21

5. Metodología **23**

5.1. Datos 24

5.2. Análisis exploratorio del grafo 24

5.3. Partición temporal y exclusión del cierre del mercado 25

5.4. Escenarios de desbalance 26

5.5. Modelos y entrenamiento 27

5.6. Explicadores 28

5.7. Medidas de estabilidad 30

5.8. Control de pesos aleatorios 31

5.9. Análisis estadístico 31

6. Resultados **33**

6.1. La partición sin el cierre recupera la prueba 33

6.2. Rendimiento según el escenario y la pérdida 34

6.3. De la validación a la prueba 36

6.4. Rendimiento en los escenarios de estrés 37

6.5. H1: el desbalance solo afecta la estabilidad cuando la pérdida no lo compensa . 38

6.6. H2: no hay un orden estable entre arquitecturas 39

6.7. H3: la técnica de compensación no cambia la estabilidad 41

6.8. Estabilidad y rendimiento del modelo 42

6.9. Qué parte de la estabilidad depende del modelo 43

6.10. Un ejemplo: la explicación de una transacción 46

6.11. Costo computacional de los explicadores 47

6.12. Síntesis 47

7. Discusión **49**

7.1. Cuándo el desbalance cambia la estabilidad 49

7.2. Arquitectura y estabilidad 51

7.3. La técnica de compensación 51

7.4. Qué mide la estabilidad de una explicación 52

7.5. Implicaciones para la auditoría 53

7.6. El cierre del mercado y la evaluación en Elliptic 53
7.7. Lecciones metodológicas del proceso 54
7.8. Limitaciones 54

8. Conclusiones **56**
8.1. Respuestas a las preguntas de investigación 56
8.2. Aportes 57
8.3. Trabajo futuro 57
8.4. Reflexión final 58

Referencias **59**

A. Corrida con la partición original **66**
A.1. La prueba cae al nivel del azar 66
A.2. Lo que se mantiene en las dos particiones 66
A.3. Lo que cambia 67
A.4. Estabilidad por escenario en la partición original 67

B. Eje sintético **69**
B.1. Diseño 69
B.2. Resultados 70
B.3. Advertencias 70

C. Reproducibilidad **71**
C.1. Entorno 71
C.2. Corrida principal (sin el cierre del mercado) 71
C.3. Tablas, figuras y bibliografía del manuscrito 71
C.4. Determinismo 72

D. Tablas completas **73**

E. Estabilidad detallada y transacciones explicadas **78**
E.1. Estabilidad por escenario y arquitectura 78
E.2. Las transacciones explicadas 78

F. Glosario **81**

Índice de figuras

5.1. Flujo experimental de la corrida principal. 23

5.2. (a) Distribución del número de aristas por transacción según su clase, en escala logarítmica. (b) Clase de las vecinas de las transacciones de cada clase, contando cada arista desde sus dos extremos. 25

5.3. (a) Transacciones ilícitas (barras) y lícitas (línea) por paso de tiempo en Elliptic. La línea discontinua marca el cierre del mercado ilegal en el paso 43. (b) Partición original y partición sin el cierre usada en el estudio principal; en esta última los pasos 43 a 49 no se usan. 26

6.1. Rendimiento en prueba con la partición original (prueba en los pasos 43 a 49, después del cierre) y con la partición sin el cierre (prueba en los pasos 36 a 42). Escenario nativo, pérdida elegida por validación. Las barras son la media de tres semillas, los puntos cada semilla y las líneas discontinuas el valor del azar: la prevalencia en (a) y 0,5 en (b). 34

6.2. Curvas de precisión y exhaustividad (a) y ROC (b) en la prueba de la partición sin el cierre, escenario nativo, con la pérdida elegida por validación para cada arquitectura. Trazo grueso: semilla 42; trazos finos: semillas 43 y 44. Entre paréntesis, el área media de las tres semillas. 35

6.3. PR-AUC de validación por escenario y pérdida (media y desviación estándar de tres semillas). La línea discontinua es el valor del azar (0,037). 35

6.4. Curvas de precisión y exhaustividad (a) y ROC (b) en validación (pasos 30 a 35), escenario nativo, con la pérdida elegida para cada arquitectura. Trazo grueso: semilla 42; trazos finos: semillas 43 y 44. 37

6.5. Curvas de precisión y exhaustividad en prueba para los escenarios de estrés, con pesos por clase. Trazo grueso: semilla 42; trazos finos: semillas 43 y 44. La línea discontinua es la prevalencia (0,023). 37

6.6. Estabilidad entre réplicas por escenario y explicador, con las tres medidas. Pesos por clase; media de 9 modelos (3 arquitecturas que aprenden por 3 semillas) e intervalo *bootstrap* del 95 %. 38

6.7. Estabilidad de GNNExplainer por arquitectura con las tres medidas (pesos por clase, media de los cuatro escenarios principales). Cada punto es una semilla; la línea horizontal es la media. 41

6.8. Estabilidad de GNNExplainer frente al PR-AUC de validación, un punto por modelo (216 modelos). La línea discontinua es el valor del azar (0,037). Los grupos de puntos casi superpuestos son modelos de una misma arquitectura, pérdida y semilla en distintos escenarios. 43

6.9. Control de pesos aleatorios en el escenario nativo con pesos por clase (12 modelos: 4 arquitecturas por 3 semillas). Para cada explicador, la estabilidad entre réplicas con la red entrenada y con una red de pesos aleatorios, y el acuerdo entre la explicación de la red entrenada y la de la red aleatoria. 44

6.10. Acuerdo entre explicaciones según qué cambia entre los dos modelos comparados (GraphSAGE, GAT y TAGCN; pesos por clase; escenarios principales). Réplicas del mismo modelo es la estabilidad entre réplicas. 45

6.11. Acuerdo entre explicadores sobre el mismo modelo (GraphSAGE, GAT y TAGCN; escenarios principales; todas las pérdidas y semillas), con el Spearman global (a) y el Jaccard top-10 (b). 46

6.12. Importancia normalizada (máximo igual a 1) de las variables más importantes para una transacción ilícita de validación. (a) Los cuatro explicadores de variables sobre el mismo modelo (GraphSAGE, pesos por clase, nativo, semilla 42). (b) GNNExplainer sobre los modelos de las tres semillas. Un punto marca las variables que están entre las 10 principales de esa fila. 46

Índice de tablas

4.1. Trabajos relacionados según su cobertura de los cuatro rasgos que combina la tesis, de acuerdo con lo que declara el resumen de cada trabajo. ✓: lo declara; –: no lo declara. Desbalance controlado significa variar el nivel de desbalance, no solo la técnica que lo compensa. 22

5.1. Factores del diseño experimental. Cada combinación se entrena con tres semillas de modelo, lo que da $4 \times 6 \times 3 \times 3 = 216$ modelos. 23

5.2. Transacciones por parte de cada partición. Negativas son lícitas más sin etiqueta; la razón es negativas por cada ilícita. 26

6.1. Rendimiento en el escenario nativo con la pérdida elegida por validación (media $\pm$ desviación estándar de tres semillas). Filtro: modelos que superan el filtro de calidad en validación. 34

6.2. PR-AUC de validación por escenario, arquitectura y pérdida (media de tres semillas). 36

6.3. Modelos que superan el filtro de calidad en validación (F1 $\geq$ 0,30 y MCC $\geq$ 0,15 con umbral 0,5), de 18 por celda (6 escenarios por 3 semillas). 36

6.4. Estabilidad entre réplicas por escenario (pesos por clase, 9 modelos). *p* Friedman: igualdad de los seis escenarios. *p* TOST: equivalencia entre nativo y 1:200 con margen 0,05 (un valor bajo indica equivalencia). 38

6.5. Estabilidad entre réplicas por escenario y pérdida (GraphSAGE, GAT y TAGCN; media de 9 modelos por celda). 40

6.6. Estabilidad por arquitectura (pesos por clase, media de los escenarios principales, 3 semillas). *p* (4): Friedman con las cuatro arquitecturas; *p* (3): sin GCN. Primera: número de semillas en que cada arquitectura obtiene el mayor valor. . 41

6.7. Estabilidad por técnica de compensación (3 arquitecturas que aprenden, media de los escenarios principales). PC $>$ FL y PC $>$ SA: en cuántos de los 9 pares de arquitectura y semilla pesos por clase supera a *focal loss* o a sin ajuste, con el p de la prueba de signos. 42

6.8. Estabilidad de PGExplainer sobre las aristas del subgrafo: Spearman entre réplicas y Jaccard de las 20 aristas principales (escenarios principales, media de 3 semillas). 42

6.9. Control de pesos aleatorios (media de 12 modelos). 43

6.10. Acuerdo entre las explicaciones de dos arquitecturas con la misma semilla, escenario y pérdida (pesos por clase, escenarios principales). Rango sobre los seis pares de arquitecturas. 45

6.11. Mediana de segundos por transacción explicada (cinco réplicas) sobre todos los modelos. GNNExplainer, Shapley y PGExplainer se ejecutaron en GPU; los dos de gradiente, en CPU. 47

6.12. Veredicto de las hipótesis con la partición sin el cierre. 48

A.1. Estabilidad entre réplicas por escenario en la partición original (pesos por clase, media de 9 modelos). 68

B.1. Eje sintético: estabilidad (Spearman global), plausibilidad de variables y fidelidad (fid+; sobre aristas en GNNExplainer y sobre variables en Shapley), con su valor al azar entre paréntesis. Medias sobre 36 configuraciones por fila. Fuente: `results_phase1_v4/summary_v4.csv`. 70

D.1. Hiperparámetros elegidos por la búsqueda (semilla 42; las semillas 43 y 44 los reutilizan). h: dimensión oculta; L: capas; p: *dropout*; η: tasa de aprendizaje; λ: decaimiento de pesos; H/K: cabezas de GAT u orden de TAGCN. 73

D.2. Resultados completos por configuración (media de tres semillas). Val. y Prueba: PR-AUC; ROC: ROC-AUC de prueba; F: modelos que superan el filtro (de 3); VP: verdaderos positivos entre las 30 transacciones explicadas; GE y SH: estabilidad (Spearman global) de GNNExplainer y Shapley; J: Jaccard top-10 de GNNExplainer. 75

E.2. Las 30 transacciones ilícitas de validación explicadas. Nodo: índice en el grafo; Paso: paso de tiempo; Grado: número de aristas; Nulas: fracción de variables que valen 0 tras el escalado; Acierto: fracción de los 54 modelos con pesos por clase (GraphSAGE, GAT y TAGCN; 6 escenarios; 3 semillas) que la clasifican como ilícita con umbral 0,5. 78

E.1. Estabilidad entre réplicas por escenario y arquitectura (pesos por clase, media de 3 semillas). 80

Lista de acrónimos

AML	*Anti-Money Laundering*, prevención del lavado de dinero
EG	*Expected Gradients*
F1	Media armónica de precisión y exhaustividad
GAT	*Graph Attention Network*, red de grafos con atención
GCN	*Graph Convolutional Network*, red convolucional de grafos
GNN	*Graph Neural Network*, red neuronal de grafos
GraphSAGE	*Graph Sample and Aggregate*
IG	*Integrated Gradients*
MCC	Coeficiente de correlación de Matthews
PR-AUC	Área bajo la curva de precisión y exhaustividad
ROC-AUC	Área bajo la curva ROC
SMOTE	*Synthetic Minority Over-sampling Technique*
TAGCN	*Topology Adaptive Graph Convolutional Network*
TOST	*Two One-Sided Tests*, prueba de equivalencia
XAI	*Explainable Artificial Intelligence*, inteligencia artificial explicable

Capítulo 1

Introducción

1.1. Planteamiento del problema

La detección de lavado de dinero se apoya cada vez más en modelos de aprendizaje automático que examinan transacciones en busca de patrones anómalos [1, 2]. En el caso de las criptomonedas, las transacciones forman de manera natural un grafo, en el que cada transacción es un nodo y cada transferencia de fondos entre transacciones es una arista. Las redes neuronales de grafos (*Graph Neural Networks*, GNN) aprovechan esa estructura combinando los atributos de cada transacción con los de sus vecinas [3, 4], y se han convertido en una de las familias de modelos más estudiadas para la detección de fraude financiero [5].

En un entorno regulado no basta con que el modelo acierte. El analista que revisa una alerta necesita saber por qué la transacción fue marcada como sospechosa, y esa justificación debe poder sostenerse ante un auditor [6, 7]. Las GNN son modelos opacos, por lo que la justificación se obtiene con métodos de explicabilidad post hoc que asignan una importancia a cada variable o a cada arista que intervino en la predicción [8]. Esos métodos, sin embargo, son en su mayoría estocásticos u optimizan una máscara a partir de una inicialización aleatoria. Si se ejecutan dos veces sobre la misma transacción y el mismo modelo, pueden devolver explicaciones distintas. Una explicación que cambia cada vez que se pide no sirve como evidencia.

A esa fuente de variación se suma otra, propia del dominio. Las transacciones ilícitas son una fracción muy pequeña del total. En el conjunto de datos Elliptic, que es la referencia pública más usada para este problema [9], hay 4.545 transacciones etiquetadas como ilícitas frente a más de 199.000 lícitas o sin etiqueta. Bajo un desbalance así, los modelos aprenden la clase rara a partir de pocos ejemplos [10, 11], y es razonable esperar que sus explicaciones sobre esa clase también sean más frágiles. Las técnicas que compensan el desbalance, como reponderar la función de pérdida o generar ejemplos sintéticos de la clase rara [12, 13], modifican lo que el modelo aprende y podrían modificar también lo que el explicador reporta.

La literatura ha evaluado la calidad de los explicadores de GNN principalmente en grafos sintéticos o en tareas de química y redes sociales [14, 15], y la evaluación de la estabilidad de las explicaciones se ha estudiado sobre todo fuera de los grafos [16, 17]. No se encontró un estudio que mida, con un diseño factorial y réplicas independientes, cómo cambian la estabilidad de las explicaciones de distintas GNN cuando se varían el nivel de desbalance y la técnica que lo compensa, en un problema real de lavado de dinero. Ese es el vacío que aborda esta tesis.

1.2. Una limitación del conjunto de datos que condiciona el diseño

Elliptic cubre 49 pasos de tiempo separados aproximadamente dos semanas entre sí. En el paso 43 se cerró un mercado ilegal importante de la red, y los autores del conjunto de datos reportaron que, a partir de ese momento, ningún modelo era capaz de detectar las nuevas transacciones ilícitas [9]. La partición temporal habitual entrena con los pasos 1 a 34, valida con los pasos 35 a 42 y evalúa con los pasos 43 a 49, de modo que la prueba cae completa después del cierre. En la primera versión de los experimentos de esta tesis, todos los modelos quedaron en el nivel del azar en esa prueba (Anexo A).

Por esa razón, el estudio principal usa solo los pasos 1 a 42, anteriores al cierre, y los reparte en entrenamiento (1 a 29), validación (30 a 35) y prueba (36 a 42). Los pasos 43 a 49 no se usan. La decisión, sus costos y la comparación con la partición habitual se exponen en el Capítulo 5.

1.3. Preguntas de investigación

Pregunta principal. ¿Cómo cambia la estabilidad de las explicaciones que producen los métodos de explicabilidad sobre redes neuronales de grafos entrenadas para detectar transacciones ilícitas en Elliptic, cuando se varía el nivel de desbalance de clases, la arquitectura de la red y la técnica empleada para compensar el desbalance?

Preguntas específicas.

1. ¿Disminuye la estabilidad de las explicaciones a medida que el desbalance de clases con el que se entrena el modelo se hace más extremo?
2. ¿Producen las distintas arquitecturas (GCN, GraphSAGE, GAT y TAGCN) explicaciones con distinta estabilidad?
3. ¿Cambia la estabilidad de las explicaciones según la técnica usada para compensar el desbalance (sin ajuste, pesos por clase o *focal loss*)?

4. ¿Refleja la estabilidad medida lo que el modelo aprendió, o se debe principalmente a los datos de entrada y al propio explicador?

1.4. Hipótesis

Las hipótesis se formularon antes de observar los resultados y se contrastan en el Capítulo 6.

H1 (desbalance). La estabilidad de las explicaciones disminuye a medida que el desbalance de clases del entrenamiento se hace más extremo, porque el modelo dispone de menos ejemplos de la clase ilícita de los cuales aprender.

H2 (arquitectura). La estabilidad de las explicaciones depende de la arquitectura, porque cada una combina de forma distinta la información de la transacción con la de sus vecinas.

H3 (compensación). La técnica de compensación del desbalance no cambia la estabilidad de las explicaciones, porque modifica cuánto pesa cada clase en la función de pérdida pero no la información disponible para el modelo.

1.5. Objetivos

1.5.1. Objetivo general

Evaluar la estabilidad de los métodos de explicabilidad aplicados a redes neuronales de grafos entrenadas para detectar lavado de dinero en el conjunto de datos Elliptic, bajo distintos niveles de desbalance de clases, arquitecturas y técnicas de compensación, con el fin de establecer qué factores la determinan y en qué medida sus explicaciones son confiables para una auditoría.

1.5.2. Objetivos específicos

1. Medir el efecto del nivel de desbalance de clases, desde 1:10 hasta 1:200, sobre la estabilidad de las explicaciones de cinco explicadores: GNNExplainer, PGExplainer, valores de Shapley por muestreo, Integrated Gradients y Expected Gradients.

2. Comparar la estabilidad de las explicaciones entre cuatro arquitecturas de GNN: GCN, GraphSAGE, GAT y TAGCN.

3. Analizar el efecto de tres técnicas de compensación del desbalance (sin ajuste, pesos por clase y *focal loss*) y del sobremuestreo con SMOTE sobre el rendimiento predictivo y sobre la estabilidad de las explicaciones.

4. Determinar, mediante un control con pesos aleatorios y medidas de acuerdo que no dependen de las variables nulas, qué parte de la estabilidad medida proviene del modelo entrenado y qué parte de los datos de entrada y del explicador.

Respecto de la propuesta aprobada, dos métodos se sustituyeron por alternativas que se ajustan mejor al problema. GNNShap se reemplazó por valores de Shapley estimados por muestreo sobre las variables de la transacción [18], porque la pregunta de la tesis es qué variables pesan y GNNShap atribuye importancia a las aristas. GraphSMOTE se reemplazó por SMOTE sobre los atributos [12], porque la estructura del grafo de Elliptic no se modifica en ningún escenario. Integrated Gradients [19] y Expected Gradients [20] se añadieron como explicadores de referencia, uno determinista y uno estocástico, que no requieren entrenamiento.

1.6. Justificación

Operativa. Los sistemas de monitoreo basados en reglas producen una proporción muy alta de falsas alarmas, que en algunos reportes supera el 90 % [21], y los especialistas del sector identifican ese costo como uno de los problemas centrales del monitoreo actual [22]. Los modelos de aprendizaje automático, y en particular los que aprovechan la estructura de las transacciones, son una vía para reducirlo [23, 5]. Pero cada alerta de un modelo tiene que ser revisada por una persona, y esa revisión es más rápida y más defendible si la alerta viene con una explicación en la que se pueda confiar.

Regulatoria. La normativa exige a las entidades financieras un monitoreo continuo capaz de detectar transacciones sospechosas, y la introducción de modelos opacos en ese monitoreo plantea problemas de transparencia y de derechos que la regulación ha comenzado a abordar [24]. Los marcos propuestos para medir el riesgo de la inteligencia artificial en finanzas incluyen la explicabilidad entre sus principios medibles [25]. Para que una explicación cumpla esa función debe ser reproducible; esta tesis mide en qué medida lo es.

Científica. Los estudios sobre el efecto del desbalance en las explicaciones se han hecho con datos tabulares y han encontrado que el desbalance las vuelve menos estables [26]. No se sabe si eso ocurre con explicadores de GNN sobre datos de transacciones, ni qué parte de la estabilidad que se mide refleja lo que el modelo aprendió. Responder esas preguntas aporta tanto al conocimiento sobre las explicaciones de GNN como a la metodología con que se evalúan.

Social. Las criptomonedas se han usado para financiar actividades ilegales a gran escala [27, 28]. Mejorar la capacidad de detectar esos flujos con herramientas que puedan auditarse contribuye a la integridad del sistema financiero y a un uso responsable de la inteligencia artificial.

1.7. Contribuciones

1. Un diseño factorial replicado (4 arquitecturas, 6 escenarios de desbalance, 3 técnicas de compensación y 3 semillas de modelo, 216 modelos) que mide la estabilidad de cinco explicadores sobre las mismas 30 transacciones ilícitas.

2. La evidencia de que la partición temporal habitual de Elliptic deja la prueba completa después del cierre del mercado ilegal y vuelve inútil la evaluación en prueba, junto con una partición alternativa que la recupera.

3. La identificación de un artefacto de medida: cerca de la mitad de las variables de una transacción valen cero después del escalado, y los explicadores las dejan siempre al final del orden de importancia, lo que infla la correlación de Spearman entre explicaciones. La tesis cuantifica ese efecto con dos medidas que no dependen de esas variables y con un control de pesos aleatorios [29].

1.8. Alcance y limitaciones

El estudio se limita al conjunto de datos Elliptic y a la tarea de clasificar transacciones como ilícitas o no ilícitas. Las transacciones sin etiqueta se tratan como no ilícitas, que es el supuesto más cercano a la operación real (Capítulo 5). La estabilidad se mide sobre explicaciones de variables, salvo en PGExplainer, que solo explica aristas. No se evalúa la plausibilidad de las explicaciones frente a un patrón real de lavado, porque Elliptic no publica el significado de sus variables ni anotaciones de tipologías; un grafo sintético construido con ese fin se resume en el Anexo B. No se contempla el despliegue del sistema ni su evaluación con analistas.

1.9. Estructura del documento

El Capítulo 2 sitúa el problema: el lavado de dinero, su relación con las criptomonedas, los sistemas de monitoreo y las exigencias regulatorias. El Capítulo 3 presenta los fundamentos de las GNN, del desbalance de clases, de las métricas, de los métodos de explicabilidad y de las pruebas estadísticas. El Capítulo 4 revisa los trabajos sobre detección de lavado con grafos y sobre la evaluación de explicaciones. El Capítulo 5 describe los datos, la partición temporal, los escenarios, el entrenamiento, los explicadores, las medidas de estabilidad y las pruebas estadísticas. El Capítulo 6 presenta los resultados y contrasta las hipótesis. El Capítulo 7 interpreta los resultados y sus límites, y el Capítulo 8 resume las respuestas a las preguntas de investigación. Los anexos recogen la corrida con la partición original, el eje sintético, los detalles de reproducibilidad y las tablas completas.

Capítulo 2

Marco contextual

Este capítulo sitúa el problema técnico de la tesis en su contexto: qué es el lavado de dinero y cómo se organiza (Sección 2.1), cómo lo transforman las criptomonedas (Sección 2.2), cómo lo detectan hoy las instituciones financieras y por qué esos sistemas son insuficientes (Sección 2.3), y qué exige la regulación a los sistemas basados en inteligencia artificial (Sección 2.4). Cierra con la descripción del conjunto de datos Elliptic como representación de ese problema (Sección 2.5).

2.1. El lavado de dinero

El lavado de dinero es el conjunto de operaciones con las que se oculta el origen ilícito de unos fondos para que puedan usarse como si fueran legítimos. Es el paso que permite a una actividad criminal convertir sus ganancias en patrimonio aprovechable, y por eso combatirlo es una forma de combatir los delitos que lo originan, desde el narcotráfico hasta la corrupción y el financiamiento del terrorismo [1, 2].

2.1.1. Las tres etapas

La descripción más extendida del proceso distingue tres etapas [30, 1]:

1. **Colocación.** Los fondos de origen ilícito entran por primera vez al sistema financiero, por ejemplo mediante depósitos fraccionados en efectivo, compras de instrumentos financieros o, en el caso de las criptomonedas, la compra de activos virtuales en plataformas de intercambio.

2. **Estratificación.** Los fondos se mueven a través de muchas operaciones, cuentas e intermediarios con el fin de separar el dinero de su origen y dificultar su rastreo. Es la etapa en la que se generan las cadenas de transacciones que un análisis de grafos puede seguir.

3. **Integración.** Los fondos vuelven a la economía como ingresos aparentemente legítimos, por ejemplo a través de inversiones, compras de bienes o empresas.

Irwin et al. analizaron 184 tipologías documentadas por organismos de prevención del lavado y del financiamiento del terrorismo, y encontraron que quienes lavan dinero y quienes financian el terrorismo prefieren técnicas algo distintas en cada etapa, y que cuantas más técnicas se combinan, más dinero se logra ocultar [30]. Esa combinación de técnicas es la que vuelve el problema difícil para un sistema de reglas: el patrón no está en una operación aislada sino en la secuencia y en la red de operaciones.

2.1.2. La escala del problema

Medir el lavado es difícil precisamente porque está diseñado para no ser visto. Las estimaciones disponibles para las criptomonedas dan una idea de su magnitud. Foley et al. estimaron, con técnicas de análisis de la cadena de bloques, que cerca de una cuarta parte de los usuarios de Bitcoin participaba en actividades ilegales, y que esas actividades movían unos 76.000 millones de dólares al año, el 46 % de las transacciones de Bitcoin, una cifra comparable a los mercados de drogas ilegales de Estados Unidos y Europa [27]. Los mismos autores observaron que la proporción ilegal disminuía a medida que crecía el interés general en Bitcoin y aparecían criptomonedas más opacas. En otro frente, Paquet-Clouston et al. reconstruyeron los pagos de 35 familias de *ransomware* en Bitcoin y estimaron un mercado de al menos 12,7 millones de dólares entre 2013 y mediados de 2017, concentrado en unos pocos actores [28].

2.2. Criptomonedas y lavado de dinero

2.2.1. Qué cambia con las criptomonedas

Las criptomonedas permiten transferir valor de persona a persona sin un intermediario financiero, y la cadena de bloques asocia las transacciones a direcciones y no a identidades. Albrecht et al. señalan que esas dos propiedades las volvieron atractivas para los mercados de la red oscura y otras redes criminales, y analizan cómo se incorporan a cada etapa del lavado [31]. Otros trabajos describen técnicas propias de este medio, como la mezcla de fondos de muchos usuarios para romper el rastro entre direcciones, el encadenamiento de transacciones y el paso por plataformas con controles débiles [32, 33].

El punto débil de quien lava con criptomonedas es la conversión a dinero tradicional, porque ahí reaparece un intermediario regulado. Dupuis y Gleason identificaron seis «puertas abiertas» para convertir criptomonedas en moneda tradicional que seguían disponibles para quien busca

lavar fondos, y las analizaron con el paradigma de la dialéctica regulatoria: cada regulación cierra una puerta y los infractores buscan la siguiente [34].

2.2.2. Seudonimato y trazabilidad

La cadena de bloques de Bitcoin es pública: cualquiera puede ver todas las transacciones, sus montos y las direcciones que intervienen. Lo que no es público es quién controla cada dirección. Esta combinación de transparencia y seudonimato es la que hace posible el análisis forense de la cadena de bloques y, a la vez, la que lo limita [35]. Un investigador puede seguir los fondos de transacción en transacción, pero necesita información externa, como la de una plataforma de intercambio, para atribuir una dirección a una persona. Por eso el problema se plantea de forma natural como el análisis de un grafo de transacciones, en el que lo que se observa son flujos y no identidades [9].

2.3. Sistemas de monitoreo de transacciones

2.3.1. Monitoreo basado en reglas

Las instituciones financieras están obligadas a monitorear las transacciones de sus clientes y reportar las sospechosas. La mayoría lo hace con sistemas de reglas: umbrales de monto, frecuencias inusuales, contrapartes en listas de riesgo o combinaciones de estos criterios. El problema conocido de estos sistemas es la proporción de falsas alarmas. Ketenci et al. reportan que la mayoría de los sistemas en uso son basados en reglas y que más del 90 % de sus alertas son falsos positivos [21]. Oztas et al. entrevistaron a especialistas en prevención del lavado, que identificaron esa tasa de falsos positivos, con sus costos e ineficiencias, como uno de los problemas centrales del monitoreo actual, y señalaron el aprendizaje automático, el análisis de grafos y la detección de anomalías como vías para superarlo [22].

2.3.2. Monitoreo con aprendizaje automático

El aprendizaje automático se ha propuesto como complemento de las reglas para reducir las falsas alarmas [1, 23]. Las revisiones distinguen enfoques supervisados, que aprenden de casos etiquetados y suelen detectar mejor pero necesitan muchas etiquetas, y no supervisados, que detectan patrones anómalos nuevos pero producen más falsas alarmas [23]. Labanca et al. combinan ambos en un esquema de aprendizaje activo en el que el modelo elige qué transacciones conviene que el analista revise, para aprovechar mejor su retroalimentación [36]. En todos los casos la escasez de ejemplos positivos es una constante: los casos confirmados de lavado son

muy pocos frente al volumen de operaciones legítimas, lo que convierte el desbalance de clases en una característica estructural del problema y no en un accidente de los datos.

Los entrevistados por Oztas et al. incluyeron la explicabilidad entre los requisitos para que un sistema de monitoreo basado en aprendizaje automático se adopte en la práctica [22]. El analista que recibe una alerta tiene que decidir si la reporta, y para hacerlo necesita saber qué la motivó.

2.4. Regulación e inteligencia artificial explicable

El uso de modelos opacos en decisiones con consecuencias para las personas plantea exigencias de transparencia que la regulación ha empezado a recoger. Bertrand et al. examinan si los sistemas de prevención del lavado basados en inteligencia artificial respetan los derechos fundamentales europeos, y recuerdan que la normativa de la Unión Europea obliga a las entidades financieras a un monitoreo continuo capaz de detectar transacciones inusuales o sospechosas; la introducción de algoritmos de aprendizaje automático más opacos en ese monitoreo plantea riesgos de protección de datos que solo se han estudiado en parte [24].

En el sector financiero en general, Giudici et al. proponen un marco de indicadores para medir los riesgos de la inteligencia artificial, en el que traducen los requisitos regulatorios propuestos para la inteligencia artificial en cuatro principios medibles: sostenibilidad, exactitud, equidad y explicabilidad [25]. Una revisión sistemática de 35 casos de uso de explicabilidad en finanzas encontró que la detección de fraude es una de las aplicaciones más frecuentes, que SHAP y LIME son los métodos más usados, y que los beneficios buscados son la transparencia, la confianza de los interesados y el cumplimiento regulatorio [37].

Esta tesis parte de una observación que esa literatura suele dar por supuesta: para que una explicación sirva como evidencia ante un auditor, debe ser reproducible. Si el mismo método, aplicado a la misma transacción y al mismo modelo, produce explicaciones distintas, no puede sostener una decisión regulatoria [6]. La estabilidad de las explicaciones es, por eso, una condición previa a su uso en un entorno regulado.

2.5. El conjunto de datos Elliptic

Los datos reales de lavado de dinero rara vez son públicos, porque contienen información protegida de clientes y de investigaciones. Elliptic es una excepción: es un grafo de transacciones de Bitcoin publicado por la empresa del mismo nombre junto con investigadores del MIT e IBM, con una parte de las transacciones etiquetadas como lícitas o ilícitas según su asociación con

entidades conocidas [9]. Las ilícitas corresponden a categorías como estafas, *malware*, organizaciones terroristas, *ransomware* y esquemas Ponzi; las lícitas, a plataformas de intercambio, proveedores de billeteras, mineros y servicios legales.

Elliptic se ha convertido en la referencia pública para la detección de transacciones ilícitas en criptomonedas [38, 39]. Tiene tres rasgos que lo hacen adecuado para esta tesis: es real, está desbalanceado de forma natural y su estructura de grafo permite aplicar GNN. Tiene también dos limitaciones que condicionan el diseño. La primera es que sus variables están anonimizadas, de modo que no se puede juzgar si una explicación señala algo con sentido para un experto; solo se puede medir si es estable. La segunda es el cierre de un mercado ilegal en el paso de tiempo 43, que cambia la distribución de las transacciones ilícitas posteriores y que ningún modelo logra capturar con las variables originales [9, 40]. El Capítulo 5 describe el conjunto de datos en detalle y la forma en que se trató esa segunda limitación.

Capítulo 3

Marco teórico

Este capítulo presenta los fundamentos técnicos que usa la tesis: el aprendizaje sobre grafos y las cuatro arquitecturas estudiadas (Secciones 3.1 a 3.3), el desbalance de clases y las métricas de rendimiento (Secciones 3.4 y 3.5), los métodos de explicabilidad (Sección 3.6) y las herramientas con que se mide y se contrasta la estabilidad de las explicaciones (Secciones 3.8 y 3.9).

3.1. Del aprendizaje automático al aprendizaje sobre grafos

Un clasificador supervisado aprende una función $f : \mathbb{R}^d \to [0,1]$ que, a partir de las d variables de un ejemplo, estima la probabilidad de que pertenezca a la clase positiva. Los modelos clásicos, como la regresión logística o los bosques aleatorios, tratan cada ejemplo de forma independiente. Las redes neuronales profundas componen muchas transformaciones no lineales con parámetros que se ajustan por descenso de gradiente, y aprenden representaciones intermedias de los datos en lugar de depender de variables diseñadas a mano [41].

En muchos problemas los ejemplos no son independientes: están conectados. Una transacción de Bitcoin recibe fondos de transacciones anteriores y los envía a otras posteriores, y lo que se sabe de esas transacciones vecinas es información sobre ella. Los datos de este tipo se representan como un grafo $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, con un conjunto de nodos $\mathcal{V}$ y un conjunto de aristas $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$. El aprendizaje profundo geométrico extiende las redes neuronales a dominios de este tipo, que no tienen la estructura regular de una imagen o una secuencia [42]. En la tarea de esta tesis, la clasificación de nodos, se conocen las etiquetas de una parte de los nodos y se quiere predecir la de los demás usando tanto sus variables como su posición en el grafo.

3.2. Redes neuronales de grafos

Sea un grafo con N nodos, matriz de atributos $\mathbf{X} \in \mathbb{R}^{N \times d}$ y conjunto de vecinos $\mathcal{N}(v)$ de cada nodo v. Una red neuronal de grafos calcula en cada capa ℓ una representación $\mathbf{h}_v^{(\ell)}$ de cada nodo combinando su propia representación con las de sus vecinos [3, 4, 43]:

$$\mathbf{h}_v^{(\ell+1)} = \phi\left(\mathbf{h}_v^{(\ell)}, \bigoplus_{u \in \mathcal{N}(v)} \psi\left(\mathbf{h}_v^{(\ell)}, \mathbf{h}_u^{(\ell)}\right)\right), \qquad \mathbf{h}_v^{(0)} = \mathbf{x}_v, \tag{3.1}$$

donde $\bigoplus$ es una agregación invariante al orden (suma, media o máximo) y ϕ y ψ son funciones con parámetros aprendidos. Tras L capas, la representación de un nodo depende solo de los nodos a L saltos o menos, su subgrafo receptivo. La última capa produce la probabilidad de cada clase. Las cuatro arquitecturas del estudio difieren en cómo combinan el nodo con sus vecinos.

GCN [44]. Promedia al nodo con sus vecinos usando la matriz de adyacencia normalizada por el grado, con un único conjunto de pesos:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\mathbf{W}^{(\ell)} \sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{\mathbf{h}_u^{(\ell)}}{\sqrt{(d_u+1)(d_v+1)}}\right). \tag{3.2}$$

El nodo entra en la suma como un vecino más y no tiene un peso propio.

GraphSAGE [45]. Separa la representación del nodo de la de sus vecinos con dos matrices de pesos:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\mathbf{W}_1^{(\ell)} \mathbf{h}_v^{(\ell)} + \mathbf{W}_2^{(\ell)} \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(\ell)}\right). \tag{3.3}$$

GAT [46]. Pondera a cada vecino, incluido el propio nodo, con un coeficiente de atención aprendido:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \alpha_{vu} \mathbf{W}^{(\ell)} \mathbf{h}_u^{(\ell)}\right), \qquad \alpha_{vu} = \mathrm{softmax}_u\left(\mathrm{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \,\|\, \mathbf{W}\mathbf{h}_u]\right)\right), \tag{3.4}$$

y repite el cálculo con varias cabezas de atención independientes cuyos resultados se concatenan.

TAGCN [47]. Aplica un filtro polinomial de orden K sobre la matriz de adyacencia normalizada $\tilde{\mathbf{A}}$, con una matriz de pesos por cada potencia:

$$\mathbf{H}^{(\ell+1)} = \sigma\left(\sum_{k=0}^{K} \tilde{\mathbf{A}}^k \mathbf{H}^{(\ell)} \mathbf{W}_k^{(\ell)}\right). \tag{3.5}$$

El término $k = 0$ da al nodo un peso propio, y cada capa alcanza vecinos a K saltos.

Comparación. Las cuatro arquitecturas difieren en dos aspectos que importan para este problema. El primero es si la transacción tiene un peso propio, separado del de sus vecinas: GraphSAGE lo tiene por construcción, GAT puede asignarse a sí misma un peso de atención alto y TAGCN lo tiene en el término $k = 0$; GCN, en cambio, mezcla a la transacción con sus vecinas con un peso que depende solo de los grados. El segundo es el alcance: con L capas, GCN, GraphSAGE y GAT ven L saltos, mientras que TAGCN ve $L \cdot K$ saltos.

3.3. Propiedades de los grafos que condicionan el aprendizaje

Homofilia y heterofilia. La mayoría de las GNN suponen, de forma implícita, que los nodos conectados tienden a pertenecer a la misma clase, una propiedad llamada homofilia. Cuando los nodos de clases distintas tienden a conectarse entre sí, situación llamada heterofilia, las GNN diseñadas bajo el supuesto contrario pierden rendimiento, porque promediar con las vecinas mezcla la señal del nodo con la de la otra clase [48]. Una medida simple es la homofilia de aristas, la fracción de aristas cuyos dos extremos tienen la misma clase:

$$h = \frac{|\{(u,v) \in \mathscr{E} : y_u = y_v\}|}{|\mathscr{E}|}. \tag{3.6}$$

Para la clase rara conviene mirar además la composición de las vecinas de sus nodos: si la mayoría de las vecinas de una transacción ilícita no son ilícitas, la información útil está sobre todo en la propia transacción.

Sobresuavizado. Cada capa de una GNN promedia la representación de un nodo con las de sus vecinos. Al apilar muchas capas, las representaciones de nodos distintos se vuelven cada vez más parecidas y el rendimiento cae, un fenómeno llamado sobresuavizado [49]. Por eso las GNN que se usan en la práctica suelen tener pocas capas, entre dos y tres, como en esta tesis.

Campo receptivo. Una consecuencia útil de la estructura por capas es que la predicción de un nodo en una GNN de L capas solo depende de su vecindario a L saltos. Ese subgrafo receptivo contiene toda la información que el modelo usa para esa predicción, de modo que una explicación puede calcularse sobre él sin perder nada.

Dispersión. En grafos de transacciones la mayoría de los nodos tiene muy pocas aristas. Un grafo disperso limita lo que la agregación de vecinas puede aportar y, como se verá, también lo que un explicador de aristas puede decir.

3.4. Desbalance de clases

Cuando una clase es mucho más frecuente que otra, la pérdida promedio está dominada por la clase mayoritaria y el modelo puede minimizarla prediciendo casi siempre esa clase [10, 11]. Las estrategias para compensarlo actúan sobre los datos o sobre la pérdida.

Sobre los datos, se puede submuestrear la clase mayoritaria o sobremuestrear la minoritaria. SMOTE [12] genera ejemplos sintéticos de la clase minoritaria interpolando entre un ejemplo y uno de sus vecinos más cercanos de la misma clase: $\tilde{\mathbf{x}} = \mathbf{x}_i + \lambda(\mathbf{x}_j - \mathbf{x}_i)$, con $\lambda \sim \mathscr{U}(0,1)$. En un grafo, el ejemplo sintético necesita además una posición en la estructura; una opción sencilla es que herede las aristas del ejemplo a partir del cual se generó. Las revisiones sobre clasificación de nodos desbalanceada agrupan los métodos en los que actúan sobre los datos, como el sobremuestreo y el submuestreo, y los que actúan sobre el algoritmo, como la reponderación de la pérdida [50].

Sobre la pérdida, sea p_i la probabilidad que el modelo asigna a la clase verdadera del ejemplo i. La entropía cruzada sin ajuste es $\mathscr{L} = -\frac{1}{n}\sum_i \log p_i$. Con pesos por clase, cada ejemplo se pondera por el inverso de la frecuencia de su clase, $w_c = n/(2n_c)$, de modo que

$$\mathscr{L}_{\mathrm{PC}} = \frac{1}{2}\frac{1}{n_+}\sum_{i:\,y_i=1}(-\log p_i) + \frac{1}{2}\frac{1}{n_-}\sum_{i:\,y_i=0}(-\log p_i), \tag{3.7}$$

y cada clase aporta la mitad de la pérdida sin importar cuántos ejemplos tenga. La *focal loss* [13] reduce el peso de los ejemplos que el modelo ya clasifica bien:

$$\mathscr{L}_{\mathrm{FL}} = -\frac{1}{n}\sum_i \alpha_{y_i}(1-p_i)^{\gamma}\log p_i, \tag{3.8}$$

donde γ controla cuánto se atenúan los ejemplos fáciles y α_{y_i} pondera la clase. Con $\alpha = 0{,}75$ para la clase rara y 0,25 para la frecuente, un ejemplo ilícito pesa tres veces lo que uno negativo, mientras que con pesos por clase y una razón de 41 a 1 pesa 41 veces más.

3.5. Métricas de rendimiento bajo desbalance

Sea un umbral τ sobre la probabilidad estimada: una transacción se clasifica como ilícita si $f(\mathbf{x}) \geq \tau$. Con los verdaderos positivos (VP), falsos positivos (FP), verdaderos negativos (VN) y falsos negativos (FN) se definen la precisión, la exhaustividad (*recall*) y la tasa de falsos positivos:

$$P = \frac{\mathrm{VP}}{\mathrm{VP}+\mathrm{FP}}, \qquad R = \frac{\mathrm{VP}}{\mathrm{VP}+\mathrm{FN}}, \qquad \mathrm{TFP} = \frac{\mathrm{FP}}{\mathrm{FP}+\mathrm{VN}}, \tag{3.9}$$

y a partir de ellas el F1 y el coeficiente de correlación de Matthews:

$$F_1 = \frac{2PR}{P+R}, \qquad \mathrm{MCC} = \frac{\mathrm{VP}\cdot\mathrm{VN} - \mathrm{FP}\cdot\mathrm{FN}}{\sqrt{(\mathrm{VP}+\mathrm{FP})(\mathrm{VP}+\mathrm{FN})(\mathrm{VN}+\mathrm{FP})(\mathrm{VN}+\mathrm{FN})}}. \tag{3.10}$$

El MCC usa las cuatro celdas de la matriz de confusión y solo es alto si el clasificador acierta en las dos clases, por lo que es más informativo que el F1 y la exactitud bajo desbalance [51].

Las métricas anteriores dependen del umbral. Las curvas resumen todos los umbrales a la vez. La curva ROC grafica R frente a TFP, y la curva de precisión y exhaustividad grafica P frente a R. Sus áreas, ROC-AUC y PR-AUC, valen 1 para un clasificador perfecto. Bajo un clasificador al azar, el ROC-AUC vale 0,5 y el PR-AUC vale la prevalencia π de la clase positiva. La diferencia importa bajo desbalance: como la TFP se divide por el número enorme de negativos, un modelo puede tener una TFP de 0,05 y un ROC-AUC alto mientras produce muchas más falsas alarmas que aciertos; la precisión sí refleja ese costo [52]. Por eso esta tesis usa el PR-AUC como métrica principal y lo compara siempre con la prevalencia.

3.6. Explicabilidad de modelos

La explicabilidad busca que las decisiones de un modelo puedan ser comprendidas por una persona [53, 54, 7]. Los métodos post hoc explican un modelo ya entrenado sin modificarlo, y los locales explican una predicción individual asignando una importancia a cada elemento de la entrada. En grafos, esos elementos pueden ser las variables de los nodos, las aristas o subgrafos completos [8, 55]. Esta tesis estudia explicaciones de variables: para una transacción, cuál de sus 165 variables pesó más en la predicción.

GNNExplainer [56]. Busca una máscara $\mathbf{M}$ sobre las variables (y opcionalmente sobre las aristas) que preserve la predicción del modelo y sea a la vez pequeña:

$$\min_{\mathbf{M}} \; -\log P(\hat{y} \mid \mathbf{X} \odot \sigma(\mathbf{M})) + \lambda_1 \|\sigma(\mathbf{M})\|_1 + \lambda_2 H(\sigma(\mathbf{M})), \tag{3.11}$$

donde H es la entropía de la máscara, que la empuja hacia valores cercanos a 0 o 1. La máscara se inicializa al azar y se optimiza por descenso de gradiente, por lo que dos ejecuciones pueden terminar en máscaras distintas.

PGExplainer [57]. Entrena una red que, a partir de las representaciones de los dos extremos de una arista, predice su importancia. Una vez entrenada, explica cualquier nodo sin optimizar de nuevo. Atribuye importancia a aristas, no a variables.

Valores de Shapley [58, 18]. Tratan a las variables como jugadores de un juego cooperativo cuyo pago es la predicción. La importancia de la variable j es su contribución marginal

promedio sobre todos los órdenes posibles en que las variables se incorporan:

$$\varphi_j = \frac{1}{|\Pi|} \sum_{\pi \in \Pi} \left[f\left(S_\pi^j \cup \{j\}\right) - f\left(S_\pi^j\right) \right], \tag{3.12}$$

donde S_π^j es el conjunto de variables que preceden a j en el orden π y las variables ausentes toman un valor de referencia. Como hay $d!$ órdenes, se estima con una muestra aleatoria de permutaciones [59], y por ello es estocástico.

Integrated Gradients [19]. Integra el gradiente de la salida a lo largo del camino recto entre una referencia $\mathbf{x}'$ y la entrada $\mathbf{x}$:

$$\mathrm{IG}_j(\mathbf{x}) = (x_j - x'_j) \int_0^1 \frac{\partial f(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_j} \, d\alpha. \tag{3.13}$$

La integral se aproxima con una suma de Riemann. Con una referencia fija, el método es determinista. Si la referencia es el vector cero y una variable vale cero, su atribución es exactamente cero.

Expected Gradients [20]. Sustituye la referencia fija por una distribución de referencias D tomadas de los datos y promedia: $\mathrm{EG}_j(\mathbf{x}) = \mathbb{E}_{\mathbf{x}' \sim D,\, \alpha \sim \mathscr{U}(0,1)} \left[(x_j - x'_j)\, \partial_j f(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}')) \right]$. La esperanza se estima por muestreo, por lo que es estocástico.

3.7. Evaluación de las explicaciones

No existe una única medida de la calidad de una explicación. Las revisiones sistemáticas distinguen varias propiedades [16, 17]: la **fidelidad**, si la explicación refleja lo que de verdad usa el modelo; la **plausibilidad**, si coincide con lo que un experto considera relevante; y la **estabilidad** o consistencia, si explicaciones de la misma predicción en condiciones equivalentes se parecen entre sí. En grafos, las evaluaciones sistemáticas han mostrado que los explicadores difieren mucho entre sí y que su desempeño depende del conjunto de datos y de la arquitectura [14, 15].

Cuando se conoce la explicación correcta, como en un grafo sintético construido con ese fin, la plausibilidad puede medirse como la coincidencia entre lo que señala el explicador y lo que de verdad define la clase, por ejemplo la fracción de las k variables principales que pertenecen al conjunto verdadero. La fidelidad no necesita esa referencia: se mide cambiando la entrada según la explicación y observando la predicción. Si f es la probabilidad de la clase predicha, S el conjunto de elementos señalados (variables o aristas, según lo que explique el método), $\mathbf{x}_{\setminus S}$ la entrada sin esos elementos y $\mathbf{x}_S$ la entrada con solo ellos,

$$\mathrm{fid}^+ = f(\mathbf{x}) - f(\mathbf{x}_{\setminus S}), \qquad \mathrm{fid}^- = f(\mathbf{x}) - f(\mathbf{x}_S). \tag{3.14}$$

Una explicación fiel tiene un fid^+ alto, porque quitar lo señalado cambia la predicción, y un fid^- bajo, porque lo señalado basta para sostenerla. Ambas se comparan con las que daría una explicación al azar del mismo tamaño. Esta tesis usa estas medidas solo en el eje sintético (Anexo B), porque en Elliptic no hay una explicación de referencia y retirar variables de una transacción produce entradas fuera de la distribución de los datos.

La estabilidad es una condición necesaria para usar una explicación como evidencia: si dos ejecuciones del mismo explicador sobre el mismo modelo y la misma transacción dan órdenes de importancia distintos, el analista no sabe a cuál creer. Pero no es suficiente. Un explicador que ignorara el modelo y devolviera siempre el mismo orden sería perfectamente estable e inútil. Adebayo et al. [29] propusieron por eso comparar la explicación de un modelo entrenado con la de la misma arquitectura con pesos aleatorios: si se parecen, la explicación no depende de lo aprendido.

Para medir el acuerdo entre dos explicaciones se usan comparaciones de rangos o de conjuntos. La correlación de Spearman [60] compara los órdenes completos de importancia, y el índice de Jaccard [61] compara los conjuntos de las k variables más importantes. También se han propuesto métricas diseñadas específicamente para la estabilidad de las explicaciones [62].

3.8. Medidas de acuerdo entre explicaciones

Sean $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$ dos vectores de importancia sobre las mismas d variables, y $r(\mathbf{a})$ el vector de rangos de $\mathbf{a}$, con rangos medios en caso de empate.

Correlación de Spearman. Es la correlación de Pearson entre los rangos [60]:

$$\rho(\mathbf{a}, \mathbf{b}) = \frac{\operatorname{cov}\left(r(\mathbf{a}), r(\mathbf{b})\right)}{\sigma_{r(\mathbf{a})}\, \sigma_{r(\mathbf{b})}}. \tag{3.15}$$

Vale 1 si los dos órdenes coinciden, 0 si no tienen relación y -1 si están invertidos. Mira el orden completo, incluidas las variables menos importantes. Esto tiene una consecuencia que la tesis examina en detalle: si muchas variables reciben importancia cero en ambas explicaciones, empatan en los últimos rangos y aportan acuerdo aunque no digan nada sobre la predicción.

Índice de Jaccard de las k principales. Sea $T_k(\mathbf{a})$ el conjunto de las k variables de mayor importancia. El índice [61]

$$J_k(\mathbf{a}, \mathbf{b}) = \frac{|T_k(\mathbf{a}) \cap T_k(\mathbf{b})|}{|T_k(\mathbf{a}) \cup T_k(\mathbf{b})|} \tag{3.16}$$

vale 1 si las dos explicaciones señalan las mismas k variables y 0 si no comparten ninguna. Si comparten m variables, $J_k = m/(2k - m)$. Solo mira lo que un analista vería primero, y no depende del orden dentro del conjunto ni de las variables poco importantes.

Las dos medidas se complementan. El Spearman es sensible a todo el orden y comparable con la literatura; el Jaccard se acerca más a cómo se usa una explicación en la práctica.

3.9. Pruebas estadísticas

Prueba de Friedman. Compara k tratamientos medidos sobre los mismos n bloques sin suponer normalidad [63]. Dentro de cada bloque se ordenan los k valores y se calcula el rango medio $\bar{R}_j$ de cada tratamiento. El estadístico

$$\chi_F^2 = \frac{12n}{k(k+1)} \sum_{j=1}^{k} \left(\bar{R}_j - \frac{k+1}{2} \right)^2 \tag{3.17}$$

sigue aproximadamente una distribución χ^2 con $k-1$ grados de libertad bajo la hipótesis de que los tratamientos son iguales. El coeficiente de concordancia de Kendall, $W = \chi_F^2/(n(k-1))$, mide entre 0 y 1 cuán consistente es el orden de los tratamientos entre bloques.

Prueba de signos. Para comparar dos condiciones sobre n pares, cuenta en cuántos pares la primera supera a la segunda. Bajo la hipótesis de igualdad ese número sigue una distribución binomial con probabilidad 0,5. Es menos potente que la prueba de Wilcoxon [64], pero no supone nada sobre la magnitud de las diferencias, lo que conviene con pocos pares.

Equivalencia (TOST). Una prueba de diferencias que no rechaza no demuestra que no haya efecto: puede que la muestra sea pequeña. La prueba de equivalencia de dos pruebas unilaterales invierte la pregunta: fija un margen δ por debajo del cual una diferencia se considera irrelevante y contrasta las hipótesis $H_{01} : \mu \leq -\delta$ y $H_{02} : \mu \geq \delta$. Si ambas se rechazan, la diferencia está dentro del margen [65].

Intervalos *bootstrap*. Remuestreando con reemplazo las unidades observadas y recalculando la media en cada remuestra se obtiene una aproximación a su distribución, de la que se toman los percentiles 2,5 y 97,5 como intervalo de confianza del 95 % [66].

Pseudorreplicación. Todas las pruebas anteriores suponen que las unidades son independientes. La pseudorreplicación consiste en tratar como independientes observaciones que no lo son, por ejemplo mediciones repetidas sobre el mismo sujeto, y conduce a contrastar el efecto con un término de error que no corresponde a la hipótesis [67]. Su consecuencia es inflar la significancia, y es un error frecuente aun en revistas de alto impacto [68]. En esta tesis la unidad que puede repetirse de forma independiente es el modelo entrenado con una semilla; los modelos de una misma semilla en distintos escenarios comparten su inicialización y sus hiperparámetros, y no son réplicas independientes.

Capítulo 4

Estado del arte

Este capítulo revisa tres líneas de trabajo que la tesis cruza: la detección de lavado de dinero con grafos, en particular sobre Elliptic (Sección 4.1); la explicabilidad aplicada a ese problema (Sección 4.2); y la evaluación de la estabilidad de las explicaciones, en especial bajo desbalance de clases (Sección 4.3). Al final se precisa el vacío que aborda el trabajo (Sección 4.5).

4.1. Detección de lavado de dinero con grafos

Las revisiones sobre aprendizaje automático para la prevención del lavado de dinero documentan el paso de sistemas basados en reglas a modelos supervisados y no supervisados [1], y, más recientemente, a modelos de aprendizaje profundo [2]. Una revisión sistemática dedicada a la detección de fraude financiero con GNN muestra que esta familia de modelos se ha extendido a tarjetas de crédito, seguros, transferencias bancarias y criptomonedas [5].

Elliptic se publicó junto con los primeros experimentos de GCN para este problema [9]. Sus autores compararon regresión logística, bosques aleatorios y GCN, y observaron que el desempeño en la prueba se desplomaba a partir del paso 43, tras el cierre de un mercado ilegal. Ese hallazgo es la base de la partición que usa esta tesis. Desde entonces el conjunto de datos se ha convertido en la referencia para la detección de transacciones ilícitas en Bitcoin. Sobre él se han propuesto modelos de *gradient boosting* [69]; modelos que combinan GCN con memorias de largo y corto plazo para aprovechar el orden temporal, junto con esquemas de aprendizaje activo [38]; representaciones de nodos aprendidas de forma autosupervisada [70]; y arquitecturas temporales basadas en transformadores sobre grafos [71]. Otras propuestas modelan el grafo de transacciones como una secuencia de instantáneas dinámicas [72, 73]. Mohan et al. modelan la detección en Bitcoin como clasificación de nodos en redes dinámicas, combinando convoluciones de grafos que evolucionan en el tiempo con bosques de decisión neuronales [39], y Asiri et al. comparan regresión logística, máquinas de soporte vectorial, bosques aleatorios, redes LSTM y GCN sobre Elliptic [74].

El cierre del mercado en el paso 43 es un problema reconocido. Alarab y Prakoonwit señalan que todos los algoritmos de aprendizaje han fallado en capturar ese evento con las variables originales de Elliptic, y proponen un modelo recurrente de grafos que hace predicciones secuenciales apoyándose en las transacciones etiquetadas más recientes [40]. Ese enfoque supone que se dispone de etiquetas recientes durante la operación, un supuesto distinto del de esta tesis, que separa estrictamente el entrenamiento de la evaluación. En el ámbito forense, Pocher et al. analizan cómo los métodos de aprendizaje automático y de análisis del grafo de transacciones se aplican a la prevención del lavado en criptomonedas y qué implicaciones tiene su uso por autoridades y empresas [35].

Dos observaciones de esta literatura son relevantes para la tesis. La primera es que la mayoría de los trabajos reporta el desempeño con la partición temporal habitual o con particiones aleatorias, y pocos discuten el efecto del cierre del mercado sobre la prueba. La segunda es que el desbalance de clases se trata como un obstáculo para el rendimiento. Lawal et al. examinan explícitamente las limitaciones de las GNN sobre Elliptic bajo ese desbalance, y muestran que un modelo puede lograr una exactitud alta en entrenamiento sin identificar de forma significativa las transacciones ilícitas, con y sin estrategias como los pesos por clase [75]. Ninguno de estos trabajos mide qué ocurre con las explicaciones cuando cambia el desbalance.

4.2. Explicabilidad en la detección de lavado de dinero

La necesidad de explicar las alertas de un sistema de detección se justifica tanto por la regulación como por la operación: el analista debe poder entender y documentar por qué revisa una transacción [6, 2]. Una revisión reciente reúne los trabajos sobre detección de lavado en criptomonedas que incorporan explicabilidad [76], y se han propuesto marcos de GNN explicables para detectar transacciones financieras ilícitas [77]. En la mayoría de estos trabajos la explicación se presenta como un resultado cualitativo, por ejemplo las variables o los vecinos más importantes para algunos casos, y no se evalúa si la explicación se mantiene al repetirla.

En el campo general de la explicabilidad de GNN, las taxonomías clasifican los métodos según lo que explican (variables, aristas o subgrafos) y cómo lo obtienen (gradientes, perturbaciones, sustitutos o descomposición) [8, 55]. Las evaluaciones comparativas han mostrado que los explicadores discrepan entre sí. Agarwal et al. construyeron un generador de grafos sintéticos con explicaciones de referencia, capaz de variar el tamaño, la distribución de grados y la homofilia del grafo, y una biblioteca para evaluar explicadores sobre esos grafos y sobre grafos reales [14]. Longa et al. probaron 12 explicadores sobre 8 arquitecturas de paso de mensajes y 6 conjuntos de datos, y analizaron por qué unas arquitecturas resultan más explicables que otras y qué explicador conviene en cada caso [15].

4.3. Estabilidad de las explicaciones y desbalance de clases

Las revisiones sobre la evaluación cuantitativa de la explicabilidad incluyen la estabilidad, o consistencia, entre las propiedades que una explicación debe cumplir, junto a la fidelidad y la plausibilidad [16, 17]. Se han propuesto métricas específicas para medirla [62]. Adebayo et al. mostraron que algunas explicaciones visualmente convincentes apenas cambian cuando se aleatorizan los pesos del modelo, y propusieron esa aleatorización como prueba de que una explicación depende de lo que el modelo aprendió [29].

La relación entre desbalance de clases y explicaciones se ha estudiado en datos tabulares. Rai et al. diseñaron un marco comparativo para evaluar la confiabilidad y la consistencia de LIME, SHAP y gráficos de dependencia parcial en la predicción de riesgo de enfermedades con historias clínicas electrónicas, bajo distintos grados de desbalance [78]. Fujiwara et al. proponen combinar destilación de conocimiento y remuestreo para mejorar tanto el rendimiento como la estabilidad de las explicaciones en clasificación desbalanceada [79].

El trabajo más cercano a la pregunta de esta tesis es el de Chen et al. sobre calificación crediticia. Partiendo de que la investigación sobre desbalance se había concentrado en su efecto sobre la exactitud y no sobre la interpretabilidad, evaluaron la estabilidad de LIME y SHAP sobre conjuntos de datos hipotecarios con desbalance creciente, y encontraron que las explicaciones se vuelven menos estables a medida que aumenta el desbalance, resultado que repitieron en otros dos conjuntos de datos de crédito [26]. Mammadov et al. propusieron un artefacto de diseño para evaluar de forma controlada el efecto del desbalance y del muestreo sobre la estabilidad de SHAP y LIME, con medidas de acuerdo de rangos y de variabilidad, sobre 1.600 simulaciones [80]. Estos trabajos muestran que el desbalance y las técnicas que lo compensan pueden afectar las explicaciones, y en el caso de Chen et al. ofrecen una hipótesis concreta, la de que más desbalance produce explicaciones menos estables, que esta tesis contrasta en un dominio distinto. Ninguno trata datos en grafo ni explicadores de GNN.

4.4. Síntesis comparativa

La Tabla 4.1 resume los trabajos más cercanos según cuatro rasgos: si usan grafos, si explican las predicciones, si miden la estabilidad de las explicaciones y si varían el desbalance de forma controlada.

4.5. Vacío de investigación

De la revisión se desprende que (1) los trabajos sobre Elliptic se concentran en el rendimiento y tratan la explicación de forma cualitativa; (2) las evaluaciones sistemáticas de explicadores de

Tabla 4.1: Trabajos relacionados según su cobertura de los cuatro rasgos que combina la tesis, de acuerdo con lo que declara el resumen de cada trabajo. ✓: lo declara; –: no lo declara. Desbalance controlado significa variar el nivel de desbalance, no solo la técnica que lo compensa.

Trabajo	Grafos	Explicación	Estabilidad medida	Desbalance controlado
Weber et al. [9]	✓	–	–	–
Alarab et al. [38, 40]	✓	✓	–	–
Lawal et al. [75]	✓	✓	–	–
He et al. [77]	✓	✓	–	–
Agarwal et al. [14]	✓	✓	–	–
Longa et al. [15]	✓	✓	–	–
Chen et al. [26]	–	✓	✓	✓
Mammadov et al. [80]	–	✓	✓	✓
Rai et al. [78]	–	✓	✓	✓
Esta tesis	✓	✓	✓	✓

GNN usan sobre todo grafos sintéticos o de otros dominios y no varían el desbalance; y (3) los estudios sobre desbalance y explicaciones usan datos tabulares. No se encontró un estudio que mida la estabilidad de varios explicadores de GNN sobre un problema real de lavado de dinero mientras se controlan, en un diseño factorial con réplicas independientes, el nivel de desbalance, la arquitectura y la técnica de compensación. Tampoco se encontró uno que verifique, con un control de pesos aleatorios, si la estabilidad medida en ese problema refleja lo que el modelo aprendió. Esta tesis aborda ambas carencias.

Capítulo 5

Metodología

Este capítulo describe el diseño experimental completo: los datos y sus etiquetas (Sección 5.1), la partición temporal y la razón para excluir el periodo del cierre del mercado (Sección 5.3), los escenarios de desbalance (Sección 5.4), el entrenamiento de los modelos (Sección 5.5), los explicadores (Sección 5.6), las medidas de estabilidad (Sección 5.7), el control de pesos aleatorios (Sección 5.8) y las pruebas estadísticas (Sección 5.9). La Figura 5.1 muestra el flujo completo y la Tabla 5.1 resume los factores.

Tabla 5.1: Factores del diseño experimental. Cada combinación se entrena con tres semillas de modelo, lo que da $4 \times 6 \times 3 \times 3 = 216$ modelos.

Factor	Niveles
Arquitectura	GCN, GraphSAGE, GAT, TAGCN
Escenario de desbalance	Nativo (1:41,1), 1:10, 1:20, 1:10 con SMOTE, 1:100, 1:200
Compensación	Sin ajuste, pesos por clase, *focal loss*
Semilla del modelo	42, 43, 44
Explicador	GNNExplainer, PGExplainer, Shapley por muestreo, Integrated Gradients, Expected Gradients
Medida de estabilidad	Spearman sobre las 165 variables (principal), Spearman sobre las variables distintas de cero, Jaccard de las 10 variables principales

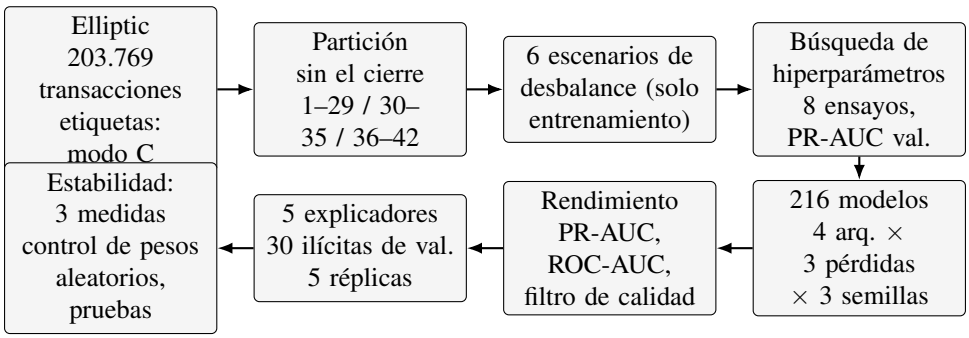


Figura 5.1: Flujo experimental de la corrida principal.

5.1. Datos

Se usa el conjunto de datos Elliptic [9], un grafo de 203.769 transacciones de Bitcoin unidas por 234.355 aristas dirigidas, que representan flujos de fondos entre transacciones. Cada transacción tiene 165 variables numéricas anonimizadas: 93 describen la propia transacción (montos, número de entradas y salidas, comisiones y otras) y 72 agregan información de sus vecinas a uno y dos saltos. Además, cada transacción pertenece a uno de 49 pasos de tiempo, separados aproximadamente dos semanas entre sí, y no hay aristas entre pasos distintos. Los autores no publican el significado de las variables.

Las etiquetas son tres: 4.545 transacciones ilícitas (2,2 %), 42.019 lícitas (20,6 %) y 157.205 sin etiqueta (77,1 %). Como clase positiva se toma la ilícita. Como clase negativa se toman las lícitas y las sin etiqueta juntas. Esta elección reproduce la situación de un sistema real, en el que la inmensa mayoría de las transacciones no ha sido revisada por nadie y no se puede descartar que haya ilícitas entre ellas. Con ella, la razón entre negativas e ilícitas en el entrenamiento es de 41 a 1, que es el desbalance real del problema. La alternativa de usar solo las lícitas como negativas da una razón de 7,6 a 1 y produce un problema más fácil y menos realista.

Las variables se escalan con la mediana y el rango intercuartílico, ajustados solo con las transacciones del periodo de entrenamiento, para que los valores extremos de los montos no dominen. Una consecuencia de este escalado, que resulta importante para la medición de la estabilidad, es que toda variable igual a su mediana queda exactamente en cero. En las 30 transacciones que se explican (Sección 5.6), en promedio el 44 % de las variables vale cero, y la mediana de variables distintas de cero por transacción es 89 de 165.

5.2. Análisis exploratorio del grafo

El grafo es muy disperso (Figura 5.2a). Cada transacción tiene en promedio 2,3 aristas y la mediana es 2; el 34,5 % de las transacciones tiene una sola arista, y la de mayor grado tiene 473. Las ilícitas tienen menos conexiones que las lícitas (mediana 1 frente a 2; media 2,0 frente a 3,1). Ninguna arista une transacciones de pasos de tiempo distintos, de modo que cada paso es un grafo separado, con entre 1.089 y 7.880 transacciones.

La homofilia de aristas entre transacciones etiquetadas es alta, 0,954, pero está dominada por las aristas entre lícitas, que son la gran mayoría. La composición de las vecinas de las ilícitas cuenta otra historia (Figura 5.2b): solo el 21,8 % de las vecinas de una transacción ilícita es ilícita, el 18,5 % es lícita y el 59,6 % no tiene etiqueta. Visto por transacción, el 70 % de las ilícitas no tiene ninguna vecina ilícita. En la tarea de esta tesis, en la que las transacciones sin etiqueta cuentan como negativas, una ilícita está rodeada sobre todo de negativas. La señal del fraude, por tanto, está principalmente en las variables de la propia transacción, y una arquitectura que

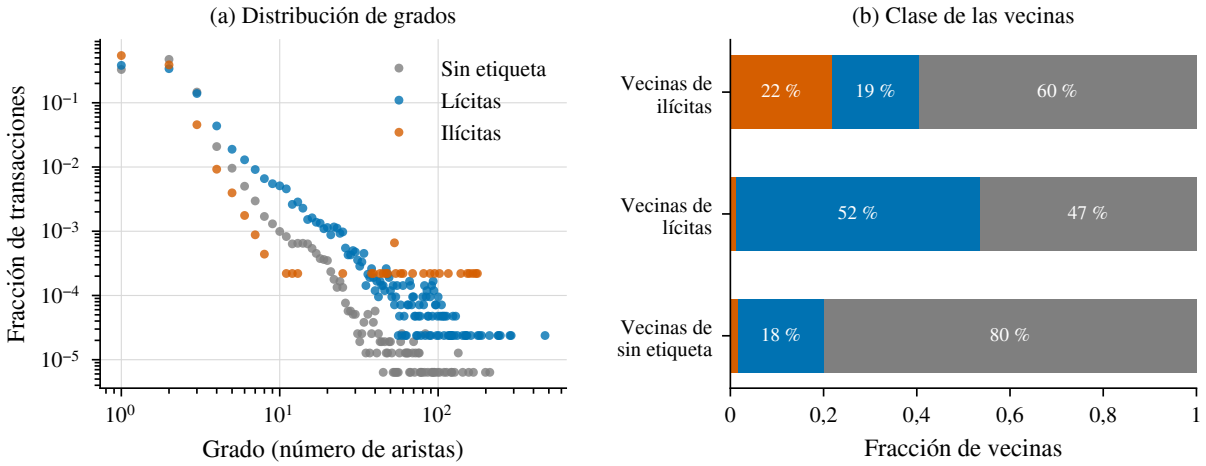


Figura 5.2: (a) Distribución del número de aristas por transacción según su clase, en escala logarítmica. (b) Clase de las vecinas de las transacciones de cada clase, contando cada arista desde sus dos extremos.

la promedia con sus vecinas sin darle un peso propio la diluye (Sección 3.3). Este rasgo anticipa el bajo rendimiento de GCN.

5.3. Partición temporal y exclusión del cierre del mercado

La partición respeta el orden del tiempo: el modelo se entrena con los pasos más antiguos y se evalúa con los más recientes, de modo que nunca ve el futuro. La partición habitual en la literatura usa los pasos 1 a 34 para entrenar y los pasos 35 a 49 para evaluar, y en este trabajo se dividió esa evaluación en validación (35 a 42) y prueba (43 a 49).

En el paso 43 se cerró un mercado ilegal importante, y los autores de Elliptic reportaron que desde ese momento ningún modelo detectaba las nuevas transacciones ilícitas [9]. Es un cambio en la distribución de los datos a lo largo del tiempo [81]: lo que el modelo aprendió de los mercados que existían deja de describir la actividad ilícita posterior. Con la partición habitual, la prueba completa cae después del cierre. En la corrida de esta tesis con esa partición, el mejor modelo de cada arquitectura obtuvo en prueba un PR-AUC entre 0,005 y 0,012, frente a 0,006 que obtendría un clasificador al azar (Anexo A).

Por ello el estudio principal usa solo los pasos 1 a 42 y los reparte en tres bloques contiguos: entrenamiento con los pasos 1 a 29, validación con los pasos 30 a 35 y prueba con los pasos 36 a 42 (Figura 5.3 y Tabla 5.2). Los pasos 43 a 49 no se usan.

La partición tiene tres costos que conviene declarar. Primero, el entrenamiento dispone de 2.871 transacciones ilícitas en lugar de 3.462, es decir, 591 menos que con la partición habitual. Segundo, los cortes en los pasos 29 y 35 los escogieron los autores para que validación y prueba tuvieran cada una cerca de una sexta parte de las ilícitas anteriores al cierre; no responden a

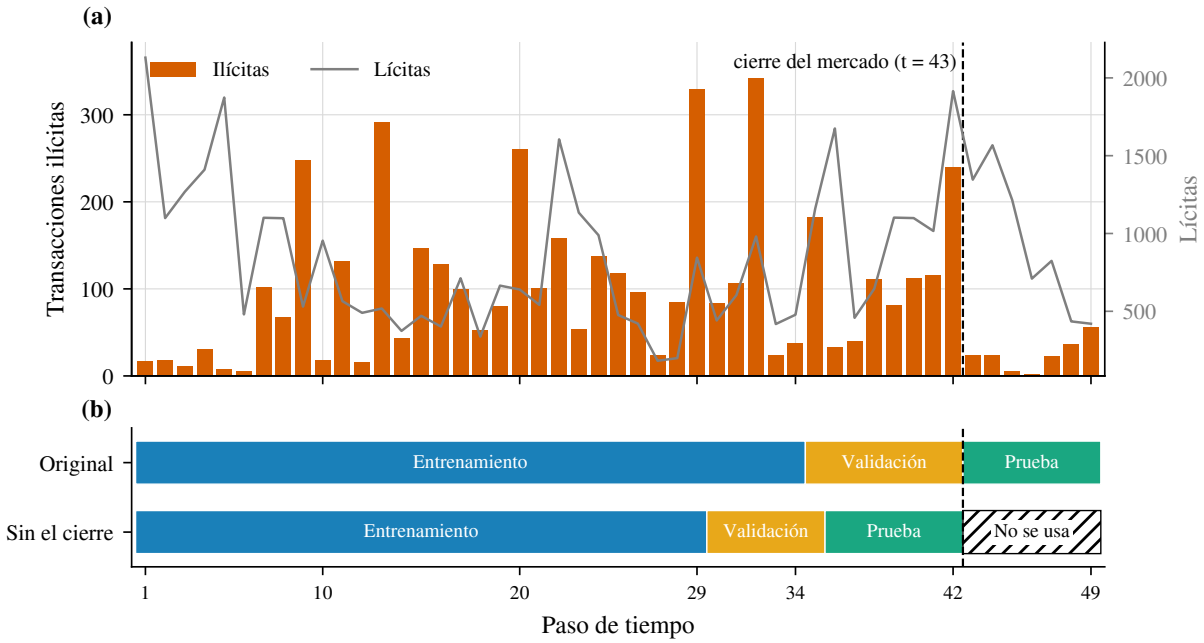


Figura 5.3: (a) Transacciones ilícitas (barras) y lícitas (línea) por paso de tiempo en Elliptic. La línea discontinua marca el cierre del mercado ilegal en el paso 43. (b) Partición original y partición sin el cierre usada en el estudio principal; en esta última los pasos 43 a 49 no se usan.

Tabla 5.2: Transacciones por parte de cada partición. Negativas son lícitas más sin etiqueta; la razón es negativas por cada ilícita.

Partición	Parte	Pasos	Ilícitas	Negativas	Razón
Original	Entrenamiento	1 a 34	3.462	132.803	1:38,4
	Validación	35 a 42	914	36.906	1:40,4
	Prueba	43 a 49	169	29.515	1:174,6
Sin el cierre	Entrenamiento	1 a 29	2.871	117.933	1:41,1
	Validación	30 a 35	773	20.195	1:26,1
	Prueba	36 a 42	732	31.581	1:43,1

un criterio externo. Tercero, la prueba ya no mide la capacidad del modelo para adaptarse a un cambio drástico del mercado, sino su desempeño en un periodo posterior al entrenamiento pero en condiciones parecidas. Esta tesis no estudia la adaptación a ese cambio, sino la estabilidad de las explicaciones, y para ello necesita modelos que detecten de verdad.

5.4. Escenarios de desbalance

El nivel de desbalance se modifica solo en el entrenamiento; validación y prueba conservan siempre su proporción real. Los escenarios principales protegen la señal del fraude: no quitan transacciones ilícitas, sino que submuestrean las negativas o sobremuestrean las ilícitas. Las negativas se submuestrean de forma estratificada, conservando la misma mezcla de lícitas y sin etiqueta que en el escenario nativo, con una semilla de datos fija (2026) que es distinta de la

semilla del modelo. Submuestrear significa sacar transacciones de la función de pérdida, no del grafo: las transacciones descartadas siguen transmitiendo información a sus vecinas.

- **Nativo (1:41,1).** Las 2.871 ilícitas y las 117.933 negativas del entrenamiento.
- **1:10 y 1:20.** Las 2.871 ilícitas y 28.710 o 57.420 negativas.
- **1:10 con SMOTE.** Las 2.871 ilícitas, 2.871 ilícitas sintéticas y 57.420 negativas. Cada sintética interpola los atributos de una ilícita y de una de sus vecinas más cercanas en el espacio de atributos [12], se añade al grafo en el mismo paso de tiempo y hereda las aristas de la ilícita de la que parte.
- **1:100 y 1:200.** Escenarios de estrés solicitados por el director para forzar el desbalance más allá del real. Como el nativo ya usa todas las negativas, la única forma de llegar a esas razones es quitar ilícitas: se conservan 1.179 y 589 ilícitas, con las 117.933 negativas.

Los escenarios de estrés cambian dos cosas a la vez, la proporción y la cantidad absoluta de ejemplos ilícitos, por lo que sus resultados se analizan junto a los principales pero se interpretan con esa salvedad.

5.5. Modelos y entrenamiento

Se entrenan cuatro arquitecturas de GNN para clasificar nodos: GCN [44], GraphSAGE [45], GAT [46] y TAGCN [47], descritas en la Sección 3.2. Todas reciben el grafo completo y producen una probabilidad de que cada transacción sea ilícita.

Compensación del desbalance. Se comparan tres funciones de pérdida (Sección 3.4): entropía cruzada sin ajuste; entropía cruzada con pesos por clase inversamente proporcionales a su frecuencia en el entrenamiento, de modo que cada clase aporta la mitad de la pérdida total; y *focal loss* [13] con $\gamma = 2$ y $\alpha = 0{,}75$ como peso de la clase ilícita.

Búsqueda de hiperparámetros. Para cada combinación de arquitectura, escenario y pérdida se hace una búsqueda bayesiana de 8 ensayos que maximiza el PR-AUC de validación. El primer ensayo usa valores tomados de la literatura y los siguientes se proponen con un estimador de Parzen. Cada ensayo entrena hasta 50 épocas, con parada temprana a las 10 épocas sin mejora. El espacio es el mismo para las cuatro arquitecturas: dimensión oculta en $\{64, 128, 148\}$, 2 o 3 capas, *dropout* entre 0,1 y 0,5, tasa de aprendizaje entre 10^{-4} y 10^{-2} y decaimiento de pesos entre 10^{-5} y 10^{-3}, las dos últimas en escala logarítmica. GAT busca además 2 o 4 cabezas de atención y TAGCN un orden de filtro K entre 2 y 4. Por memoria, GAT se limita a 2 capas: con 148 dimensiones y 8 cabezas el grafo completo no cabe en una GPU de 8 GB. Todas las

arquitecturas reciben el mismo número de ensayos y de épocas, para que la comparación no confunda arquitectura con presupuesto de cómputo.

Entrenamiento final y semillas. Con los mejores hiperparámetros se entrena el modelo final hasta 150 épocas con Adam, con parada temprana a las 20 épocas sin mejora del F1 de validación. Esto se repite con tres semillas de modelo (42, 43 y 44); las semillas 43 y 44 reutilizan los hiperparámetros encontrados con la 42, de modo que la diferencia entre semillas refleja solo la aleatoriedad del entrenamiento. Los hiperparámetros elegidos se listan en el Anexo D. La búsqueda prefirió para TAGCN siempre 128 dimensiones y 3 capas, con órdenes de filtro de 2 a 4, y para GAT siempre 2 capas; GraphSAGE alternó entre 2 y 3 capas y GCN eligió casi siempre 2. El cálculo en GPU no es determinista a nivel de pesos, porque las sumas de mensajes usan operaciones atómicas, lo que se suma a esa aleatoriedad.

Evaluación del rendimiento. La métrica principal es el PR-AUC, el área bajo la curva de precisión y exhaustividad, porque bajo un desbalance fuerte el ROC-AUC puede ser alto aunque el modelo produzca muchas más falsas alarmas que aciertos [52]. Se reporta también el ROC-AUC. El umbral de decisión se fija en la validación, como el que maximiza el F1, sin usar información de la prueba.

Filtro de calidad. Explicar un modelo que no aprendió no informa sobre el problema. Por eso cada modelo se marca como aprobado si en validación, con umbral 0,5, alcanza un F1 de al menos 0,30 y un coeficiente de Matthews [51] de al menos 0,15. El filtro se usa para interpretar los resultados, no para excluir modelos de la medición: todos los modelos se explican, y los análisis principales se restringen a las arquitecturas que aprenden.

5.6. Explicadores

Transacciones explicadas. Se explican las mismas 30 transacciones para todos los modelos: 30 ilícitas de validación sorteadas con una semilla fija (1234) entre las 773 ilícitas de los pasos 30 a 35. Usar el mismo conjunto permite comparar explicaciones entre modelos. Se explica la validación y no la prueba porque es el periodo con el que se seleccionó el modelo y donde su comportamiento está mejor caracterizado. Las 30 transacciones son ilícitas, pero no todas son verdaderos positivos para cada modelo: con el umbral de 0,5, los modelos con pesos por clase aciertan en promedio entre 16,5 (TAGCN) y 22,3 (GCN) de ellas. Se explican todas, porque la pregunta es cuán reproducible es la explicación de una predicción, sea acertada o no, y porque restringirse a los aciertos de cada modelo impediría comparar modelos sobre las mismas transacciones.

Subgrafo receptivo. Una GNN de L capas solo usa, para predecir una transacción, la información de sus vecinas a L saltos o menos (a $L \cdot K$ saltos en TAGCN). Cada explicación se calcula sobre ese subgrafo, y se verifica que la predicción del modelo en el subgrafo coincida con la

del grafo completo. Esto reduce la memoria de varios gigabytes a unos pocos megabytes sin cambiar lo que se explica.

Se usan cinco explicadores, descritos en la Sección 3.6:

- **GNNExplainer** [56], que optimiza una máscara sobre las variables de los nodos del subgrafo durante 100 iteraciones con tasa de aprendizaje 0,01. La importancia de cada variable es la media de su máscara sobre los nodos del subgrafo.
- **PGExplainer** [57], que entrena una red que asigna importancia a las aristas. Se entrena durante 100 épocas con 50 nodos, de los cuales al menos 25 son ilícitos. Como explica aristas y no variables, su estabilidad se reporta aparte.
- **Shapley por muestreo** [18, 59], que estima la contribución de cada variable promediando 50 permutaciones aleatorias; una variable ausente toma el valor medio de esa variable en el subgrafo.
- **Integrated Gradients** [19], que integra el gradiente a lo largo del camino recto entre una referencia y la transacción, con 100 pasos. La referencia es la mediana del entrenamiento, que tras el escalado es el vector cero. Es determinista.
- **Expected Gradients** [20], que promedia Integrated Gradients sobre 200 referencias tomadas al azar de 5.000 transacciones negativas del entrenamiento.

Ninguno de los explicadores de variables se entrena previamente: los dos de gradiente y el de Shapley se aplican directamente al modelo, y GNNExplainer optimiza una máscara nueva para cada transacción. Solo PGExplainer tiene una fase de entrenamiento propia.

Réplicas. Cada explicador se ejecuta cinco veces sobre cada transacción y cada modelo, con semillas distintas. La estabilidad mide cuánto se parecen esas cinco explicaciones.

Detalles de implementación

Los modelos y los explicadores se implementaron con PyTorch 2.6 y PyTorch Geometric 2.7; la búsqueda de hiperparámetros con Optuna 4.8; las métricas con scikit-learn 1.8 y las pruebas estadísticas con SciPy 1.17. GNNExplainer y PGExplainer son las implementaciones de PyTorch Geometric. GNNExplainer se configuró para producir una máscara por variable y por nodo (`node_mask_type=.ªttributes"`) y una máscara por arista, y explica la salida del modelo, no la etiqueta verdadera. Dos parámetros por defecto de PGExplainer se modificaron porque producían resultados inválidos en Elliptic: el coeficiente de tamaño de la máscara se bajó de 0,05 a 0,005, porque con el valor por defecto la máscara colapsaba a cero en todas las aristas, y la temperatura se fijó en 1 en lugar del esquema de 5 a 2, porque con este último la

pérdida desbordaba a valores no numéricos. Además, los 50 nodos con que se entrena PGExplainer se toman solo entre los que reciben al menos una arista: cerca del 30 % de los nodos de entrenamiento no recibe ninguna, y en ellos la pérdida de PGExplainer no está definida. Shapley por muestreo, Integrated Gradients y Expected Gradients son implementaciones propias (`src/explainability/`).

5.7. Medidas de estabilidad

Sea $\mathbf{a}^{(r)} \in \mathbb{R}^{165}$ el vector de importancias que la réplica r asigna a las variables de una transacción. La estabilidad de esa transacción es el promedio, sobre los $\binom{5}{2} = 10$ pares de réplicas, de una medida de acuerdo s:

$$E = \frac{1}{10} \sum_{r<r'} s\left(\mathbf{a}^{(r)}, \mathbf{a}^{(r')}\right), \tag{5.1}$$

y la estabilidad de un modelo es el promedio de E sobre las 30 transacciones. Se usan tres medidas de acuerdo:

Spearman global (principal). La correlación de rangos de Spearman [60] entre los dos vectores completos de 165 importancias. Es la medida más usada en la literatura y permite comparar con otros trabajos.

Spearman sobre las variables distintas de cero. La misma correlación, calculada solo sobre las variables cuyo valor en la transacción es distinto de cero.

Jaccard top-10. El índice de Jaccard [61] entre los conjuntos de las 10 variables más importantes de cada réplica, $|A \cap B|/|A \cup B|$. Si las dos réplicas comparten k de sus 10 variables, el índice vale $k/(20-k)$: compartir 3, 5 o 7 variables da 0,18, 0,33 y 0,54.

Las dos medidas adicionales responden a un artefacto que se detectó durante el estudio. Una variable que vale cero recibe una importancia nula o casi nula de los cuatro explicadores de variables, por razones distintas en cada uno: Integrated Gradients multiplica el gradiente por la diferencia entre la variable y la referencia, que es cero; la máscara de GNNExplainer anula las entradas sin gradiente; y en Shapley y Expected Gradients la referencia suele coincidir con cero. Esas variables, cerca de la mitad, quedan siempre al final del orden en cualquier réplica y en cualquier modelo, y el Spearman global cuenta su acuerdo como si fuera información. El tamaño de ese efecto se puede calcular. Si dos explicaciones solo coinciden en dejar empatadas al final las variables nulas de la transacción y ordenan las demás al azar, su correlación de Spearman media sobre las 30 transacciones es 0,81 (entre 0,54 y 0,86 según la transacción, porque la fracción de variables nulas va del 23 % al 50 %). Ese es el piso que la

medida principal da sin que las explicaciones compartan nada sobre las variables que sí importan (`tesis_v4/scripts/piso_spearman.py`). El Spearman sobre las variables distintas de cero elimina el bloque, y el Jaccard top-10 solo mira las variables que el analista vería primero.

Además de la estabilidad entre réplicas de un mismo modelo, se mide el acuerdo entre las explicaciones medias de dos modelos distintos sobre las mismas transacciones: entre semillas, entre escenarios, entre arquitecturas y entre explicadores.

5.8. Control de pesos aleatorios

Una explicación estable no es necesariamente una explicación del modelo. Si el explicador dependiera solo de los datos de la transacción, daría el mismo resultado con cualquier modelo. Para comprobarlo se aplica la prueba de aleatorización de parámetros de Adebayo et al. [29]: para cada arquitectura y semilla, en el escenario nativo con pesos por clase (12 modelos), se explican las mismas 30 transacciones con el modelo entrenado y con dos redes de la misma arquitectura e hiperparámetros pero con pesos aleatorios, sin entrenar. Se reporta el acuerdo entre la explicación del modelo entrenado y la de la red aleatoria. Si ese acuerdo es tan alto como la estabilidad entre réplicas, el orden de las variables no depende de lo que el modelo aprendió.

5.9. Análisis estadístico

La unidad independiente es el modelo entrenado, es decir, una arquitectura con una semilla. Los modelos de una misma semilla en distintos escenarios parten de los mismos pesos iniciales y de los mismos hiperparámetros, y sus explicaciones resultan casi idénticas (acuerdo de 0,96 entre escenarios con la misma semilla frente a 0,92 entre semillas). Tratarlos como observaciones independientes inflaría la significancia, por lo que las pruebas se construyen así:

H1 (escenario). Prueba de Friedman [63] con 9 bloques (3 arquitecturas que aprenden por 3 semillas) y 6 tratamientos (los escenarios), con pesos por clase. Se complementa con una prueba de equivalencia de dos pruebas unilaterales (TOST) [65] entre el nativo y 1:200 con margen de 0,05: rechazar ambas hipótesis nulas indica que la diferencia, si existe, es menor que el margen.

H2 (arquitectura). Prueba de Friedman con 3 bloques (las semillas) y 4 tratamientos (las arquitecturas), sobre la media de los cuatro escenarios principales con pesos por clase. Con tres bloques la prueba tiene poca potencia, por lo que se reporta además en cuántas semillas queda primera cada arquitectura.

H3 (compensación). Prueba de signos pareada sobre 9 pares (arquitectura por semilla, media de los cuatro escenarios principales), de pesos por clase frente a *focal loss* y frente a sin ajuste.

Los intervalos de confianza de las figuras se obtienen por *bootstrap* sobre los modelos [66]. Todas las pruebas se repiten con las tres medidas de estabilidad, y una conclusión se considera sólida solo si las tres coinciden.

Capítulo 6

Resultados

Este capítulo presenta los resultados de la corrida principal, con la partición sin el cierre del mercado. Primero se muestra que esa partición recupera una prueba informativa (Sección 6.1) y cómo rinden los modelos según el escenario y la pérdida (Sección 6.2). Después se contrastan las tres hipótesis (Secciones 6.5 a 6.7) y se examina qué parte de la estabilidad medida depende del modelo (Sección 6.9). Salvo que se indique otra cosa, cada cifra es la media de tres semillas de modelo.

6.1. La partición sin el cierre recupera la prueba

La Figura 6.1 compara el rendimiento en prueba de las dos particiones, en el escenario nativo y con la pérdida que obtuvo el mayor PR-AUC de validación para cada arquitectura. Con la partición original, cuya prueba cae después del cierre, ningún modelo supera al azar. Sin el cierre, tres de las cuatro arquitecturas detectan: GraphSAGE alcanza un PR-AUC de 0,273, TAGCN de 0,244 y GAT de 0,172, entre 7 y 12 veces el valor del azar (0,023). GCN queda en 0,085. El ROC-AUC sube de valores entre 0,48 y 0,71 a valores entre 0,73 y 0,88. El mejor modelo de toda la matriz, GraphSAGE con pesos por clase en el escenario 1:20, obtiene en prueba un PR-AUC de 0,306 y un ROC-AUC de 0,901.

La Figura 6.2 muestra las curvas completas. La curva de precisión y exhaustividad deja ver lo que el ROC-AUC oculta: con la mitad de las ilícitas recuperadas, la precisión de GraphSAGE y de TAGCN ronda 0,3, es decir, unas dos falsas alarmas por cada acierto. La Tabla 6.1 resume las cifras.

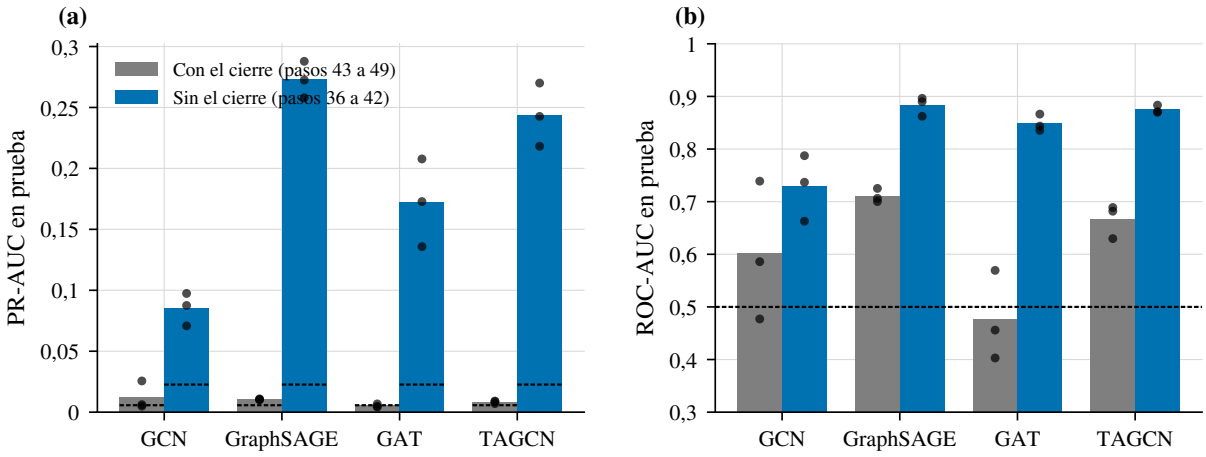


Figura 6.1: Rendimiento en prueba con la partición original (prueba en los pasos 43 a 49, después del cierre) y con la partición sin el cierre (prueba en los pasos 36 a 42). Escenario nativo, pérdida elegida por validación. Las barras son la media de tres semillas, los puntos cada semilla y las líneas discontinuas el valor del azar: la prevalencia en (a) y 0,5 en (b).

Tabla 6.1: Rendimiento en el escenario nativo con la pérdida elegida por validación (media ± desviación estándar de tres semillas). Filtro: modelos que superan el filtro de calidad en validación.

Arquitectura	Pérdida elegida	PR-AUC val.	PR-AUC prueba	ROC-AUC prueba	Filtro
GCN	Focal loss	0,214 ± 0,054	0,085 ± 0,013	0,729 ± 0,063	0/3
GraphSAGE	Focal loss	0,478 ± 0,038	0,273 ± 0,015	0,883 ± 0,018	3/3
GAT	Sin ajuste	0,282 ± 0,027	0,172 ± 0,036	0,848 ± 0,016	2/3
TAGCN	Focal loss	0,529 ± 0,074	0,244 ± 0,026	0,875 ± 0,007	3/3
Azar		0,037	0,023	0,5	

6.2. Rendimiento según el escenario y la pérdida

La Figura 6.3 y la Tabla 6.2 muestran el PR-AUC de validación de cada combinación. Tres regularidades se observan.

La mejor pérdida depende del escenario. En el escenario nativo, *focal loss* iguala o supera a los pesos por clase: en GraphSAGE, 0,478 frente a 0,326; en TAGCN, 0,529 frente a 0,212. En 1:10, la entropía sin ajuste es la mejor en ambas (0,551 y 0,594). No hay una pérdida que sea la mejor en todos los escenarios.

Con poco fraude solo resisten los pesos por clase. En 1:100 y 1:200, la *focal loss* de GraphSAGE cae a 0,189 y 0,054, y la de TAGCN a 0,282 y 0,054. Los pesos por clase mantienen a GraphSAGE en 0,447 y 0,420, y a TAGCN en 0,357 y 0,307. Es el mismo comportamiento que en la partición original.

La entropía sin ajuste es frágil. En GraphSAGE pasa de 0,551 en 1:10 a 0,035 en 1:20 y se queda en el nivel del azar en todos los escenarios más desbalanceados; en TAGCN baja de 0,594 a 0,046 entre 1:10 y 1:100. Cuando el desbalance supera cierto punto, el modelo minimiza la

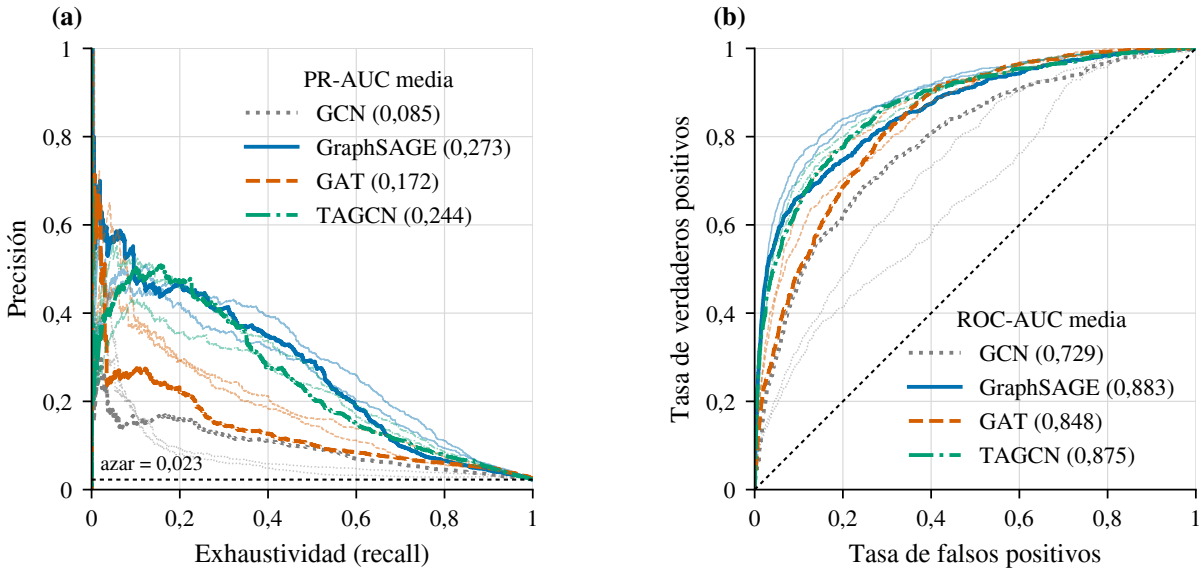


Figura 6.2: Curvas de precisión y exhaustividad (a) y ROC (b) en la prueba de la partición sin el cierre, escenario nativo, con la pérdida elegida por validación para cada arquitectura. Trazo grueso: semilla 42; trazos finos: semillas 43 y 44. Entre paréntesis, el área media de las tres semillas.

pérdida prediciendo casi siempre la clase negativa. GAT es la excepción: es la arquitectura en la que la entropía sin ajuste supera más veces el filtro de calidad (Tabla 6.3).

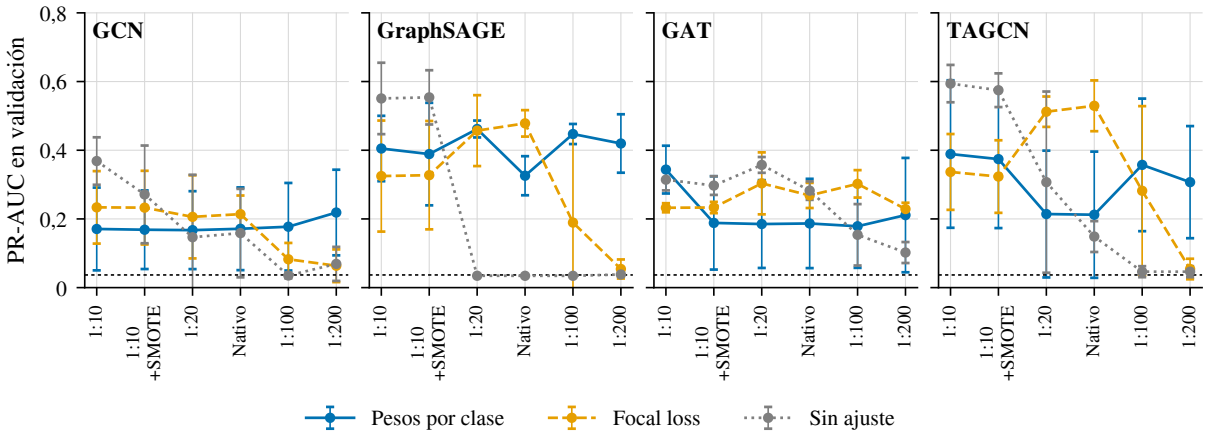


Figura 6.3: PR-AUC de validación por escenario y pérdida (media y desviación estándar de tres semillas). La línea discontinua es el valor del azar (0,037).

GCN no aprende en este problema. Ningún modelo GCN con pesos por clase supera el filtro de calidad (Tabla 6.3). GCN promedia a cada transacción con sus vecinas sin darle un peso propio (Sección 3.2), y en Elliptic la señal está sobre todo en la propia transacción. Por eso los análisis de estabilidad por escenario y por pérdida se restringen a GraphSAGE, GAT y TAGCN, y GCN se incluye solo en la comparación entre arquitecturas, con esa salvedad. Hay que añadir que GAT con pesos por clase tampoco supera el filtro en ningún modelo, aunque su PR-AUC de validación esté entre 0,18 y 0,34, muy por encima del azar: el filtro usa el umbral fijo de 0,5, y con ese umbral su F1 queda entre 0,13 y 0,17. Se mantiene en el análisis porque su PR-AUC

Tabla 6.2: PR-AUC de validación por escenario, arquitectura y pérdida (media de tres semillas).

Arquitectura	Pérdida	1:10	1:10 + SMOTE	1:20	Nativo	1:100	1:200
GCN	Pesos por clase	0,171	0,169	0,167	0,171	0,177	0,219
	Focal loss	0,234	0,233	0,206	0,214	0,083	0,064
	Sin ajuste	0,369	0,271	0,147	0,159	0,035	0,069
GraphSAGE	Pesos por clase	0,405	0,389	0,462	0,326	0,447	0,420
	Focal loss	0,325	0,327	0,457	0,478	0,189	0,054
	Sin ajuste	0,551	0,554	0,035	0,034	0,034	0,038
GAT	Pesos por clase	0,344	0,188	0,185	0,187	0,179	0,211
	Focal loss	0,233	0,233	0,304	0,268	0,302	0,229
	Sin ajuste	0,314	0,297	0,357	0,282	0,154	0,102
TAGCN	Pesos por clase	0,389	0,374	0,214	0,212	0,357	0,307
	Focal loss	0,337	0,323	0,512	0,529	0,282	0,054
	Sin ajuste	0,594	0,575	0,307	0,149	0,046	0,046

muestra que sí ordena las transacciones mejor que el azar.

Tabla 6.3: Modelos que superan el filtro de calidad en validación (F1 $\geq$ 0,30 y MCC $\geq$ 0,15 con umbral 0,5), de 18 por celda (6 escenarios por 3 semillas).

Arquitectura	Pesos por clase	Focal loss	Sin ajuste
GCN	0/18	3/18	6/18
GraphSAGE	10/18	11/18	6/18
GAT	0/18	8/18	11/18
TAGCN	8/18	13/18	7/18

6.3. De la validación a la prueba

La Figura 6.4 muestra las curvas de validación del escenario nativo. Los valores de validación son mayores que los de prueba en todas las arquitecturas: GraphSAGE pasa de 0,478 a 0,273, TAGCN de 0,529 a 0,244, GAT de 0,282 a 0,172 y GCN de 0,214 a 0,085. Parte de la diferencia es esperable, porque la prevalencia de ilícitas es menor en prueba (2,3 % frente a 3,7 %) y el PR-AUC depende de ella. Pero la caída relativa es mayor que la de la prevalencia: aun sin el cierre del mercado, la actividad ilícita de los pasos 36 a 42 no es idéntica a la de los pasos 30 a 35.

La validación tampoco ordena del todo bien las configuraciones para la prueba. En el escenario nativo, la *focal loss* supera claramente a los pesos por clase en validación en GraphSAGE (0,478 frente a 0,326), pero en prueba ambas quedan iguales (0,273 frente a 0,270). En TAGCN la ventaja de la *focal loss* en validación sí se mantiene en prueba (0,244 frente a 0,113). El mejor resultado de prueba de toda la matriz, GraphSAGE con pesos por clase en 1:20 (0,306), no es el mejor en validación. Con tres semillas y un periodo de validación de seis pasos, las diferencias de unas pocas centésimas entre configuraciones no deben interpretarse como un orden estable.

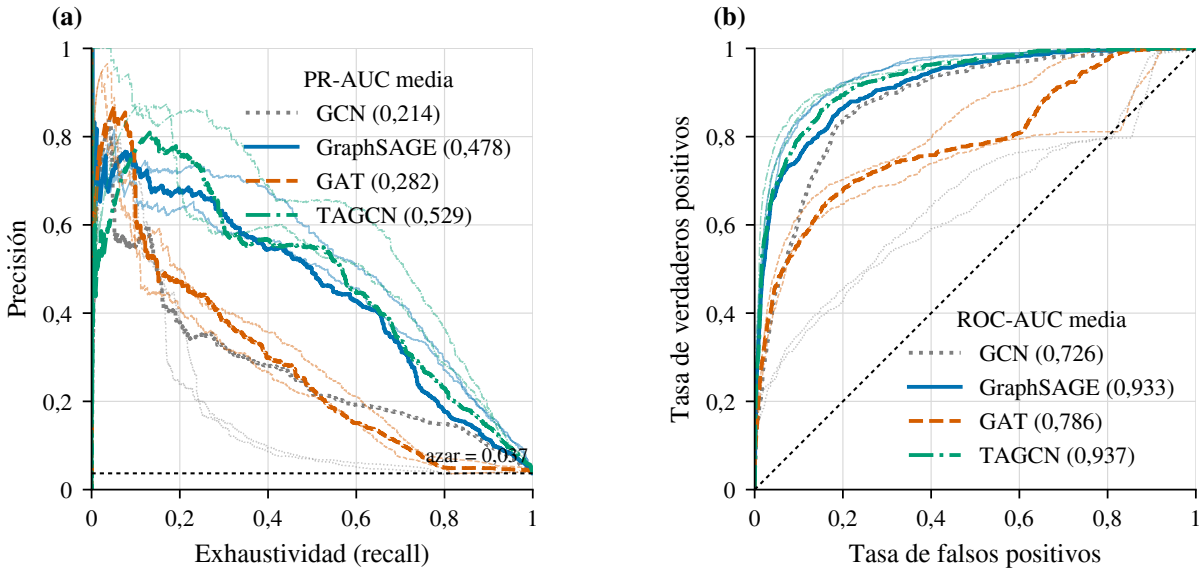


Figura 6.4: Curvas de precisión y exhaustividad (a) y ROC (b) en validación (pasos 30 a 35), escenario nativo, con la pérdida elegida para cada arquitectura. Trazo grueso: semilla 42; trazos finos: semillas 43 y 44.

6.4. Rendimiento en los escenarios de estrés

Los escenarios 1:100 y 1:200 quitan ilícitas del entrenamiento (quedan 1.179 y 589). La Figura 6.5 muestra que, con pesos por clase, los modelos siguen detectando en prueba: GraphSAGE obtiene 0,268 en 1:100, prácticamente lo mismo que en el nativo, y 0,183 en 1:200; GAT, TAGCN y GCN quedan entre 0,08 y 0,14. Entrenar con la quinta parte de los ejemplos ilícitos reduce el rendimiento de prueba de GraphSAGE en un tercio, no lo anula. Con las otras dos pérdidas la caída es mayor (Tabla 6.2).

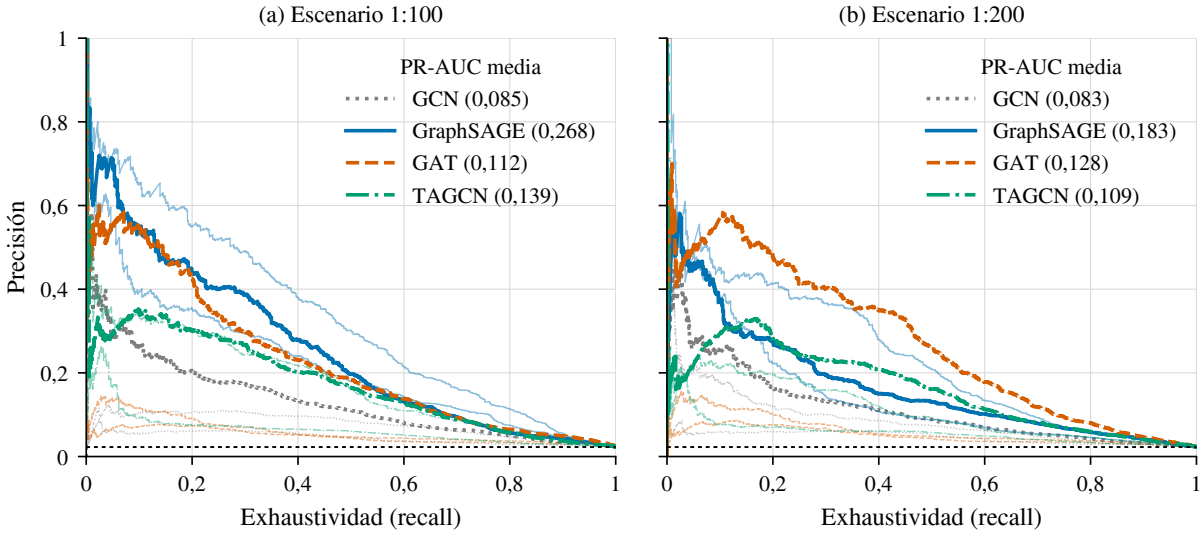


Figura 6.5: Curvas de precisión y exhaustividad en prueba para los escenarios de estrés, con pesos por clase. Trazo grueso: semilla 42; trazos finos: semillas 43 y 44. La línea discontinua es la prevalencia (0,023).

6.5. H1: el desbalance solo afecta la estabilidad cuando la pérdida no lo compensa

La Figura 6.6 y la Tabla 6.4 muestran la estabilidad entre réplicas de cada explicador en los seis escenarios, con pesos por clase y las tres arquitecturas que aprenden. Con la medida principal, la estabilidad de GNNExplainer se mueve entre 0,917 (nativo) y 0,939 (1:100), la de Shapley entre 0,991 y 0,994 y la de Expected Gradients entre 0,918 y 0,934. Integrated Gradients es determinista y vale 1 en todos los casos. No hay una tendencia a bajar a medida que el desbalance se hace más extremo: 1:100 y 1:200 no son los escenarios menos estables. Las dos medidas robustas confirman la ausencia de tendencia, aunque con valores más bajos (Sección 6.9).

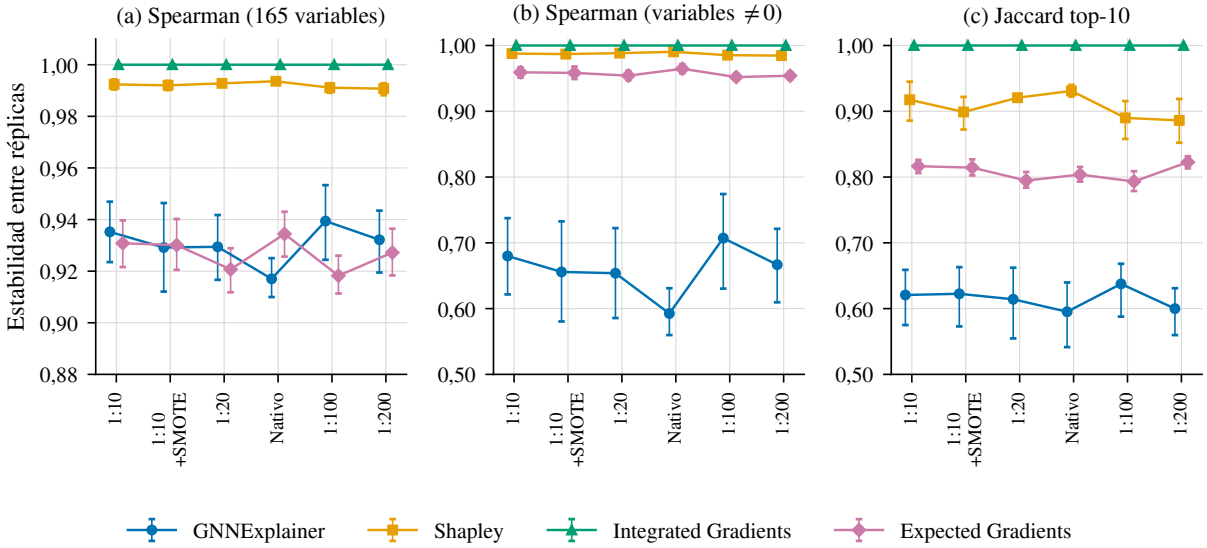


Figura 6.6: Estabilidad entre réplicas por escenario y explicador, con las tres medidas. Pesos por clase; media de 9 modelos (3 arquitecturas que aprenden por 3 semillas) e intervalo *bootstrap* del 95 %.

Tabla 6.4: Estabilidad entre réplicas por escenario (pesos por clase, 9 modelos). *p* Friedman: igualdad de los seis escenarios. *p* TOST: equivalencia entre nativo y 1:200 con margen 0,05 (un valor bajo indica equivalencia).

Medida	Explicador	1:10	1:10 + SMOTE	1:20	Nativo	1:100	1:200	*p* Friedman	*p* TOST
Spearman global	GNNExplainer	0,935	0,929	0,929	0,917	0,939	0,932	0,269	$< 0{,}001$
	Shapley	0,992	0,992	0,993	0,994	0,991	0,991	0,534	$< 0{,}001$
	Integrated Gradients	1,000	1,000	1,000	1,000	1,000	1,000	–	$< 0{,}001$
	Expected Gradients	0,931	0,930	0,921	0,934	0,918	0,927	0,248	$< 0{,}001$
Spearman $\neq 0$	GNNExplainer	0,680	0,656	0,654	0,593	0,707	0,667	0,117	0,791
	Shapley	0,988	0,987	0,988	0,990	0,985	0,985	0,448	$< 0{,}001$
	Integrated Gradients	1,000	1,000	1,000	1,000	1,000	1,000	–	$< 0{,}001$
	Expected Gradients	0,959	0,958	0,954	0,965	0,952	0,954	0,552	$< 0{,}001$
Jaccard top-10	GNNExplainer	0,621	0,623	0,614	0,595	0,638	0,600	0,233	0,012
	Shapley	0,918	0,899	0,921	0,931	0,890	0,886	0,117	0,394
	Integrated Gradients	1,000	1,000	1,000	1,000	1,000	1,000	–	$< 0{,}001$
	Expected Gradients	0,816	0,814	0,795	0,804	0,793	0,823	0,018	$< 0{,}001$

Las pruebas confirman la lectura visual. Con la medida principal, la prueba de Friedman no

rechaza la igualdad de los seis escenarios para ningún explicador ($p = 0{,}27$ en GNNExplainer, 0,53 en Shapley y 0,25 en Expected Gradients), y la prueba de equivalencia entre el nativo y 1:200 la acepta en los tres ($p \leq 0{,}0002$): la diferencia entre el escenario real y el más extremo es menor que 0,05. Con las medidas robustas hay dos excepciones que conviene examinar. Con el Jaccard top-10, Friedman rechaza la igualdad en Expected Gradients ($p = 0{,}018$), pero la diferencia entre el mejor y el peor escenario es de 0,03 y el escenario más estable es precisamente 1:200; no hay degradación. Con el Spearman sobre las variables distintas de cero, GNNExplainer no alcanza la equivalencia ($p = 0{,}79$): sus valores van de 0,593 (nativo) a 0,707 (1:100), con un intervalo amplio, pero de nuevo el escenario menos estable es el nativo y no los más desbalanceados.

Con las otras pérdidas, GNNExplainer sí pierde estabilidad. El análisis anterior usa los modelos con pesos por clase. Al repetirlo para cada pérdida (Tabla 6.5), el resultado cambia para GNNExplainer. Con *focal loss*, la prueba de Friedman sobre los seis escenarios rechaza la igualdad con las tres medidas (p entre 0,010 y 0,035), y la estabilidad baja del nativo a 1:200: de 0,923 a 0,909 con la medida principal, de 0,632 a 0,562 sin las variables nulas y de 0,623 a 0,561 con el Jaccard top-10. Sin ajuste la caída es mayor y muy significativa ($p < 0{,}001$ con las tres medidas): de 0,942 en 1:10 a 0,889 en 1:200 con la medida principal, y de 0,589 a 0,502 con el Jaccard top-10 entre el nativo y 1:200. Con pesos por clase, en cambio, ninguna prueba rechaza ($p \geq 0{,}12$). Con Shapley y Expected Gradients el efecto es marginal: de las nueve combinaciones de pérdida y medida, solo una es significativa con cada uno (Shapley sin ajuste con la medida principal, de 0,993 a 0,985; y Expected Gradients con pesos por clase y el Jaccard top-10, que sube de 0,804 a 0,823). Juntando las tres pérdidas (27 modelos), la estabilidad de GNNExplainer baja de forma significativa con las tres medidas ($p \leq 0{,}003$).

Veredicto sobre H1. La hipótesis se sostiene solo en parte. La estabilidad de GNNExplainer disminuye a medida que el desbalance se hace más extremo cuando el modelo se entrena con *focal loss* o sin ajuste, que son también las pérdidas con las que el rendimiento cae con el desbalance (Sección 6.2). Con pesos por clase, que mantienen el rendimiento, la estabilidad no cambia, ni siquiera en 1:100 y 1:200, donde además se quitaron ilícitas. Los explicadores Shapley y Expected Gradients prácticamente no responden al desbalance con ninguna pérdida. El desbalance no afecta la estabilidad de forma directa, sino a través de cuánto se degrada el modelo, y solo con el explicador que optimiza una máscara.

6.6. H2: no hay un orden estable entre arquitecturas

Con GNNExplainer, las cuatro arquitecturas quedan dentro de dos centésimas con la medida principal (de 0,923 en GAT a 0,937 en GCN), y las semillas de una misma arquitectura se dispersan tanto como las arquitecturas entre sí (Figura 6.7). Ninguna prueba de Friedman rechaza

Tabla 6.5: Estabilidad entre réplicas por escenario y pérdida (GraphSAGE, GAT y TAGCN; media de 9 modelos por celda).

Medida	Explicador	Pérdida	1:10	1:10 + SMOTE	1:20	Nativo	1:100	1:200
Spearman global	GNNExplainer	Pesos por clase	0,935	0,929	0,929	0,917	0,939	0,932
		Focal loss	0,919	0,920	0,926	0,923	0,912	0,909
		Sin ajuste	0,942	0,940	0,920	0,915	0,905	0,889
	Shapley	Pesos por clase	0,992	0,992	0,993	0,994	0,991	0,991
		Focal loss	0,995	0,994	0,995	0,995	0,995	0,993
		Sin ajuste	0,993	0,994	0,994	0,993	0,989	0,985
	Expected Gradients	Pesos por clase	0,931	0,930	0,921	0,934	0,918	0,927
		Focal loss	0,929	0,929	0,936	0,937	0,940	0,936
		Sin ajuste	0,934	0,940	0,938	0,940	0,921	0,933
Spearman $\neq 0$	GNNExplainer	Pesos por clase	0,680	0,656	0,654	0,593	0,707	0,667
		Focal loss	0,612	0,616	0,644	0,632	0,580	0,562
		Sin ajuste	0,731	0,721	0,622	0,599	0,544	0,460
	Shapley	Pesos por clase	0,988	0,987	0,988	0,990	0,985	0,985
		Focal loss	0,992	0,991	0,992	0,992	0,992	0,990
		Sin ajuste	0,989	0,991	0,990	0,989	0,981	0,976
	Expected Gradients	Pesos por clase	0,959	0,958	0,954	0,965	0,952	0,954
		Focal loss	0,958	0,958	0,961	0,963	0,961	0,958
		Sin ajuste	0,960	0,964	0,962	0,962	0,948	0,953
Jaccard top-10	GNNExplainer	Pesos por clase	0,621	0,623	0,614	0,595	0,638	0,600
		Focal loss	0,567	0,565	0,636	0,623	0,576	0,561
		Sin ajuste	0,659	0,650	0,604	0,589	0,526	0,502
	Shapley	Pesos por clase	0,918	0,899	0,921	0,931	0,890	0,886
		Focal loss	0,940	0,948	0,940	0,936	0,935	0,926
		Sin ajuste	0,915	0,918	0,932	0,923	0,904	0,891
	Expected Gradients	Pesos por clase	0,816	0,814	0,795	0,804	0,793	0,823
		Focal loss	0,796	0,791	0,806	0,804	0,789	0,801
		Sin ajuste	0,812	0,808	0,803	0,795	0,789	0,798

la igualdad con GNNExplainer (p entre 0,06 y 0,53) ni con Expected Gradients, y sin GCN ninguna lo hace con ningún explicador ni medida ($p \geq 0{,}097$). La única diferencia significativa aparece con Shapley y las cuatro arquitecturas ($p = 0{,}042$): GCN es la más estable en las tres semillas, pero la diferencia con la última es de cinco milésimas (0,996 frente a 0,991). Con tres semillas como bloques la prueba tiene poca potencia, por lo que la ausencia de diferencias no prueba la igualdad; lo que sí muestra es que no hay un orden que se repita.

Un resultado llama la atención: GCN, la arquitectura que no aprende, obtiene las explicaciones más estables con GNNExplainer en dos de las tres semillas y en las tres medidas. Esto anticipa lo que muestra la Sección 6.9: una explicación estable no es necesariamente una explicación de algo aprendido.

Veredicto sobre H2. La hipótesis no se sostiene entre las arquitecturas que aprenden: GraphSAGE, GAT y TAGCN producen explicaciones con estabilidad indistinguible. En la partición original GraphSAGE había resultado la más estable en las tres semillas; ese orden no se repite sin el cierre, y por tanto no se puede afirmar.

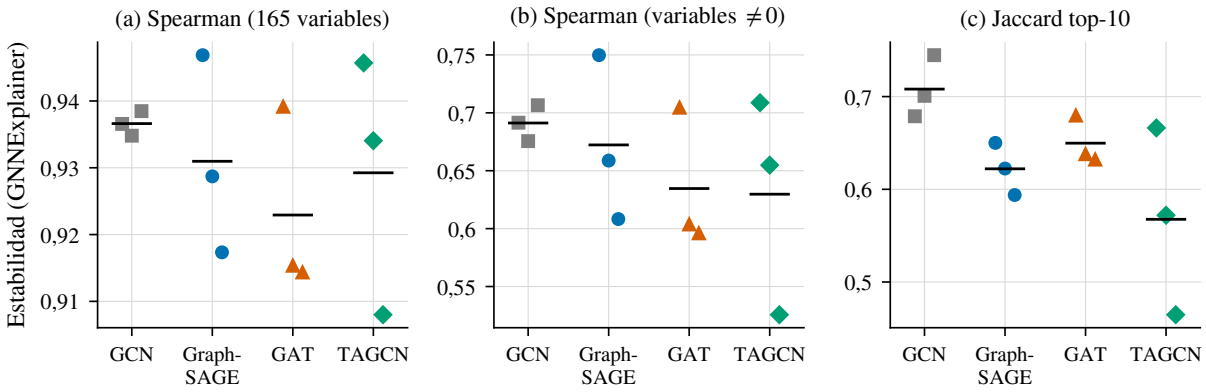


Figura 6.7: Estabilidad de GNNExplainer por arquitectura con las tres medidas (pesos por clase, media de los cuatro escenarios principales). Cada punto es una semilla; la línea horizontal es la media.

Tabla 6.6: Estabilidad por arquitectura (pesos por clase, media de los escenarios principales, 3 semillas). p (4): Friedman con las cuatro arquitecturas; p (3): sin GCN. Primera: número de semillas en que cada arquitectura obtiene el mayor valor.

Medida	Explicador	GCN	GraphSAGE	GAT	TAGCN	p (4)	p (3)	Primera
Spearman global	GNNExplainer	0,937	0,931	0,923	0,929	0,532	0,264	GCN 2, GraphSAGE 1
	Shapley	0,996	0,994	0,994	0,991	0,042	0,097	GCN 3
	Expected Gradients	0,923	0,935	0,932	0,920	0,172	0,097	GraphSAGE 2, GAT 1
Spearman $\neq 0$	GNNExplainer	0,691	0,672	0,635	0,630	0,334	0,097	GCN 2, GraphSAGE 1
	Shapley	0,993	0,991	0,990	0,985	0,042	0,097	GCN 3
	Expected Gradients	0,958	0,965	0,960	0,952	0,457	0,264	GAT 1, GraphSAGE 1, GCN 1
Jaccard top-10	GNNExplainer	0,708	0,622	0,650	0,568	0,060	0,097	GCN 2, GAT 1
	Shapley	0,948	0,929	0,927	0,896	0,122	0,264	GCN 2, GraphSAGE 1
	Expected Gradients	0,798	0,800	0,816	0,806	0,532	0,264	GAT 2, GCN 1

6.7. H3: la técnica de compensación no cambia la estabilidad

Con GNNExplainer, ninguna comparación es significativa con ninguna medida: pesos por clase supera a *focal loss* en 6 de los 9 pares con la medida principal ($p = 0{,}51$) y a la entropía sin ajuste en 4 ($p = 1$), y las diferencias medias son menores que 0,025 con las tres medidas. Con Shapley y Expected Gradients aparecen diferencias sistemáticas pero minúsculas: con Shapley, *focal loss* es más estable que pesos por clase en los 9 pares ($p = 0{,}004$), por dos milésimas; con Expected Gradients, la entropía sin ajuste supera a pesos por clase en 8 de 9 ($p = 0{,}039$), por nueve milésimas. El sentido de la diferencia cambia según el explicador, y en la corrida original con GNNExplainer era el contrario (Anexo A).

Veredicto sobre H3. La hipótesis se sostiene en términos prácticos: la técnica de compensación cambia mucho el rendimiento (Sección 6.2) pero mueve la estabilidad de los explicadores de variables en milésimas o pocas centésimas, sin un sentido constante.

PGExplainer. PGExplainer explica aristas y no variables, por lo que se analiza aparte (Tabla 6.8). El grafo de Elliptic es muy disperso: la mediana de aristas en el subgrafo receptivo de las transacciones explicadas es de 2 o 3, y la estabilidad solo se pudo medir, en promedio, en

Tabla 6.7: Estabilidad por técnica de compensación (3 arquitecturas que aprenden, media de los escenarios principales). PC > FL y PC > SA: en cuántos de los 9 pares de arquitectura y semilla pesos por clase supera a *focal loss* o a sin ajuste, con el p de la prueba de signos.

Medida	Explicador	Pesos por clase	Focal loss	Sin ajuste	PC > FL	PC > SA
Spearman global	GNNExplainer	0,928	0,922	0,930	6.0/9 (0,508)	4.0/9 (1,000)
	Shapley	0,993	0,995	0,994	0.0/9 (0,004)	5.0/9 (1,000)
	Expected Gradients	0,929	0,933	0,938	2.0/9 (0,180)	1.0/9 (0,039)
Spearman $\neq 0$	GNNExplainer	0,646	0,626	0,668	5.0/9 (1,000)	2.0/9 (0,180)
	Shapley	0,988	0,992	0,990	1.0/9 (0,039)	4.0/9 (1,000)
	Expected Gradients	0,959	0,960	0,962	4.0/9 (1,000)	4.0/9 (1,000)
Jaccard top-10	GNNExplainer	0,613	0,598	0,625	5.0/9 (1,000)	5.0/9 (1,000)
	Shapley	0,917	0,941	0,922	2.0/9 (0,180)	5.0/9 (1,000)
	Expected Gradients	0,807	0,799	0,805	7.0/9 (0,180)	4.0/9 (1,000)

Tabla 6.8: Estabilidad de PGExplainer sobre las aristas del subgrafo: Spearman entre réplicas y Jaccard de las 20 aristas principales (escenarios principales, media de 3 semillas).

	Pesos por clase		Focal loss		Sin ajuste	
Arquitectura	Spearman	Jaccard	Spearman	Jaccard	Spearman	Jaccard
GCN	0,517	0,630	0,325	0,645	0,322	0,608
GraphSAGE	0,251	0,560	0,064	0,628	0,381	0,658
GAT	0,305	0,621	0,514	0,625	0,458	0,675
TAGCN	0,504	0,769	0,121	0,628	0,288	0,650

13 de las 30 transacciones (las demás tienen menos aristas de las necesarias para comparar órdenes). Con tan pocas aristas, el Spearman entre réplicas es una medida muy gruesa y el Jaccard de las 20 principales compara casi siempre conjuntos que incluyen todas las aristas disponibles. Aun así, el Spearman va de 0,06 a 0,52 según la combinación, sin patrón por arquitectura ni por pérdida. La conclusión es que, en este problema, PGExplainer no produce una explicación de aristas evaluable, y que la variación entre pérdidas no puede atribuirse a la técnica de compensación.

6.8. Estabilidad y rendimiento del modelo

Una pregunta complementaria es si los modelos que detectan mejor producen explicaciones más estables. La Figura 6.8 muestra, para cada uno de los 216 modelos, la estabilidad de GNNExplainer frente a su PR-AUC de validación. Con el Spearman global la relación es positiva: la correlación de rangos es de 0,61 sobre todos los modelos y de 0,68 entre los entrenados con pesos por clase. El rango de estabilidad es, sin embargo, estrecho (de 0,87 a 0,97), y los modelos menos estables son sobre todo los entrenados sin ajuste que colapsaron a predecir la clase negativa. Con el Jaccard top-10 la relación desaparece entre los modelos con pesos por clase (correlación de $-0{,}02$) y solo persiste entre los entrenados sin ajuste (0,59), de nuevo por los modelos colapsados. Con Shapley y Expected Gradients la relación es débil o nula con las tres

medidas (correlaciones entre $-0{,}14$ y $0{,}20$).

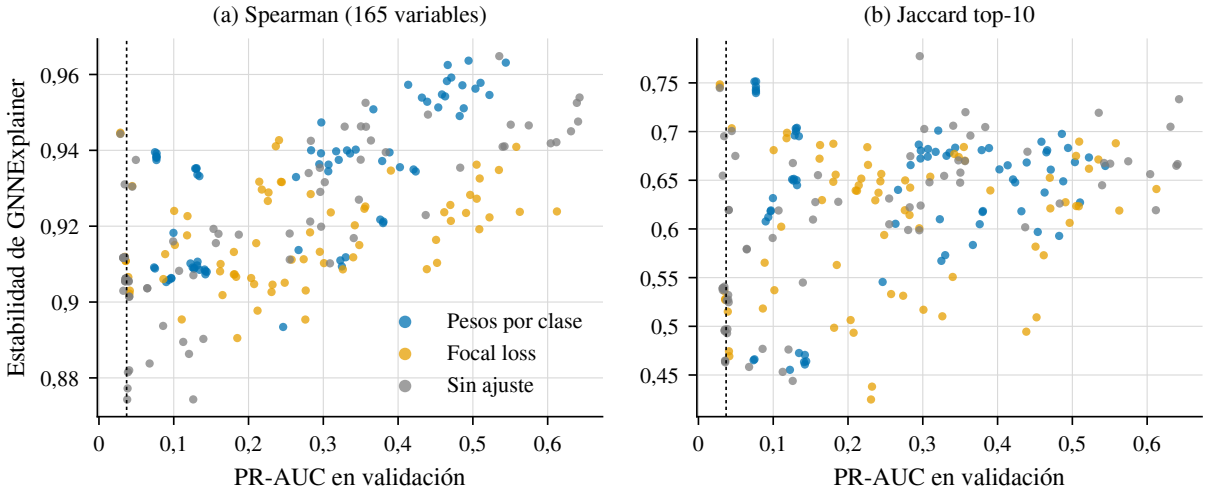


Figura 6.8: Estabilidad de GNNExplainer frente al PR-AUC de validación, un punto por modelo (216 modelos). La línea discontinua es el valor del azar (0,037). Los grupos de puntos casi superpuestos son modelos de una misma arquitectura, pérdida y semilla en distintos escenarios.

Estas correlaciones son descriptivas. Los puntos no son independientes: los modelos de una misma semilla en distintos escenarios forman grupos casi superpuestos en la figura, y una prueba que los contara por separado estaría pseudorreplicada. Lo que sí puede afirmarse es que, con GNNExplainer, un modelo que no aprende tiende a dar explicaciones algo menos estables entre réplicas, y que esa tendencia no se traslada a las variables principales.

6.9. Qué parte de la estabilidad depende del modelo

Las secciones anteriores muestran que la estabilidad apenas cambia con el escenario, la arquitectura o la pérdida. Hay dos explicaciones posibles: que los modelos aprendan casi lo mismo en todas las condiciones, o que la estabilidad medida no dependa de lo aprendido. El control de pesos aleatorios (Sección 5.8) permite distinguirlas (Figura 6.9 y Tabla 6.9).

Tabla 6.9: Control de pesos aleatorios (media de 12 modelos).

	Spearman global			Jaccard top-10		
Explicador	Estab. entrenada	Estab. azar	Entrenada vs azar	Estab. entrenada	Estab. azar	Entrenada vs azar
GNNExplainer	0,924	0,927	0,917	0,636	0,646	0,220
Shapley	0,948	0,920	0,914	0,915	0,904	0,498
Integrated Gradients	1,000	1,000	0,950	1,000	1,000	0,357
Expected Gradients	0,932	0,924	0,724	0,801	0,799	0,220

Con la medida principal, la explicación no depende del modelo. La explicación de GNNExplainer sobre el modelo entrenado coincide con la de una red sin entrenar con un Spearman de 0,917, prácticamente lo mismo que coinciden dos réplicas sobre el mismo modelo (0,924).

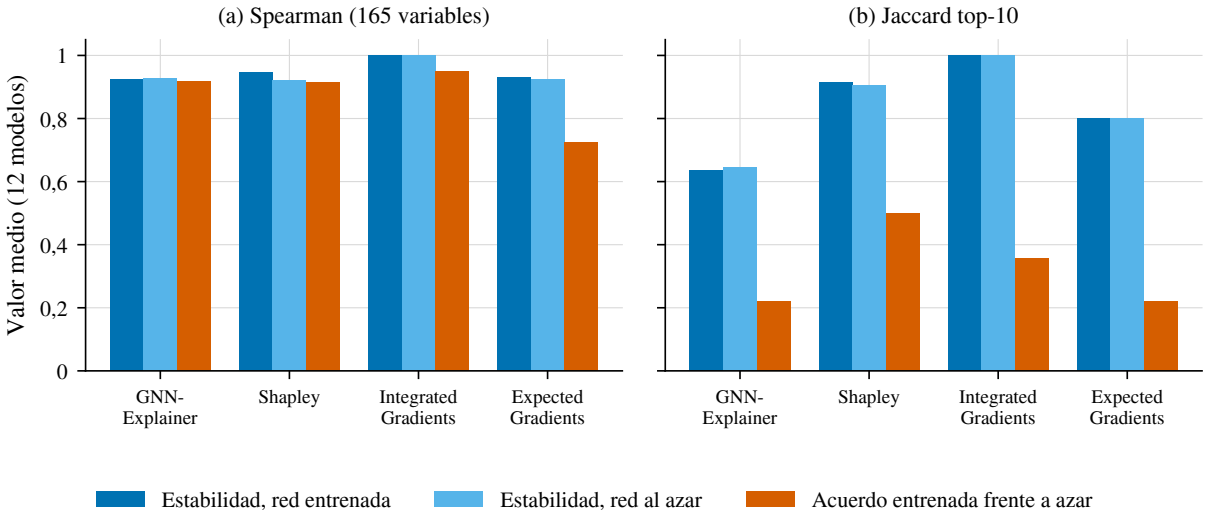


Figura 6.9: Control de pesos aleatorios en el escenario nativo con pesos por clase (12 modelos: 4 arquitecturas por 3 semillas). Para cada explicador, la estabilidad entre réplicas con la red entrenada y con una red de pesos aleatorios, y el acuerdo entre la explicación de la red entrenada y la de la red aleatoria.

Lo mismo ocurre con Shapley (0,914 frente a 0,948) e Integrated Gradients (0,950). Una red con pesos aleatorios, que no aprendió nada, recibe además explicaciones igual de estables que la entrenada (0,927 en GNNExplainer). El orden de Integrated Gradients coincide con el orden de las variables por su valor absoluto con un Spearman de 0,97. La medida principal está dominada por el bloque de variables en cero (Sección 5.7), que ocupa las mismas posiciones del orden con cualquier red. Por comparación, dos explicaciones que solo coincidieran en ese bloque y ordenaran el resto al azar tendrían un Spearman de 0,81 (Sección 5.7): la estabilidad de GNNExplainer, entre 0,92 y 0,94, está poco por encima de ese piso.

Con el Jaccard top-10, sí depende. Las 10 variables principales de la red entrenada y de la aleatoria comparten en promedio un índice de 0,22 con GNNExplainer y Expected Gradients, 0,36 con Integrated Gradients y 0,50 con Shapley, muy por debajo de la estabilidad entre réplicas (0,64, 0,80, 1 y 0,92). Las variables que el analista vería primero sí cambian con los pesos del modelo. Expected Gradients es el único explicador cuyo acuerdo con la red aleatoria también es claramente menor que su estabilidad con la medida principal (0,724 frente a 0,932), porque sus referencias no son el vector cero y su atribución no se anula en las variables nulas.

Modelos igual de buenos dan explicaciones distintas. El acuerdo entre explicaciones de modelos distintos refuerza esta lectura. Con GNNExplainer y pesos por clase, dos semillas de la misma arquitectura y el mismo escenario coinciden con un Spearman global de 0,929, pero con un Jaccard top-10 de solo 0,30: comparten unas 5 de sus 10 variables principales. Cambiar de escenario manteniendo la semilla mueve menos la explicación (Jaccard 0,60) que cambiar de semilla manteniendo el escenario. La principal fuente de variación de la explicación no es el desbalance, sino la aleatoriedad del entrenamiento.

La Figura 6.10 resume esa comparación para los tres explicadores estocásticos. Con el Spearman global todas las barras están por encima de 0,7 y casi no se distinguen; con el Jaccard top-10 aparece un escalón claro entre las comparaciones con la misma semilla y las comparaciones entre semillas.

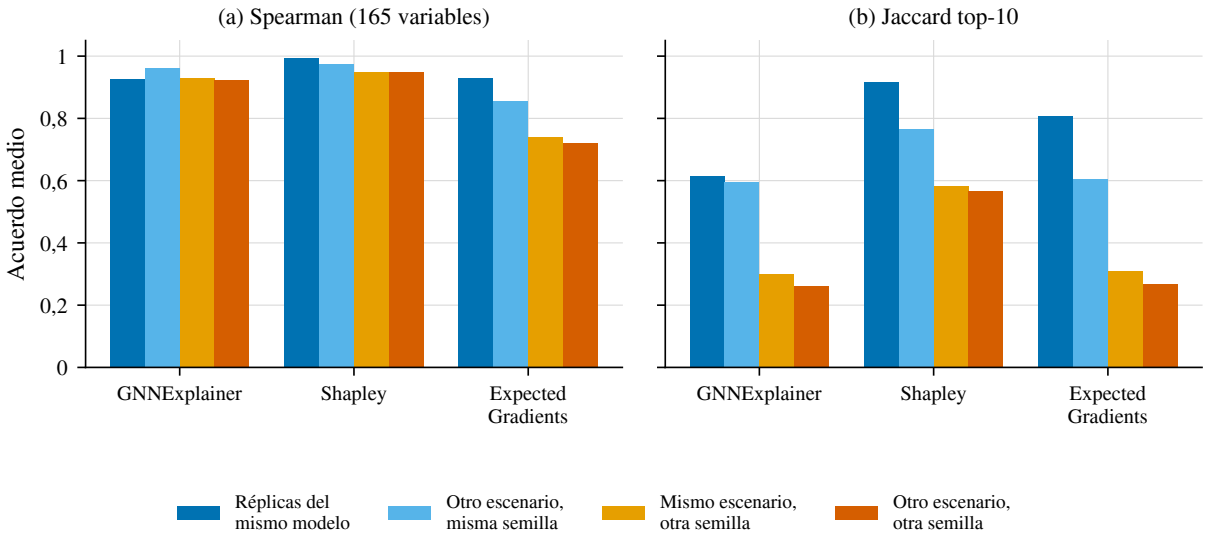


Figura 6.10: Acuerdo entre explicaciones según qué cambia entre los dos modelos comparados (GraphSAGE, GAT y TAGCN; pesos por clase; escenarios principales). Réplicas del mismo modelo es la estabilidad entre réplicas.

Arquitecturas distintas, explicaciones distintas. Comparando arquitecturas con la misma semilla, escenario y pérdida (Tabla 6.10), el Spearman global sigue siendo alto, entre 0,71 y 0,95 según el explicador, pero el Jaccard top-10 cae a valores entre 0,21 y 0,58. Con GNNExplainer, dos arquitecturas comparten en promedio unas 4 de sus 10 variables principales. Las arquitecturas no difieren en estabilidad, pero sí en qué variables señalan sus explicaciones.

Tabla 6.10: Acuerdo entre las explicaciones de dos arquitecturas con la misma semilla, escenario y pérdida (pesos por clase, escenarios principales). Rango sobre los seis pares de arquitecturas.

Explicador	Spearman global	Spearman $\neq 0$	Jaccard top-10
GNNExplainer	0,88 a 0,92	0,58 a 0,69	0,21 a 0,29
Shapley	0,83 a 0,95	0,80 a 0,91	0,50 a 0,58
Integrated Gradients	0,95	0,76 a 0,77	0,37 a 0,42
Expected Gradients	0,71 a 0,74	0,51 a 0,56	0,26 a 0,31

Los explicadores no coinciden entre sí. Sobre el mismo modelo, los cuatro explicadores de variables señalan conjuntos de variables principales muy distintos (Figura 6.11): con el Jaccard top-10 el acuerdo va de 0,10 (GNNExplainer frente a Shapley) a 0,49 (Integrated frente a Expected Gradients). El alto Spearman global entre GNNExplainer e Integrated Gradients (0,84) desaparece con el Jaccard top-10 (0,29), lo que indica que su parecido se debe sobre todo al bloque de variables nulas.

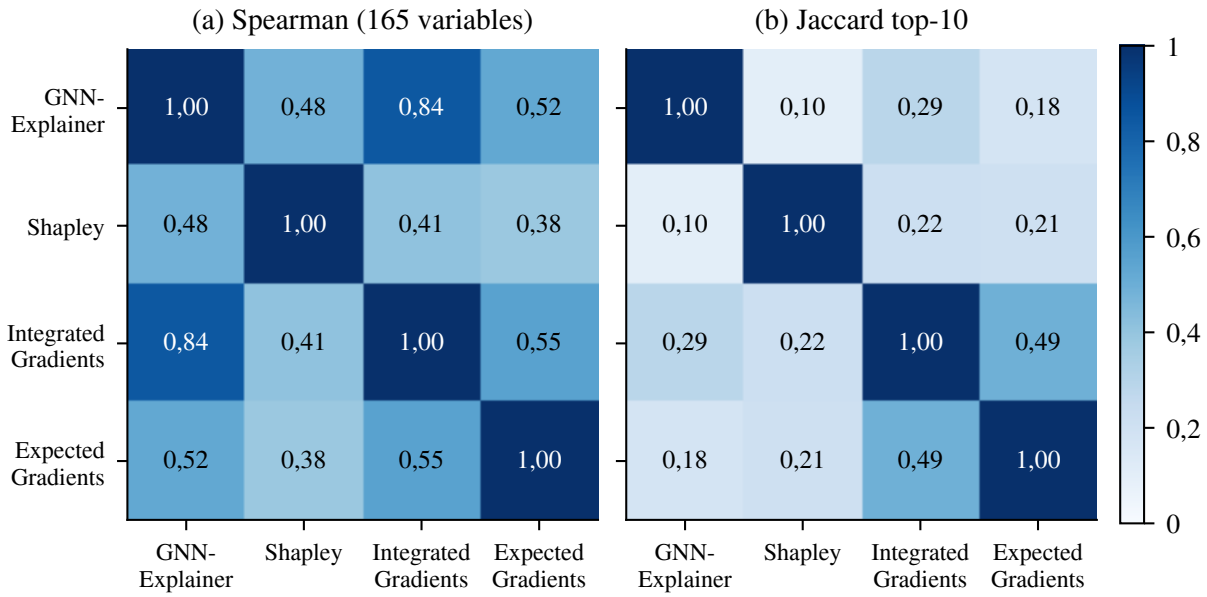


Figura 6.11: Acuerdo entre explicadores sobre el mismo modelo (GraphSAGE, GAT y TAGCN; escenarios principales; todas las pérdidas y semillas), con el Spearman global (a) y el Jaccard top-10 (b).

6.10. Un ejemplo: la explicación de una transacción

Las cifras anteriores son promedios sobre 30 transacciones. La Figura 6.12 muestra lo que significan para una sola, la primera del conjunto explicado, que GraphSAGE con pesos por clase en el escenario nativo clasifica correctamente como ilícita. Las variables se nombran L1 a L93 (las 93 variables propias de la transacción, en el orden del conjunto de datos) y A1 a A72 (las 72 agregadas de sus vecinas).

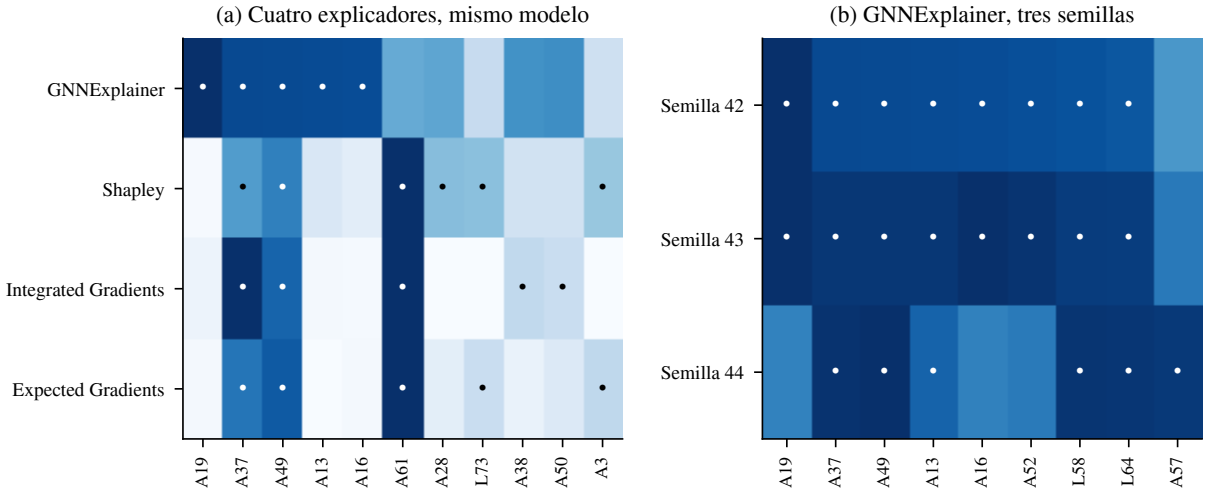


Figura 6.12: Importancia normalizada (máximo igual a 1) de las variables más importantes para una transacción ilícita de validación. (a) Los cuatro explicadores de variables sobre el mismo modelo (GraphSAGE, pesos por clase, nativo, semilla 42). (b) GNNExplainer sobre los modelos de las tres semillas. Un punto marca las variables que están entre las 10 principales de esa fila.

Dos rasgos de los resultados agregados se ven en este caso. Primero, los explicadores no coinciden (panel a). GNNExplainer reparte una importancia alta entre muchas variables: 84 de las 165 reciben más de la mitad de la importancia máxima, y 8 de sus 10 principales son variables agregadas. Los otros tres concentran la importancia en pocas variables, y no en las mismas: de las 10 principales, GNNExplainer comparte 2 con Shapley, 5 con Integrated Gradients y 2 con Expected Gradients; el par que más coincide es Shapley con Expected Gradients, con 6. Segundo, el mismo explicador sobre modelos de semillas distintas cambia parte de sus variables principales (panel b): los modelos de las semillas 42 y 43 comparten 8 de sus 10 variables principales, pero el de la semilla 44 comparte solo 5 con cada uno. Un analista que recibiera la alerta de uno u otro modelo, que solo difieren en la semilla del entrenamiento (sus PR-AUC de validación son 0,27, 0,38 y 0,33), leería explicaciones distintas para la misma transacción.

6.11. Costo computacional de los explicadores

La Tabla 6.11 muestra el tiempo que tarda cada explicador en producir las cinco réplicas de la explicación de una transacción. Los explicadores de gradiente son entre 40 y más de 100 veces más rápidos que GNNExplainer y Shapley, aun ejecutándose en CPU: calculan sus gradientes en lote, mientras que GNNExplainer optimiza una máscara durante 100 iteraciones y Shapley evalúa el modelo sobre muchas combinaciones de variables. PGExplainer es el más costoso, porque incluye el entrenamiento de su red explicadora. Para un sistema que deba explicar muchas alertas, la diferencia de costo es relevante, y debe sopesarse con lo que muestra la Sección 6.9: los explicadores rápidos no son menos estables.

Tabla 6.11: Mediana de segundos por transacción explicada (cinco réplicas) sobre todos los modelos. GNNExplainer, Shapley y PGExplainer se ejecutaron en GPU; los dos de gradiente, en CPU.

Explicador	GCN	GraphSAGE	GAT	TAGCN	Todos
GNNExplainer	1,02	0,92	1,31	1,54	1,26
PGExplainer	2,72	2,49	3,38	4,15	3,33
Shapley	0,76	0,72	0,80	0,85	0,80
Integrated Gradients	0,01	0,01	0,01	0,01	0,01
Expected Gradients	0,01	0,01	0,02	0,03	0,02

6.12. Síntesis

La Tabla 6.12 resume el contraste de las hipótesis.

Los tres veredictos tienen una lectura común que el control de pesos aleatorios aclara: con la medida principal, la estabilidad refleja sobre todo la estructura de los datos de entrada y el explicador, no lo que el modelo aprendió, y por eso responde poco a los factores del diseño. El efecto

Tabla 6.12: Veredicto de las hipótesis con la partición sin el cierre.

Hipótesis	Resultado	Veredicto
H1: la estabilidad baja con el desbalance	Con pesos por clase, sin tendencia y con equivalencia entre nativo y 1:200. Con *focal loss* y sin ajuste, GNNExplainer sí baja con las tres medidas	Se sostiene en parte: solo GNNExplainer y solo si la pérdida no compensa el desbalance
H2: la estabilidad depende de la arquitectura	Sin orden repetido entre GraphSAGE, GAT y TAGCN; GCN, que no aprende, es la más estable	No se sostiene
H3: la compensación no cambia la estabilidad	Diferencias de milésimas o pocas centésimas, con sentido que cambia según el explicador	Se sostiene en términos prácticos

que sí aparece, la caída de GNNExplainer con el desbalance cuando la pérdida no lo compensa, acompaña a la caída del rendimiento de esos mismos modelos. Las 10 variables principales, en cambio, sí dependen del modelo, y varían más entre semillas que entre escenarios.

Capítulo 7

Discusión

Los resultados del Capítulo 6 se resumen en una observación: la técnica de compensación del desbalance cambia mucho el rendimiento de los modelos, y la estabilidad de sus explicaciones responde poco a los factores del diseño. La arquitectura no la cambia, y el desbalance solo la reduce con GNNExplainer y cuando la pérdida no lo compensa, a la par que se degrada el modelo. El control de pesos aleatorios muestra por qué la respuesta es tan débil: con la correlación de Spearman sobre todas las variables, la estabilidad refleja sobre todo los datos de la transacción y el propio explicador. Este capítulo interpreta cada resultado, lo contrasta con la literatura y discute sus consecuencias y sus límites.

7.1. Cuándo el desbalance cambia la estabilidad

La hipótesis H1 esperaba que, con menos ejemplos ilícitos en proporción, el modelo aprendiera peor la clase rara y sus explicaciones sobre ella fueran menos estables. La literatura sobre datos tabulares apoyaba esa expectativa: Chen et al. encontraron que las explicaciones de LIME y SHAP pierden estabilidad a medida que aumenta el desbalance en datos de crédito [26], y Rai et al. que el desbalance puede afectar de forma significativa la confiabilidad y la consistencia de LIME y SHAP en datos clínicos [78]. En Elliptic el resultado depende de la pérdida. Con *focal loss* y sin ajuste, la estabilidad de GNNExplainer baja con el desbalance, en la dirección que encontraron esos estudios. Con pesos por clase no cambia. Cuatro razones, que no son excluyentes, explican por qué con pesos por clase el desbalance no tiene efecto, y por qué sí lo tiene con las otras pérdidas.

Los escenarios principales no quitan fraude. Entre 1:10 y el nativo, el entrenamiento conserva siempre las 2.871 transacciones ilícitas; lo que cambia es cuántas negativas las acompañan. El modelo dispone de la misma información sobre la clase rara en todos esos escenarios.

Con pesos por clase, la proporción desaparece de la pérdida. La pérdida con pesos por clase

(Sección 3.4) es la media de las dos medias por clase, cada una con peso un medio. La razón entre clases no aparece en la expresión: el gradiente que recibe el modelo de la clase ilícita es el mismo con 28.710 negativas que con 117.933. Lo que cambia es la estimación de la media de las negativas, que es más ruidosa con menos ejemplos. Es coherente que el modelo aprenda casi lo mismo en todos los escenarios y que, como se observó, la explicación de un modelo cambie menos entre escenarios con la misma semilla (Jaccard top-10 de 0,60) que entre semillas en el mismo escenario (0,30). Con la entropía sin ajuste ocurre lo contrario: la clase ilícita pesa cada vez menos a medida que crece el desbalance, el modelo se degrada hasta predecir casi siempre la clase negativa, y GNNExplainer, que optimiza una máscara sobre la salida de ese modelo, pierde estabilidad con él. La *focal loss* queda en un punto intermedio, en rendimiento y en estabilidad. El desbalance actúa, entonces, a través de la degradación del modelo, y una pérdida que lo compensa por completo corta esa vía.

Con pesos por clase, los escenarios de estrés quitan fraude y aun así no cambian la estabilidad. En 1:100 y 1:200 se entrena con 1.179 y 589 ilícitas, y con pesos por clase la estabilidad tampoco baja, porque el rendimiento tampoco se derrumba. La única excepción parcial apareció en la partición original: con el Spearman sobre las variables distintas de cero, GNNExplainer bajaba de 0,753 en el nativo a 0,717 y 0,708 en 1:100 y 1:200 (Anexo A). Esa caída, de unas cuatro centésimas, no se repite sin el cierre, donde el escenario menos estable con esa medida es el nativo. Esto descarta que la ausencia de efecto se deba solo a que la información sobre la clase rara no cambió. Lo que el control de pesos aleatorios sugiere es que la estabilidad entre réplicas, tal como se mide aquí, depende poco del modelo: si con una red sin entrenar el explicador es igual de estable, un modelo entrenado con menos ejemplos ilícitos no tiene por qué producir explicaciones menos estables.

Qué varía en la medida. Una diferencia de diseño puede explicar parte del contraste con los estudios tabulares. Esta tesis mide la estabilidad de un explicador sobre un modelo fijo: las cinco réplicas cambian la semilla del explicador, no el modelo. Cuando la evaluación compara explicaciones de modelos entrenados con distintas muestras de los datos, la variación del entrenamiento entra en la medida. Los resultados de esta tesis son compatibles con esa lectura: entre semillas, que es una forma de variación del entrenamiento, el acuerdo de las 10 variables principales cae a 0,30. Si el desbalance afectara la estabilidad sobre todo a través de la variabilidad del modelo, una medida entre réplicas no lo detectaría. Comprobar esta explicación exige medir la variación entre modelos entrenados con distintas submuestras en cada nivel de desbalance, lo que queda como trabajo futuro.

7.2. Arquitectura y estabilidad

La hipótesis H2 esperaba diferencias entre arquitecturas, porque combinan de forma distinta a la transacción con sus vecinas. Entre las que aprenden no se encontraron, y el orden que sugería la corrida original, con GraphSAGE primera, no se repitió al cambiar la partición. Dos resultados permiten interpretar esta ausencia.

El primero es el análisis exploratorio. En Elliptic, el 70 % de las transacciones ilícitas no tiene ninguna vecina ilícita, y en la tarea de esta tesis una ilícita está rodeada sobre todo de negativas (Sección 5.2). La información útil está en la propia transacción. Las arquitecturas que le dan un peso propio (GraphSAGE, GAT y TAGCN) aprenden; la que la promedia con sus vecinas sin peso propio (GCN) no lo logra. Esta es una manifestación del problema de la heterofilia descrito en la literatura [48], aunque aquí la mezcla de clases proviene en parte de contar como negativas las transacciones sin etiqueta. Si las arquitecturas que aprenden se apoyan sobre todo en las variables propias de la transacción, es razonable que sus explicaciones de variables se parezcan en estabilidad.

El segundo es la paradoja de GCN: la arquitectura que no aprende produce las explicaciones más estables con GNNExplainer en dos de las tres semillas. Una explicación posible, que esta tesis no comprueba, es que un modelo que apenas distingue clases tiene una función de salida más simple, y que un explicador que busca una máscara sobre ella converge más fácilmente al mismo resultado. Esta observación es la que motivó el control de pesos aleatorios y la que sostiene la conclusión central de la tesis: la estabilidad, sola, no certifica que una explicación diga algo sobre lo aprendido.

Longa et al. encontraron que la explicabilidad depende de la arquitectura en sus experimentos [15]. Los resultados de esta tesis no contradicen ese hallazgo: se refieren solo a la estabilidad entre réplicas, y en un grafo en el que la estructura aporta poco a la predicción. Es posible que en grafos donde la información de las vecinas es decisiva las diferencias entre arquitecturas se hagan visibles también en la estabilidad.

7.3. La técnica de compensación

La técnica de compensación es el factor con mayor efecto sobre el rendimiento. Sobre la estabilidad su efecto directo es mínimo, aunque, como se vio, decide si el desbalance la afecta. Sobre el rendimiento, dos resultados merecen comentario.

El primero es que la mejor pérdida depende del escenario, y que la afirmación, sostenida en versiones anteriores de este trabajo, de que los pesos por clase detectan mejor que la *focal loss* no es general: en el escenario nativo de la partición sin el cierre, la *focal loss* la iguala en GraphSAGE y la supera en TAGCN. La diferencia de magnitud entre ambas pérdidas ayuda a

entenderlo. Con $\alpha = 0{,}75$, la *focal loss* da a cada ilícita tres veces el peso de una negativa; con pesos por clase, la razón es 41 a 1 en el nativo. Pero la *focal loss* además atenúa los ejemplos que el modelo ya clasifica bien, que son casi todas las negativas; una explicación plausible, no comprobada aquí, es que en el nativo esa atenuación compense la diferencia de pesos. Cuando se quitan ilícitas (1:100 y 1:200), la compensación ya no alcanza y solo los pesos por clase mantienen la detección, en las dos particiones.

El segundo es la fragilidad de la entropía sin ajuste: funciona bien en 1:10, donde la clase rara ya pesa una undécima parte del total, y cae al nivel del azar a partir de cierto desbalance, porque predecir siempre la clase negativa minimiza la pérdida. GAT sin ajuste es la excepción, lo que sugiere que su mecanismo de atención o sus hiperparámetros la protegen en parte de ese colapso; los datos de esta tesis no permiten distinguir entre ambas causas.

Sobre la estabilidad, las diferencias entre pérdidas son de milésimas o pocas centésimas y su sentido depende del explicador. Con Shapley, la *focal loss* es más estable que los pesos por clase en los 9 pares, pero por dos milésimas. Una diferencia sistemática pero tan pequeña no tiene consecuencias para el uso de las explicaciones. La hipótesis H3 se sostiene en el sentido práctico en que fue formulada.

7.4. Qué mide la estabilidad de una explicación

El hallazgo con más consecuencias metodológicas es el del control de pesos aleatorios. Con la correlación de Spearman sobre las 165 variables, la explicación de un modelo entrenado coincide con la de una red sin entrenar casi tanto como dos réplicas sobre el mismo modelo. Es la misma situación que Adebayo et al. describieron para mapas de saliencia de imágenes, en los que explicaciones visualmente convincentes apenas cambiaban al aleatorizar los pesos [29].

La causa, en este caso, es concreta y se puede cuantificar. Después del escalado, cerca del 44 % de las variables de una transacción vale cero, y los cuatro explicadores de variables les asignan importancia nula o casi nula por la forma en que están definidos. Ese bloque ocupa los últimos rangos en cualquier explicación de cualquier modelo, y la correlación de Spearman cuenta ese acuerdo. Además, en Integrated Gradients la atribución es el producto de la variable por un gradiente, y el orden resultante coincide en 0,97 con el orden de las variables por su valor absoluto: lo fija sobre todo la entrada.

Con el Jaccard de las 10 variables principales, en cambio, la explicación del modelo entrenado y la de la red aleatoria comparten solo una fracción de sus variables, mucho menos que dos réplicas. Las variables que un analista leería primero sí dependen del modelo. Esto lleva a una recomendación concreta para quien evalúe la estabilidad de explicaciones de variables: reportar, junto con la correlación de rangos completa, una medida sobre las k variables principales y un

control de pesos aleatorios. Sin el control, una estabilidad alta puede deberse a la estructura de los datos y no al modelo; sin una medida de las principales, el bloque de variables nulas puede ocultar desacuerdos en lo que importa.

7.5. Implicaciones para la auditoría

Los resultados tienen tres consecuencias para el uso de explicaciones en un sistema de detección de lavado de dinero.

La estabilidad entre réplicas no basta como garantía. Que un explicador devuelva lo mismo al repetirse es necesario, pero, como muestra el control, puede lograrse sin que la explicación dependa de lo aprendido. Una política de auditoría que exija explicaciones estables debería exigir también pruebas de que dependen del modelo.

La mayor fuente de variación es el entrenamiento. Dos modelos que difieren solo en la semilla comparten en promedio unas cinco de sus diez variables principales para la misma transacción. Si una institución reentrena su modelo, las explicaciones de sus alertas cambiarán aunque los datos y el rendimiento no cambien. Una forma de mitigarlo es explicar con varios modelos entrenados con semillas distintas y reportar las variables en las que coinciden.

El explicador importa más que la arquitectura o el desbalance. Los cuatro explicadores de variables señalan conjuntos distintos sobre el mismo modelo, con acuerdos del top-10 entre 0,10 y 0,49. La elección del explicador determina qué variables verá el analista más que cualquiera de los factores del diseño, y esa elección debería documentarse y justificarse. El eje sintético (Anexo B) añade que el explicador más estable no es el que señala las variables correctas: Shapley es el más estable en tres de las cuatro arquitecturas y su plausibilidad no supera al azar.

7.6. El cierre del mercado y la evaluación en Elliptic

La partición temporal habitual de Elliptic deja la prueba completa después del cierre del mercado, y con ella ningún modelo de esta tesis supera al azar. Alarab y Prakoonwit señalaron que ningún algoritmo había logrado capturar ese evento con las variables originales [40]. Para un estudio cuyo objetivo es comparar modelos o explicaciones, y no la adaptación a un cambio del mercado, una prueba en la que todos están al nivel del azar no discrimina. La partición sin el cierre recupera una prueba informativa a costa de 591 ilícitas de entrenamiento.

La lección para la literatura es que los resultados de prueba sobre Elliptic deberían reportarse indicando si incluyen los pasos posteriores al cierre, y que un rendimiento alto en la partición habitual merece examinarse con cuidado. También muestra el valor de reportar el PR-AUC

junto a la prevalencia: en la corrida original el ROC-AUC de GraphSAGE en prueba era 0,711, una cifra que parece aceptable, mientras su PR-AUC era 0,010, prácticamente el azar.

7.7. Lecciones metodológicas del proceso

Los resultados de esta tesis son los de una segunda versión completa del estudio. La primera versión tenía errores que cambiaban sus conclusiones, y vale la pena registrarlos porque son errores fáciles de cometer en este tipo de trabajo.

Etiquetas invertidas. En la primera versión, la conversión de las etiquetas de Elliptic trató las 157.205 transacciones sin etiqueta como lícitas y dejó fuera las 42.019 lícitas reales. Los modelos aprendieron una tarea distinta de la descrita. La versión actual verifica los conteos oficiales de cada clase y de cada partición antes de entrenar, y se detiene si no coinciden.

Una medida truncada. La primera implementación de la correlación de Spearman entre explicaciones dimensionaba el vector de rangos por el número de variables principales y no por el número real de variables, de modo que descartaba casi todas. El error favorecía a una arquitectura y bastaba para invertir el orden de estabilidad entre arquitecturas.

Explicadores mal configurados. Los valores por defecto de PGExplainer producían máscaras colapsadas o pérdidas no numéricas en Elliptic, y su entrenamiento usaba los primeros nodos del entrenamiento, entre los que no había ninguna ilícita (Sección 5.6).

Pseudorreplicación. Una versión intermedia de las pruebas trató cada combinación de semilla y escenario como un bloque independiente y obtuvo diferencias muy significativas entre arquitecturas. Los modelos de una misma semilla en distintos escenarios resultaron casi idénticos, y con la unidad correcta esas diferencias desaparecieron [67, 68].

Una prueba sin información. La partición habitual de Elliptic dejaba la prueba después del cierre del mercado. La consecuencia no era un error de código sino de diseño: todas las comparaciones de rendimiento en prueba eran comparaciones entre modelos al nivel del azar.

La lección común es que, en un estudio de esta naturaleza, la mayor parte del riesgo no está en el modelo sino en la medición: en cómo se definen las etiquetas, cómo se calcula una medida, cómo se configura un explicador y qué se toma como unidad independiente. Cada uno de estos errores producía resultados con apariencia razonable.

7.8. Limitaciones

- **La partición la escogieron los autores.** Los cortes en los pasos 29 y 35 no responden a un criterio externo, y otra partición anterior al cierre podría dar cifras de rendimiento

algo distintas. Las conclusiones sobre estabilidad, en cambio, se repiten con la partición original.

- **Tres semillas.** La comparación entre arquitecturas usa las semillas como bloques y tiene poca potencia. La ausencia de diferencias no prueba la igualdad; muestra que no hay un orden que se repita.
- **Treinta transacciones.** Se explican 30 ilícitas de validación, las mismas para todos los modelos. Es suficiente para comparar condiciones, pero no describe la variedad completa de las ilícitas.
- **Un solo conjunto de datos, con variables anónimas.** Los resultados se refieren a Elliptic. Sin el significado de las variables no se puede juzgar si una explicación es correcta; el eje sintético aborda esa pregunta solo de forma indirecta.
- **Presupuesto de búsqueda reducido.** Ocho ensayos por combinación, igual para todas las arquitecturas, y GAT limitada a 2 capas por memoria. Con más presupuesto el rendimiento podría mejorar, en especial el de GAT.
- **El filtro de calidad usa un umbral fijo de 0,5.** Por eso GAT con pesos por clase no lo supera aunque su PR-AUC muestre que aprende. Un filtro sobre el PR-AUC habría sido más adecuado bajo desbalance.
- **PGExplainer no es evaluable en este grafo.** Con 2 o 3 aristas en el subgrafo mediano, su estabilidad de aristas solo se pudo medir en una parte de las transacciones, y con medidas muy gruesas.
- **No determinismo del cálculo en GPU.** Repetir el entrenamiento reproduce las conclusiones, pero no necesariamente todos los decimales.

Capítulo 8

Conclusiones

Esta tesis estudió la estabilidad de las explicaciones que producen cinco métodos de explicabilidad sobre redes neuronales de grafos entrenadas para detectar transacciones ilícitas en Elliptic, variando el nivel de desbalance de clases, la arquitectura y la técnica de compensación, con 216 modelos y tres semillas de modelo. Este capítulo responde las preguntas de investigación, resume los aportes y señala el trabajo futuro.

8.1. Respuestas a las preguntas de investigación

¿Disminuye la estabilidad de las explicaciones a medida que el desbalance se hace más extremo? Depende de la pérdida y del explicador. Con pesos por clase, la estabilidad de los cuatro explicadores de variables no muestra ninguna tendencia entre 1:10 y 1:200, y la diferencia entre el escenario real y el más extremo es menor que 0,05. Con *focal loss* y, sobre todo, sin ajuste, la estabilidad de GNNExplainer baja con el desbalance con las tres medidas, a la par que baja el rendimiento de esos modelos. Shapley y Expected Gradients apenas responden. La hipótesis H1 se sostiene solo en parte: el desbalance afecta la estabilidad a través de la degradación del modelo, y una pérdida que lo compensa por completo evita el efecto.

¿Producen las distintas arquitecturas explicaciones con distinta estabilidad? No entre las que aprenden. GraphSAGE, GAT y TAGCN quedan dentro de dos centésimas con la medida principal, sin un orden que se repita entre semillas ni entre particiones. GCN no aprende en este problema, porque la señal del fraude está en la propia transacción y GCN la promedia con sus vecinas, y aun así obtiene las explicaciones más estables. La hipótesis H2 no se sostiene.

¿Cambia la estabilidad según la técnica de compensación? En la práctica no. La técnica de compensación es el factor que más cambia el rendimiento (la mejor pérdida depende del escenario y, con poco fraude, solo los pesos por clase mantienen la detección), pero mueve la estabilidad en milésimas o pocas centésimas, con un sentido que cambia según el explicador.

La hipótesis H3 se sostiene.

¿Refleja la estabilidad lo que el modelo aprendió? Con la correlación de Spearman sobre todas las variables, no: la explicación de un modelo entrenado se parece a la de una red sin entrenar casi tanto como dos réplicas entre sí, porque cerca de la mitad de las variables de cada transacción vale cero y todos los explicadores la dejan al final del orden. Las diez variables principales, en cambio, sí dependen del modelo, y varían más entre semillas del entrenamiento que entre niveles de desbalance.

8.2. Aportes

1. **Evidencia sobre la estabilidad de explicaciones de GNN en un problema real de lavado de dinero**, con un diseño factorial replicado y pruebas que respetan la unidad independiente: la arquitectura no determina la estabilidad, y el desbalance solo la reduce cuando la pérdida no lo compensa y a través de la degradación del modelo.

2. **Un artefacto de medida y su corrección.** El bloque de variables nulas infla la correlación de Spearman entre explicaciones. La tesis lo cuantifica, propone dos medidas que no dependen de él y muestra con un control de pesos aleatorios que la estabilidad alta no garantiza que la explicación dependa del modelo.

3. **Una partición de Elliptic que recupera una prueba informativa**, y la evidencia de que la partición habitual deja la prueba después del cierre del mercado, donde ningún modelo supera al azar.

4. **Recomendaciones para auditar explicaciones**: acompañar la estabilidad de un control de pesos aleatorios y de una medida sobre las variables principales, explicar con varios modelos entrenados con semillas distintas, y documentar la elección del explicador, que es lo que más cambia lo que ve el analista.

8.3. Trabajo futuro

- Medir la variación de las explicaciones entre modelos entrenados con distintas submuestras en cada nivel de desbalance, para comprobar si el desbalance afecta la estabilidad a través de la variabilidad del modelo y no del explicador.

- Repetir el estudio en conjuntos de datos de transacciones donde la información de las vecinas sea más decisiva, para ver si allí aparecen diferencias entre arquitecturas.

- Estudiar explicaciones de consenso entre semillas y entre explicadores como forma de producir explicaciones más reproducibles para un analista.

- Evaluar explicadores de subgrafos o de caminos, más adecuados que los de aristas individuales en un grafo tan disperso como Elliptic.
- Sustituir el filtro de calidad basado en un umbral fijo por uno basado en el PR-AUC.

8.4. Reflexión final

El objetivo de explicar un modelo de detección de lavado de dinero es que una persona pueda entender y defender una alerta. Esta tesis muestra que, antes de preguntar si una explicación es correcta, conviene preguntar si es del modelo. Una explicación puede repetirse con precisión y, aun así, decir más sobre los datos de entrada y sobre el método que la produjo que sobre lo que el modelo aprendió. Medirlo exige poco: una red sin entrenar como referencia y una medida que mire lo que el analista lee primero. Incorporarlo a la evaluación de explicaciones es, quizá, la lección más útil de este trabajo para quienes construyan sistemas explicables en entornos regulados.

Referencias

[1] Zhiyuan Chen, Le Dinh Van Khoa, Ee Na Teoh, Amril Nazir, Ettikan Kandasamy Karuppiah y Kim Sim Lam. "Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: a review". En: *Knowledge and Information Systems* 57.2 (2018), págs. 245-285. DOI: 10.1007/s10115-017-1144-z.

[2] Dattatray Vishnu Kute, Biswajeet Pradhan, Nagesh Shukla y Abdullah Alamri. "Deep Learning and Explainable Artificial Intelligence Techniques Applied for Detecting Money Laundering–A Critical Review". En: *IEEE Access* 9 (2021), págs. 82300-82317. DOI: 10.1109/access.2021.3086230.

[3] F. Scarselli, M. Gori, Ah Chung Tsoi, M. Hagenbuchner y G. Monfardini. "The Graph Neural Network Model". En: *IEEE Transactions on Neural Networks* 20.1 (2009), págs. 61-80. DOI: 10.1109/tnn.2008.2005605.

[4] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang y Philip S. Yu. "A Comprehensive Survey on Graph Neural Networks". En: *IEEE Transactions on Neural Networks and Learning Systems* 32.1 (2021), págs. 4-24. DOI: 10.1109/tnnls.2020.2978386.

[5] Soroor Motie y Bijan Raahemi. "Financial fraud detection using graph neural networks: A systematic review". En: *Expert Systems with Applications* 240 (2024), pág. 122156. DOI: 10.1016/j.eswa.2023.122156.

[6] Cynthia Rudin. "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead". En: *Nature Machine Intelligence* 1.5 (2019), págs. 206-215. DOI: 10.1038/s42256-019-0048-x.

[7] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser et al. "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI". En: *Information Fusion* 58 (2020), págs. 82-115. DOI: 10.1016/j.inffus.2019.12.012.

[8] Hao Yuan, Haiyang Yu, Shurui Gui y Shuiwang Ji. "Explainability in Graph Neural Networks: A Taxonomic Survey". En: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2022), págs. 1-19. DOI: 10.1109/tpami.2022.3204236.

[9] Mark Weber, Giacomo Domeniconi, Jie Chen et al. *Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics*. KDD Workshop on Anomaly Detection in Finance. 2019. arXiv: 1908.02591. URL: https://arxiv.org/abs/1908.02591.

[10] Haibo He y E.A. Garcia. “Learning from Imbalanced Data”. En: *IEEE Transactions on Knowledge and Data Engineering* 21.9 (2009), págs. 1263-1284. DOI: 10.1109/tkde.2008.239.

[11] Justin M. Johnson y Taghi M. Khoshgoftaar. “Survey on deep learning with class imbalance”. En: *Journal of Big Data* 6.1 (2019), pág. 27. DOI: 10.1186/s40537-019-0192-5.

[12] N. V. Chawla, K. W. Bowyer, L. O. Hall y W. P. Kegelmeyer. “SMOTE: Synthetic Minority Oversampling Technique”. En: *Journal of Artificial Intelligence Research* 16 (2002), págs. 321-357. DOI: 10.1613/jair.953.

[13] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He y Piotr Dollar. “Focal Loss for Dense Object Detection”. En: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.2 (2020), págs. 318-327. DOI: 10.1109/tpami.2018.2858826.

[14] Chirag Agarwal, Owen Queen, Himabindu Lakkaraju y Marinka Zitnik. “Evaluating explainability for graph neural networks”. En: *Scientific Data* 10.1 (2023), pág. 144. DOI: 10.1038/s41597-023-01974-x.

[15] Antonio Longa, Steve Azzolin, Gabriele Santin et al. “Explaining the Explainers in Graph Neural Networks: a Comparative Study”. En: *ACM Computing Surveys* 57.5 (2025), págs. 1-37. DOI: 10.1145/3696444.

[16] Meike Nauta, Jan Trienes, Shreyasi Pathak et al. “From Anecdotal Evidence to Quantitative Evaluation Methods: A Systematic Review on Evaluating Explainable AI”. En: *ACM Computing Surveys* 55.13s (2023), págs. 1-42. DOI: 10.1145/3583558.

[17] Francesco Bodria, Fosca Giannotti, Riccardo Guidotti, Francesca Naretto, Dino Pedreschi y Salvatore Rinzivillo. “Benchmarking and survey of explanation methods for black box models”. En: *Data Mining and Knowledge Discovery* 37.5 (2023), págs. 1719-1778. DOI: 10.1007/s10618-023-00933-9.

[18] Erik Štrumbelj e Igor Kononenko. “Explaining prediction models and individual predictions with feature contributions”. En: *Knowledge and Information Systems* 41.3 (2014), págs. 647-665. DOI: 10.1007/s10115-013-0679-x.

[19] Mukund Sundararajan, Ankur Taly y Qiqi Yan. “Axiomatic Attribution for Deep Networks”. En: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Vol. 70. Proceedings of Machine Learning Research. 2017, págs. 3319-3328. URL: https://proceedings.mlr.press/v70/sundararajan17a.html.

[20] Gabriel Erion, Joseph D. Janizek, Pascal Sturmfels, Scott M. Lundberg y Su-In Lee. “Improving performance of deep learning models with axiomatic attribution priors and expected gradients”. En: *Nature Machine Intelligence* 3.7 (2021), págs. 620-631. DOI: 10.1038/s42256-021-00343-w.

[21] Utku Gorkem Ketenci, Tolga Kurt, Selim Onal, Cenk Erbil, Sinan Akturkoglu y Hande Serban Ilhan. “A Time-Frequency Based Suspicious Activity Detection for Anti-Money Laundering”. En: *IEEE Access* 9 (2021), págs. 59957-59967. DOI: 10.1109/access.2021.3072114.

[22] Berkan Oztas, Deniz Cetinkaya, Festus Adedoyin, Marcin Budka, Gokhan Aksu y Huseyin Dogan. “Transaction monitoring in anti-money laundering: A qualitative analysis and points of view from industry”. En: *Future Generation Computer Systems* 159 (2024), págs. 161-171. DOI: 10.1016/j.future.2024.05.027.

[23] Waleed Hilal, S. Andrew Gadsden y John Yawney. “Financial Fraud: A Review of Anomaly Detection Techniques and Recent Advances”. En: *Expert Systems with Applications* 193 (2022), pág. 116429. DOI: 10.1016/j.eswa.2021.116429.

[24] Astrid Bertrand, Winston Maxwell y Xavier Vamparys. “Do AI-based anti-money laundering (AML) systems violate European fundamental rights?” En: *International Data Privacy Law* 11.3 (2021), págs. 276-293. DOI: 10.1093/idpl/ipab010.

[25] Paolo Giudici, Mattia Centurelli y Stefano Turchetta. “Artificial Intelligence risk measurement”. En: *Expert Systems with Applications* 235 (2024), pág. 121220. DOI: 10.1016/j.eswa.2023.121220.

[26] Yujia Chen, Raffaella Calabrese y Belen Martin-Barragan. “Interpretable machine learning for imbalanced credit scoring datasets”. En: *European Journal of Operational Research* 312.1 (2024), págs. 357-372. DOI: 10.1016/j.ejor.2023.06.036.

[27] Sean Foley, Jonathan R Karlsen y Tālis J Putniņš. “Sex, Drugs, and Bitcoin: How Much Illegal Activity Is Financed through Cryptocurrencies?” En: *The Review of Financial Studies* 32.5 (2019), págs. 1798-1853. DOI: 10.1093/rfs/hhz015.

[28] Masarah Paquet-Clouston, Bernhard Haslhofer y Benoît Dupont. “Ransomware payments in the Bitcoin ecosystem”. En: *Journal of Cybersecurity* 5.1 (2019), tyz003. DOI: 10.1093/cybsec/tyz003.

[29] Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt y Been Kim. “Sanity Checks for Saliency Maps”. En: *Advances in Neural Information Processing Systems 31 (NeurIPS)*. 2018, págs. 9505-9515. URL: https://papers.nips.cc/paper/8160-sanity-checks-for-saliency-maps.

[30] Angela Samantha Maitland Irwin, Kim-Kwang Raymond Choo y Lin Liu. “An analysis of money laundering and terrorism financing typologies”. En: *Journal of Money Laundering Control* 15.1 (2011), págs. 85-111. DOI: 10.1108/13685201211194745.

[31] Chad Albrecht, Kristopher McKay Duffin, Steven Hawkins y Victor Manuel Morales Rocha. “The use of cryptocurrencies in the money laundering process”. En: *Journal of Money Laundering Control* 22.2 (2019), págs. 210-216. DOI: 10.1108/jmlc-12-2017-0074.

[32] Hugo Almeida, Pedro Pinto y Ana Fernández Vilas. “From Placement to Integration: A Parametric Study of Cryptocurrency-Based Money Laundering Techniques”. En: *Risks* 13.12 (2025), pág. 249. DOI: 10.3390/risks13120249.

[33] Roland Subashi. “Cryptocurrencies and Money Laundering”. En: *Balkan Journal of Interdisciplinary Research* 10.1 (2024), págs. 55-62. DOI: 10.2478/bjir-2024-0005.

[34] Daniel Dupuis y Kimberly Gleason. “Money laundering with cryptocurrency: open doors and the regulatory dialectic”. En: *Journal of Financial Crime* 28.1 (2020), págs. 60-74. DOI: 10.1108/jfc-06-2020-0113.

[35] Nadia Pocher, Mirko Zichichi, Fabio Merizzi, Muhammad Zohaib Shafiq y Stefano Ferretti. “Detecting anomalous cryptocurrency transactions: An AML/CFT application of machine learning-based forensics”. En: *Electronic Markets* 33.1 (2023), pág. 37. DOI: 10.1007/s12525-023-00654-3.

[36] Danilo Labanca, Luca Primerano, Marcus Markland-Montgomery, Mario Polino, Michele Carminati y Stefano Zanero. “Amaretto: An Active Learning Framework for Money Laundering Detection”. En: *IEEE Access* 10 (2022), págs. 41720-41739. DOI: 10.1109/access.2022.3167699.

[37] Lorenz Sailer. “Explainable artificial intelligence in finance: Mapping business problems and technological solutions”. En: *Finance Research Open* 2.2 (2026), pág. 100128. DOI: 10.1016/j.finr.2026.100128.

[38] Ismail Alarab y Simant Prakoonwit. “Graph-Based LSTM for Anti-money Laundering: Experimenting Temporal Graph Convolutional Network with Bitcoin Data”. En: *Neural Processing Letters* 55.1 (2023), págs. 689-707. DOI: 10.1007/s11063-022-10904-8.

[39] Anuraj Mohan, Karthika P.V., Parvathi Sankar, K. Maya Manohar y Amala Peter. “Improving anti-money laundering in bitcoin using evolving graph convolutions and deep neural decision forest”. En: *Data Technologies and Applications* 57.3 (2023), págs. 313-329. DOI: 10.1108/dta-06-2021-0167.

[40] Ismail Alarab y Simant Prakoonwit. “Robust recurrent graph convolutional network approach based sequential prediction of illicit transactions in cryptocurrencies”. En: *Multimedia Tools and Applications* 83.20 (2023), págs. 58449-58464. DOI: 10.1007/s11042-023-17323-4.

[41] Yann LeCun, Yoshua Bengio y Geoffrey Hinton. “Deep learning”. En: *Nature* 521.7553 (2015), págs. 436-444. DOI: 10.1038/nature14539.

[42] Michael M. Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam y Pierre Vandergheynst. “Geometric Deep Learning: Going beyond Euclidean data”. En: *IEEE Signal Processing Magazine* 34.4 (2017), págs. 18-42. DOI: 10.1109/msp.2017.2693418.

[43] Jie Zhou, Ganqu Cui, Shengding Hu et al. “Graph neural networks: A review of methods and applications”. En: *AI Open* 1 (2020), págs. 57-81. DOI: 10.1016/j.aiopen.2021.01.001.

[44] Thomas N. Kipf y Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. En: *International Conference on Learning Representations (ICLR)*. 2017. arXiv: 1609.02907. URL: https://arxiv.org/abs/1609.02907.

[45] William L. Hamilton, Rex Ying y Jure Leskovec. “Inductive Representation Learning on Large Graphs”. En: *Advances in Neural Information Processing Systems 30 (NeurIPS)*. 2017, págs. 1024-1034. URL: https://papers.nips.cc/paper/6703-inductive-representation-learning-on-large-graphs.

[46] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò y Yoshua Bengio. “Graph Attention Networks”. En: *International Conference on Learning Representations (ICLR)*. 2018. arXiv: 1710.10903. URL: https://arxiv.org/abs/1710.10903.

[47] Jian Du, Shanghang Zhang, Guanhang Wu, José M. F. Moura y Soummya Kar. *Topology Adaptive Graph Convolutional Networks*. 2017. arXiv: 1710.10370. URL: https://arxiv.org/abs/1710.10370.

[48] Xin Zheng, Yi Wang, Yixin Liu et al. “Graph Neural Networks for Graphs With Heterophily: A Survey”. En: *IEEE Transactions on Knowledge and Data Engineering* 38.7 (2026), págs. 4385-4404. DOI: 10.1109/tkde.2026.3680353.

[49] Dimitrios Kelesis, Dimitris Fotakis y Georgios Paliouras. “Analyzing the effect of residual connections to oversmoothing in graph neural networks”. En: *Machine Learning* 114.8 (2025), pág. 184. DOI: 10.1007/s10994-025-06822-0.

[50] Changhai Wang, Software Engineering College, Zhengzhou University of Light Industry, Zhengzhou 450000, China, Zhe Huang et al. “Imbalanced node classification on graphs with graph neural networks: A survey”. En: *Electronic Research Archive* 34.3 (2026), págs. 1742-1784. DOI: 10.3934/era.2026079.

[51] Davide Chicco y Giuseppe Jurman. “The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation”. En: *BMC Genomics* 21.1 (2020), pág. 6. DOI: 10.1186/s12864-019-6413-7.

[52] Takaya Saito y Marc Rehmsmeier. “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets”. En: *PLOS ONE* 10.3 (2015), e0118432. DOI: 10.1371/journal.pone.0118432.

[53] Amina Adadi y Mohammed Berrada. “Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI)”. En: *IEEE Access* 6 (2018), págs. 52138-52160. DOI: 10.1109/access.2018.2870052.

[54] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Franco Turini, Fosca Giannotti y Dino Pedreschi. “A Survey of Methods for Explaining Black Box Models”. En: *ACM Computing Surveys* 51.5 (2019), págs. 1-42. DOI: 10.1145/3236009.

[55] Enyan Dai, Tianxiang Zhao, Huaisheng Zhu et al. “A Comprehensive Survey on Trustworthy Graph Neural Networks: Privacy, Robustness, Fairness, and Explainability”. En: *Machine Intelligence Research* 21.6 (2024), págs. 1011-1061. DOI: 10.1007/s11633-024-1510-8.

[56] Rex Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik y Jure Leskovec. “GNNExplainer: Generating Explanations for Graph Neural Networks”. En: *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 2019, págs. 9240-9251. URL: https://papers.nips.cc/paper/9123-gnnexplainer-generating-explanations-for-graph-neural-networks.

[57] Dongsheng Luo, Wei Cheng, Dongkuan Xu et al. “Parameterized Explainer for Graph Neural Network”. En: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. 2020, págs. 19620-19631. URL: https://proceedings.neurips.cc/paper/2020/hash/e37b08dd3015330dcbb5d6663667b8b8-Abstract.html.

[58] Scott M. Lundberg y Su-In Lee. “A Unified Approach to Interpreting Model Predictions”. En: *Advances in Neural Information Processing Systems 30 (NeurIPS)*. 2017, págs. 4765-4774. URL: https://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.

[59] Javier Castro, Daniel Gómez y Juan Tejada. “Polynomial calculation of the Shapley value based on sampling”. En: *Computers & Operations Research* 36.5 (2009), págs. 1726-1730. DOI: 10.1016/j.cor.2008.04.004.

[60] C. Spearman. “The Proof and Measurement of Association between Two Things”. En: *The American Journal of Psychology* 15.1 (1904), pág. 72. DOI: 10.2307/1412159.

[61] Paul Jaccard. “THE DISTRIBUTION OF THE FLORA IN THE ALPINE ZONE. 1”. En: *New Phytologist* 11.2 (1912), págs. 37-50. DOI: 10.1111/j.1469-8137.1912.tb05611.x.

[62] Falko Gawantka, Franz Just, Marina Savelyeva, Markus Wappler y Jörg Lässig. “A Novel Metric for Evaluating the Stability of XAI Explanations”. En: *Advances in Science, Technology and Engineering Systems Journal* 9 (2024), págs. 133-142. DOI: 10.25046/aj090113.

[63] Milton Friedman. “The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance”. En: *Journal of the American Statistical Association* 32.200 (1937), págs. 675-701. DOI: 10.1080/01621459.1937.10503522.

[64] Frank Wilcoxon. “Individual Comparisons by Ranking Methods”. En: *Biometrics Bulletin* 1.6 (1945), pág. 80. DOI: 10.2307/3001968.

[65] Daniël Lakens. “Equivalence Tests”. En: *Social Psychological and Personality Science* 8.4 (2017), págs. 355-362. DOI: 10.1177/1948550617697177.

[66] B. Efron. “Bootstrap Methods: Another Look at the Jackknife”. En: *The Annals of Statistics* 7.1 (1979). DOI: 10.1214/aos/1176344552.

[67] Stuart H. Hurlbert. “Pseudoreplication and the Design of Ecological Field Experiments”. En: *Ecological Monographs* 54.2 (1984), págs. 187-211. DOI: 10.2307/1942661.

[68] Stanley E Lazic. “The problem of pseudoreplication in neuroscientific studies: is it affecting your analysis?” En: *BMC Neuroscience* 11.1 (2010), pág. 5. DOI: 10.1186/1471-2202-11-5.

[69] Dylan Vassallo, Vincent Vella y Joshua Ellul. “Application of Gradient Boosting Algorithms for Anti-money Laundering in Cryptocurrencies”. En: *SN Computer Science* 2.3 (2021), pág. 143. DOI: 10.1007/s42979-021-00558-z.

[70] Wai Weng Lo, Gayan K. Kulatilleke, Mohanad Sarhan, Siamak Layeghy y Marius Portmann. “Inspection-L: self-supervised GNN node embeddings for money laundering detection in bitcoin”. En: *Applied Intelligence* 53.16 (2023), págs. 19406-19417. DOI: 10.1007/s10489-023-04504-9.

[71] Ziqian Lin, Qining Luo, Dongze Wu et al. “Detecting illicit transactions in bitcoin: a wavelet-temporal graph transformer approach for anti-money laundering”. En: *Scientific Reports* 16.1 (2026), pág. 1548. DOI: 10.1038/s41598-025-23901-3.

[72] Fei Wan y Ping Li. “A Novel Money Laundering Prediction Model Based on a Dynamic Graph Convolutional Neural Network and Long Short-Term Memory”. En: *Symmetry* 16.3 (2024), pág. 378. DOI: 10.3390/sym16030378.

[73] Yulong Wang, Qingxiao Zheng, Xuedong Li, Lingfeng Wang y Ling Lin. “CoSemiGNN: Blockchain fraud detection with dynamic graph neural networks based on co-association of semi-supervised”. En: *Expert Systems with Applications* 298 (2026), pág. 129853. DOI: 10.1016/j.eswa.2025.129853.

[74] Ahmad Asiri y K. Somasundaram. “Graph convolution network for fraud detection in bitcoin transactions”. En: *Scientific Reports* 15.1 (2025), pág. 11076. DOI: 10.1038/s41598-025-95672-w.

[75] Oluwatosin Lawal, Awele Okolie y Callistus Obunadike. “AN EXPLAINABLE GRAPH NEURAL NETWORK FRAMEWORK FOR ANTI–MONEY LAUNDERING IN CRYPTOCURRENCY TRANSACTIONS USING THE ELLIPTIC DATASET”. En: *International Journal of Network Security & Its Applications* 17.6 (2025), págs. 27-39. DOI: 10.5121/ijnsa.2025.17602.

[76] Ekta Unagar y Bhavesh Borisaniya. “Survey on Detection of Cryptocurrency Money Laundering and Its Explanation Using XAI”. En: *SN Computer Science* 6.3 (2025), pág. 291. DOI: 10.1007/s42979-025-03828-2.

[77] Xiaosong He, Jie Huang, Kai Ma, Hongjing He y Min Li. “An explainable graph neural network framework for illicit financial transaction detection”. En: *Applied Intelligence* 56.4 (2026), pág. 102. DOI: 10.1007/s10489-026-07138-9.

[78] Teena Rai, Jun He, Jaspreet Kaur et al. “Evaluating XAI techniques under class imbalance using CPRD data”. En: *Frontiers in Artificial Intelligence* 8 (2025), pág. 1682919. DOI: 10.3389/frai.2025.1682919.

[79] Kazuki Fujiwara. “Knowledge distillation with resampling for imbalanced data classification: Enhancing predictive performance and explainability stability”. En: *Results in Engineering* 24 (2024), pág. 103406. DOI: 10.1016/j.rineng.2024.103406.

[80] Shamkhal Mammadov, Ismail Abdulrashid y Jiuh-Biing Sheu. “Managing Trust in AI Systems: Evidence From Controlled Evaluation of Explanation Stability Under Data Imbalance”. En: *IEEE Transactions on Engineering Management* 73 (2026), págs. 3083-3096. DOI: 10.1109/tem.2026.3687235.

[81] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy y Abdelhamid Bouchachia. “A survey on concept drift adaptation”. En: *ACM Computing Surveys* 46.4 (2014), págs. 1-37. DOI: 10.1145/2523813.

Apéndice A

Corrida con la partición original

Antes de excluir el periodo del cierre del mercado se ejecutó el estudio completo con la partición habitual de Elliptic: entrenamiento con los pasos 1 a 34 (3.462 ilícitas y 132.803 negativas, razón 1:38,4), validación con los pasos 35 a 42 y prueba con los pasos 43 a 49. Se entrenaron 252 modelos: las mismas 4 arquitecturas, 3 pérdidas y 3 semillas, con los escenarios nativo, 1:10, 1:20, 1:10 con SMOTE, 1:1 (que se apartó del análisis principal), 1:100 y 1:200. En esta partición los escenarios de estrés conservan 1.328 y 664 ilícitas. Este anexo resume lo que esa corrida muestra y por qué motivó el cambio de partición. Los resultados completos están en `results_v4/`.

A.1. La prueba cae al nivel del azar

En la prueba, después del cierre, la prevalencia de ilícitas es de 0,6 %, que es el PR-AUC de un clasificador al azar. Con la pérdida elegida en validación, en el escenario nativo, el PR-AUC de prueba fue de 0,010 para GraphSAGE, 0,008 para TAGCN, 0,005 para GAT y 0,012 para GCN. El ROC-AUC fue 0,711, 0,667, 0,476 y 0,601. Ningún modelo detecta las transacciones ilícitas posteriores al cierre, lo que reproduce lo reportado por los autores del conjunto de datos [9]. En validación, en cambio, los modelos sí aprendían: GraphSAGE con pesos por clase alcanzaba un PR-AUC de 0,462 frente a 0,037 del azar.

A.2. Lo que se mantiene en las dos particiones

La estabilidad de las explicaciones se mide en validación, por lo que no depende de la prueba, y sus conclusiones generales se repiten en las dos corridas:

- Con pesos por clase, el nivel de desbalance no cambia la estabilidad. Con GNNExplainer y las tres arquitecturas que aprenden, el Spearman global fue 0,946, 0,947, 0,946 y 0,947

en 1:10, 1:10 con SMOTE, 1:20 y nativo, y 0,940 y 0,938 en 1:100 y 1:200. La prueba de Friedman sobre los seis escenarios no rechazó la igualdad ($p = 0{,}84$).

- GCN es la arquitectura más débil en las dos particiones.
- Con poco fraude (1:100 y 1:200) solo los pesos por clase mantienen la detección.
- El control de pesos aleatorios dio, sobre 12 modelos, un acuerdo entre la explicación del modelo entrenado y la de la red sin entrenar de 0,896 con GNNExplainer, 0,911 con Shapley, 0,956 con Integrated Gradients y 0,688 con Expected Gradients, frente a una estabilidad entre réplicas de 0,949, 0,970, 1 y 0,929.

A.3. Lo que cambia

Tres resultados de la corrida original no se repiten sin el cierre, y por eso no se presentan como conclusiones de la tesis:

- Con la partición original, GraphSAGE fue la arquitectura más estable con GNNExplainer en las tres semillas (Spearman global 0,964 frente a 0,939 de GAT y 0,937 de TAGCN). Sin el cierre ese orden desaparece (Sección 6.6).
- Con la partición original, pesos por clase dio explicaciones de GNNExplainer más estables que *focal loss* en los 9 pares de arquitectura y semilla (prueba de signos, $p = 0{,}004$), pero con Shapley el sentido se invertía. Sin el cierre la diferencia con GNNExplainer tampoco se sostiene (Sección 6.7).
- En validación, pesos por clase era la mejor pérdida en el escenario nativo. Sin el cierre, *focal loss* la iguala o la supera en ese escenario.

Una advertencia metodológica surgió de esta corrida y se aplica a toda la tesis: los modelos de una misma semilla en distintos escenarios son casi idénticos, por lo que una prueba que trata cada combinación de semilla y escenario como un bloque independiente (12 bloques) está pseudorreplicada y exagera la significancia. Las pruebas del Capítulo 6 usan como unidad el modelo.

A.4. Estabilidad por escenario en la partición original

La Tabla A.1 reproduce, para la partición original, la estabilidad por escenario con las tres medidas (pesos por clase, GraphSAGE, GAT y TAGCN, 9 modelos por celda). Los

Tabla A.1: Estabilidad entre réplicas por escenario en la partición original (pesos por clase, media de 9 modelos).

Medida	Explicador	1:10	1:10 + SMOTE	1:20	Nativo	1:100	1:200
Spearman global	GNNExplainer	0,946	0,947	0,946	0,947	0,940	0,938
	Shapley	0,993	0,992	0,992	0,992	0,991	0,992
	Integrated Gradients	1,000	1,000	1,000	1,000	1,000	1,000
	Expected Gradients	0,926	0,922	0,925	0,923	0,927	0,922
Spearman $\neq 0$	GNNExplainer	0,749	0,753	0,751	0,753	0,717	0,708
	Shapley	0,985	0,984	0,985	0,985	0,982	0,983
	Integrated Gradients	1,000	1,000	1,000	1,000	1,000	1,000
	Expected Gradients	0,958	0,956	0,957	0,956	0,954	0,953
Jaccard top-10	GNNExplainer	0,623	0,641	0,625	0,620	0,608	0,616
	Shapley	0,822	0,799	0,813	0,812	0,819	0,826
	Integrated Gradients	1,000	1,000	1,000	1,000	1,000	1,000
	Expected Gradients	0,771	0,779	0,765	0,767	0,769	0,759

valores y su ausencia de tendencia son muy parecidos a los de la Tabla 6.4. Fuente: `results_v4/reunion_0710/metrica_robusta.log`.

En la partición original, el acuerdo de GNNExplainer entre modelos fue de 0,942 entre semillas en el mismo escenario, 0,987 entre escenarios con la misma semilla y 0,941 entre escenarios y semillas distintos con la medida principal, y de 0,391, 0,751 y 0,383 con el Jaccard top-10: el mismo patrón que se observa sin el cierre. Cambiar de escenario mueve menos la explicación que cambiar de semilla. Con el Spearman sobre las variables distintas de cero, GNNExplainer baja algo en 1:100 y 1:200 (0,717 y 0,708 frente a 0,753 en el nativo) en esta partición; sin el cierre esa caída no aparece (Tabla 6.4), por lo que no se considera un efecto del desbalance.

Apéndice B

Eje sintético

Elliptic no publica el significado de sus variables ni anota qué patrón de lavado sigue cada transacción ilícita, de modo que sobre él solo se puede medir la estabilidad de las explicaciones, no si señalan lo correcto. Para observar esa segunda propiedad se construyó un grafo sintético de transacciones en el que cada transacción ilícita pertenece a una tipología conocida y en el que se sabe qué aristas y qué variables la delatan. Este anexo resume sus resultados como evidencia complementaria; no forma parte del contraste de hipótesis.

B.1. Diseño

El generador (`phase1/synthetic_aml_generator.py`, opción `-v4`, semilla 42) produce un grafo con tipologías de lavado insertadas sobre un fondo de transacciones legítimas. Se verificó que ninguna variable aislada permite separar las clases (AUC de una sola variable entre 0,50 y 0,62) y que las cuatro arquitecturas aprenden la tipología con un margen de al menos 0,10. Se entrenaron 144 configuraciones: las 4 arquitecturas, 4 escenarios, 3 técnicas de compensación y 3 semillas, explicadas con GNNExplainer, PGExplainer y Shapley por muestreo.

Los escenarios del eje sintético difieren de los de Elliptic: los niveles 1:50 y 1:100 se obtienen quitando transacciones ilícitas. Además se miden la **plausibilidad** (coincidencia de las variables o aristas señaladas con las que de verdad definen la tipología; para las variables, la fracción de las variables verdaderas que aparecen entre las 3 principales) y la **fidelidad** (cuánto cambia la predicción al retirar lo señalado, fid+, o al conservar solo lo señalado, fid$-$), cada una frente a una explicación al azar del mismo tamaño. La fidelidad se calcula sobre lo que cada explicador explica: retirando aristas en GNNExplainer y PGExplainer, y retirando variables en Shapley. Por eso la fidelidad de GNNExplainer y la de Shapley no son comparables entre sí; cada una solo se compara con su propio valor al azar.

B.2. Resultados

Tabla B.1: Eje sintético: estabilidad (Spearman global), plausibilidad de variables y fidelidad (fid+; sobre aristas en GNNExplainer y sobre variables en Shapley), con su valor al azar entre paréntesis. Medias sobre 36 configuraciones por fila. Fuente: `results_phase1_v4/summary_v4.csv`.

Arquitectura	Explicador	Estabilidad	Plausibilidad de variables	Fidelidad (fid+)
GCN	GNNExplainer	0,848	0,296 (0,150)	0,147 (0,030)
	Shapley	0,984	0,114 (0,150)	0,195 (0,063)
GraphSAGE	GNNExplainer	0,907	0,404 (0,150)	0,363 (0,048)
	Shapley	0,953	0,157 (0,150)	0,135 (0,092)
GAT	GNNExplainer	0,850	0,244 (0,150)	0,264 (0,054)
	Shapley	0,959	0,118 (0,150)	0,118 (0,101)
TAGCN	GNNExplainer	0,957	0,447 (0,150)	0,277 (0,053)
	Shapley	0,945	0,159 (0,150)	0,154 (0,083)

La Tabla B.1 muestra el resultado central del eje: el explicador más estable no es el más correcto. Shapley por muestreo es el más estable en tres de las cuatro arquitecturas (Spearman entre 0,95 y 0,98), pero su plausibilidad de variables no supera al azar (entre 0,11 y 0,16 frente a 0,15). GNNExplainer es menos estable, pero es el único cuya plausibilidad de variables supera claramente al azar (entre 0,24 y 0,45). Ambos explicadores superan a su propia línea base de fidelidad, salvo Shapley en GAT (0,118 frente a 0,101, $p = 0{,}22$), pero esas cifras no permiten decir cuál es más fiel, porque miden cosas distintas. La estabilidad de GNNExplainer baja con el desbalance, de 0,92 en los niveles nativo y 1:1 a 0,86 en 1:100, mientras que la de Shapley no cambia.

Este resultado apoya una de las conclusiones del estudio principal: la estabilidad es necesaria pero no suficiente, y debe complementarse con pruebas de que la explicación depende del modelo.

B.3. Advertencias

- La plausibilidad de aristas no es comparable entre arquitecturas. TAGCN ve 6 saltos y su subgrafo receptivo contiene casi todo el grafo (mediana de 9.311 nodos y 47.311 aristas), mientras que las otras tres ven 2 saltos (unos 36 nodos y 78 aristas). La plausibilidad al azar vale 0,001 en TAGCN y cerca de 0,47 en las demás, y la ganancia aparente de TAGCN se debe a esa línea base. Por eso la tabla no la incluye.
- En las arquitecturas de dos saltos, la plausibilidad de aristas de los explicadores apenas supera al azar (entre 0,01 y 0,03), y la de PGExplainer queda por debajo.
- PGExplainer falló en 24 de 432 combinaciones, todas en los escenarios 1:50 y 1:100.

Apéndice C

Reproducibilidad

C.1. Entorno

Los experimentos se ejecutaron en una estación con GPU NVIDIA RTX 4060 de 8 GB, con Python 3.12 y PyTorch Geometric. Las dependencias exactas están fijadas en `uv.lock`, en la raíz del repositorio, y se instalan con `uv sync`. El conjunto de datos Elliptic se descarga automáticamente en `data/` la primera vez.

C.2. Corrida principal (sin el cierre del mercado)

- Configuración: `configs/experiment_v4_precierre.yaml`.
- Entrenamiento y explicación: `scripts/v4/run_precierre.sh` y `scripts/v4/cola_precierre.sh`. Producen 216 modelos en `results_models_v4_precierre/` y las explicaciones en `results_v4_precierre/` y `results_v4_precierre_grad/`.
- Curvas y métricas de rendimiento: `scripts/v4/reunion_0410/curvas.py -variante precierre`.
- Medidas robustas y control de pesos aleatorios: `scripts/v4/cola_robusta_precierre.sh`, que ejecuta en secuencia `metrica_robusta.py` y `control_aleatorio.py` con `-variante precierre`.

C.3. Tablas, figuras y bibliografía del manuscrito

Las tablas y figuras se regeneran sin GPU desde los resultados guardados:

- `tesis_v4/scripts/analisis.py`: tablas LaTeX y pruebas estadísticas (`tesis_v4/tablas/`).
- `tesis_v4/scripts/figuras.py`: figuras en PDF vectorial (`tesis_v4/figuras/`).
- `tesis_v4/scripts/verificar_referencias.py`: genera `bibliografia.bib` consultando Crossref por DOI y rechaza todo lo que no sea un artículo de revista o tenga una retractación registrada. Las únicas excepciones son los artículos originales de los métodos y del conjunto de datos usados, listados en `tesis_v4/referencias/excepciones.bib` con la URL donde se verificaron.

C.4. Determinismo

Las semillas de datos (2026), de modelo (42, 43 y 44), de selección de transacciones (1234) y de los explicadores están fijadas. Aun así, el entrenamiento en GPU no es determinista a nivel de pesos, porque las agregaciones de mensajes usan sumas atómicas. Repetir la corrida reproduce las conclusiones, pero no necesariamente todos los decimales.

Apéndice D

Tablas completas

Este anexo recoge los hiperparámetros elegidos para cada configuración (Tabla D.1) y los resultados completos de rendimiento y estabilidad (Tabla D.2) de la corrida principal, sin el cierre del mercado. Ambas tablas se generan con `tesis_v4/scripts/analisis.py` desde los resultados guardados.

Tabla D.1: Hiperparámetros elegidos por la búsqueda (semilla 42; las semillas 43 y 44 los reutilizan). h: dimensión oculta; L: capas; p: *dropout*; η: tasa de aprendizaje; λ: decaimiento de pesos; H/K: cabezas de GAT u orden de TAGCN.

Arquitectura	Escenario	Pérdida	h	L	p	η	λ	H/K
GCN	1:10	Pesos por clase	64	2	0,49	$4{,}6\times10^{-3}$	$2{,}7\times10^{-5}$	–
GCN	1:10	Focal loss	148	2	0,13	$7{,}9\times10^{-3}$	$8{,}5\times10^{-4}$	–
GCN	1:10	Sin ajuste	148	2	0,13	$7{,}9\times10^{-3}$	$8{,}5\times10^{-4}$	–
GCN	1:10 + SMOTE	Pesos por clase	64	2	0,49	$4{,}6\times10^{-3}$	$2{,}7\times10^{-5}$	–
GCN	1:10 + SMOTE	Focal loss	148	2	0,13	$7{,}9\times10^{-3}$	$8{,}5\times10^{-4}$	–
GCN	1:10 + SMOTE	Sin ajuste	64	2	0,49	$4{,}6\times10^{-3}$	$2{,}7\times10^{-5}$	–
GCN	1:20	Pesos por clase	64	2	0,49	$4{,}6\times10^{-3}$	$2{,}7\times10^{-5}$	–
GCN	1:20	Focal loss	148	2	0,13	$7{,}9\times10^{-3}$	$8{,}5\times10^{-4}$	–
GCN	1:20	Sin ajuste	128	3	0,22	$1{,}1\times10^{-3}$	$1{,}2\times10^{-4}$	–
GCN	Nativo	Pesos por clase	64	2	0,49	$4{,}6\times10^{-3}$	$2{,}7\times10^{-5}$	–
GCN	Nativo	Focal loss	148	2	0,13	$7{,}9\times10^{-3}$	$8{,}5\times10^{-4}$	–
GCN	Nativo	Sin ajuste	148	2	0,13	$7{,}9\times10^{-3}$	$8{,}5\times10^{-4}$	–
GCN	1:100	Pesos por clase	64	2	0,49	$4{,}6\times10^{-3}$	$2{,}7\times10^{-5}$	–
GCN	1:100	Focal loss	64	2	0,49	$4{,}6\times10^{-3}$	$2{,}7\times10^{-5}$	–
GCN	1:100	Sin ajuste	64	2	0,49	$4{,}6\times10^{-3}$	$2{,}7\times10^{-5}$	–
GCN	1:200	Pesos por clase	64	2	0,49	$4{,}6\times10^{-3}$	$2{,}7\times10^{-5}$	–
GCN	1:200	Focal loss	64	2	0,49	$4{,}6\times10^{-3}$	$2{,}7\times10^{-5}$	–
GCN	1:200	Sin ajuste	148	2	0,13	$7{,}9\times10^{-3}$	$8{,}5\times10^{-4}$	–

Arquitectura	Escenario	Pérdida	h	L	p	η	λ	H/K
GraphSAGE	1:10	Pesos por clase	148	2	0,13	$7{,}9 \times 10^{-3}$	$8{,}5 \times 10^{-4}$	–
GraphSAGE	1:10	Focal loss	148	3	0,27	$3{,}8 \times 10^{-4}$	$1{,}7 \times 10^{-4}$	–
GraphSAGE	1:10	Sin ajuste	148	2	0,13	$7{,}9 \times 10^{-3}$	$8{,}5 \times 10^{-4}$	–
GraphSAGE	1:10 + SMOTE	Pesos por clase	148	2	0,13	$7{,}9 \times 10^{-3}$	$8{,}5 \times 10^{-4}$	–
GraphSAGE	1:10 + SMOTE	Focal loss	148	3	0,27	$3{,}8 \times 10^{-4}$	$1{,}7 \times 10^{-4}$	–
GraphSAGE	1:10 + SMOTE	Sin ajuste	148	2	0,13	$7{,}9 \times 10^{-3}$	$8{,}5 \times 10^{-4}$	–
GraphSAGE	1:20	Pesos por clase	64	2	0,49	$4{,}6 \times 10^{-3}$	$2{,}7 \times 10^{-5}$	–
GraphSAGE	1:20	Focal loss	148	3	0,27	$3{,}8 \times 10^{-4}$	$1{,}7 \times 10^{-4}$	–
GraphSAGE	1:20	Sin ajuste	128	3	0,22	$1{,}1 \times 10^{-3}$	$1{,}2 \times 10^{-4}$	–
GraphSAGE	Nativo	Pesos por clase	148	2	0,13	$7{,}9 \times 10^{-3}$	$8{,}5 \times 10^{-4}$	–
GraphSAGE	Nativo	Focal loss	148	3	0,27	$3{,}8 \times 10^{-4}$	$1{,}7 \times 10^{-4}$	–
GraphSAGE	Nativo	Sin ajuste	128	3	0,22	$1{,}1 \times 10^{-3}$	$1{,}2 \times 10^{-4}$	–
GraphSAGE	1:100	Pesos por clase	64	2	0,49	$4{,}6 \times 10^{-3}$	$2{,}7 \times 10^{-5}$	–
GraphSAGE	1:100	Focal loss	148	3	0,27	$3{,}8 \times 10^{-4}$	$1{,}7 \times 10^{-4}$	–
GraphSAGE	1:100	Sin ajuste	128	3	0,22	$1{,}1 \times 10^{-3}$	$1{,}2 \times 10^{-4}$	–
GraphSAGE	1:200	Pesos por clase	148	2	0,13	$7{,}9 \times 10^{-3}$	$8{,}5 \times 10^{-4}$	–
GraphSAGE	1:200	Focal loss	148	3	0,27	$3{,}8 \times 10^{-4}$	$1{,}7 \times 10^{-4}$	–
GraphSAGE	1:200	Sin ajuste	148	2	0,13	$7{,}9 \times 10^{-3}$	$8{,}5 \times 10^{-4}$	–
GAT	1:10	Pesos por clase	148	2	0,25	$7{,}0 \times 10^{-4}$	$5{,}0 \times 10^{-4}$	4
GAT	1:10	Focal loss	128	2	0,22	$1{,}7 \times 10^{-3}$	$1{,}9 \times 10^{-5}$	4
GAT	1:10	Sin ajuste	128	2	0,22	$1{,}7 \times 10^{-3}$	$1{,}9 \times 10^{-5}$	4
GAT	1:10 + SMOTE	Pesos por clase	128	2	0,22	$1{,}7 \times 10^{-3}$	$1{,}9 \times 10^{-5}$	4
GAT	1:10 + SMOTE	Focal loss	128	2	0,22	$1{,}7 \times 10^{-3}$	$1{,}9 \times 10^{-5}$	4
GAT	1:10 + SMOTE	Sin ajuste	128	2	0,22	$1{,}7 \times 10^{-3}$	$1{,}9 \times 10^{-5}$	4
GAT	1:20	Pesos por clase	128	2	0,22	$1{,}7 \times 10^{-3}$	$1{,}9 \times 10^{-5}$	4
GAT	1:20	Focal loss	128	2	0,22	$1{,}7 \times 10^{-3}$	$1{,}9 \times 10^{-5}$	4
GAT	1:20	Sin ajuste	148	2	0,41	$7{,}6 \times 10^{-3}$	$6{,}2 \times 10^{-4}$	4
GAT	Nativo	Pesos por clase	128	2	0,22	$1{,}7 \times 10^{-3}$	$1{,}9 \times 10^{-5}$	4
GAT	Nativo	Focal loss	128	2	0,22	$1{,}7 \times 10^{-3}$	$1{,}9 \times 10^{-5}$	4
GAT	Nativo	Sin ajuste	148	2	0,41	$7{,}6 \times 10^{-3}$	$6{,}2 \times 10^{-4}$	4
GAT	1:100	Pesos por clase	128	2	0,22	$1{,}7 \times 10^{-3}$	$1{,}9 \times 10^{-5}$	4
GAT	1:100	Focal loss	148	2	0,41	$7{,}6 \times 10^{-3}$	$6{,}2 \times 10^{-4}$	4
GAT	1:100	Sin ajuste	148	2	0,41	$7{,}6 \times 10^{-3}$	$6{,}2 \times 10^{-4}$	4
GAT	1:200	Pesos por clase	128	2	0,22	$1{,}7 \times 10^{-3}$	$1{,}9 \times 10^{-5}$	4
GAT	1:200	Focal loss	148	2	0,41	$7{,}6 \times 10^{-3}$	$6{,}2 \times 10^{-4}$	4
GAT	1:200	Sin ajuste	148	2	0,41	$7{,}6 \times 10^{-3}$	$6{,}2 \times 10^{-4}$	4
TAGCN	1:10	Pesos por clase	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:10	Focal loss	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:10	Sin ajuste	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3

Arquitectura	Escenario	Pérdida	h	L	p	η	λ	H/K
TAGCN	1:10 + SMOTE	Pesos por clase	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:10 + SMOTE	Focal loss	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:10 + SMOTE	Sin ajuste	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:20	Pesos por clase	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:20	Focal loss	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:20	Sin ajuste	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	Nativo	Pesos por clase	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	Nativo	Focal loss	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	Nativo	Sin ajuste	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:100	Pesos por clase	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:100	Focal loss	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:100	Sin ajuste	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:200	Pesos por clase	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:200	Focal loss	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3
TAGCN	1:200	Sin ajuste	128	3	0,20	$2{,}1 \times 10^{-3}$	$4{,}2 \times 10^{-5}$	3

Tabla D.2: Resultados completos por configuración (media de tres semillas). Val. y Prueba: PR-AUC; ROC: ROC-AUC de prueba; F: modelos que superan el filtro (de 3); VP: verdaderos positivos entre las 30 transacciones explicadas; GE y SH: estabilidad (Spearman global) de GNNExplainer y Shapley; J: Jaccard top-10 de GNNExplainer.

Arq.	Escenario	Pérdida	Val.	Prueba	ROC	F	VP	GE / SH	J
GCN	1:10	Pesos por clase	0,171	0,088	0,770	0	22,3	0,936 / 0,996	0,703
GCN	1:10	Focal loss	0,234	0,081	0,715	1	12,7	0,914 / 0,997	0,675
GCN	1:10	Sin ajuste	0,369	0,090	0,765	3	13,7	0,941 / 0,997	0,693
GCN	1:10 + SMOTE	Pesos por clase	0,169	0,087	0,768	0	22,3	0,937 / 0,996	0,709
GCN	1:10 + SMOTE	Focal loss	0,233	0,088	0,718	1	12,7	0,914 / 0,997	0,664
GCN	1:10 + SMOTE	Sin ajuste	0,271	0,099	0,748	2	10,3	0,931 / 0,996	0,662
GCN	1:20	Pesos por clase	0,167	0,087	0,768	0	22,3	0,937 / 0,996	0,709
GCN	1:20	Focal loss	0,206	0,080	0,703	1	10,0	0,906 / 0,997	0,636
GCN	1:20	Sin ajuste	0,147	0,045	0,645	1	8,3	0,940 / 0,995	0,680
GCN	Nativo	Pesos por clase	0,171	0,088	0,768	0	22,3	0,937 / 0,996	0,710
GCN	Nativo	Focal loss	0,214	0,085	0,729	0	8,3	0,902 / 0,997	0,633
GCN	Nativo	Sin ajuste	0,159	0,049	0,633	0	4,3	0,907 / 0,997	0,647
GCN	1:100	Pesos por clase	0,177	0,085	0,757	0	22,3	0,936 / 0,996	0,713
GCN	1:100	Focal loss	0,083	0,041	0,603	0	5,0	0,930 / 0,995	0,709
GCN	1:100	Sin ajuste	0,035	0,023	0,508	0	3,0	0,926 / 0,997	0,700
GCN	1:200	Pesos por clase	0,219	0,083	0,738	0	22,0	0,942 / 0,996	0,699
GCN	1:200	Focal loss	0,064	0,032	0,536	0	5,0	0,931 / 0,996	0,714

Arq.	Escenario	Pérdida	Val.	Prueba	ROC	F	VP	GE / SH	J
GCN	1:200	Sin ajuste	0,069	0,035	0,593	0	1,7	0,897 / 0,997	0,604
GraphSAGE	1:10	Pesos por clase	0,405	0,252	0,864	1	21,7	0,929 / 0,994	0,604
GraphSAGE	1:10	Focal loss	0,325	0,169	0,840	2	21,3	0,916 / 0,996	0,572
GraphSAGE	1:10	Sin ajuste	0,551	0,250	0,887	3	17,0	0,939 / 0,993	0,646
GraphSAGE	1:10 + SMOTE	Pesos por clase	0,389	0,246	0,846	1	19,3	0,926 / 0,994	0,605
GraphSAGE	1:10 + SMOTE	Focal loss	0,327	0,171	0,841	2	22,7	0,918 / 0,996	0,571
GraphSAGE	1:10 + SMOTE	Sin ajuste	0,554	0,274	0,894	3	16,7	0,943 / 0,994	0,645
GraphSAGE	1:20	Pesos por clase	0,462	0,306	0,901	3	27,0	0,953 / 0,992	0,669
GraphSAGE	1:20	Focal loss	0,457	0,253	0,878	3	21,7	0,925 / 0,997	0,609
GraphSAGE	1:20	Sin ajuste	0,035	0,022	0,524	0	2,3	0,908 / 0,995	0,499
GraphSAGE	Nativo	Pesos por clase	0,326	0,270	0,846	0	18,7	0,916 / 0,995	0,611
GraphSAGE	Nativo	Focal loss	0,478	0,273	0,883	3	21,0	0,921 / 0,997	0,606
GraphSAGE	Nativo	Sin ajuste	0,034	0,022	0,523	0	2,3	0,908 / 0,995	0,500
GraphSAGE	1:100	Pesos por clase	0,447	0,268	0,867	3	24,0	0,957 / 0,992	0,660
GraphSAGE	1:100	Focal loss	0,189	0,107	0,633	1	6,3	0,912 / 0,996	0,542
GraphSAGE	1:100	Sin ajuste	0,034	0,022	0,523	0	2,3	0,908 / 0,995	0,500
GraphSAGE	1:200	Pesos por clase	0,420	0,183	0,823	2	18,3	0,937 / 0,992	0,600
GraphSAGE	1:200	Focal loss	0,054	0,026	0,563	0	1,7	0,906 / 0,995	0,507
GraphSAGE	1:200	Sin ajuste	0,038	0,024	0,544	0	0,0	0,878 / 0,994	0,506
GAT	1:10	Pesos por clase	0,344	0,259	0,885	0	28,7	0,937 / 0,994	0,656
GAT	1:10	Focal loss	0,233	0,126	0,826	0	18,7	0,934 / 0,995	0,643
GAT	1:10	Sin ajuste	0,314	0,213	0,867	3	16,0	0,934 / 0,995	0,627
GAT	1:10 + SMOTE	Pesos por clase	0,188	0,124	0,767	0	18,0	0,919 / 0,993	0,645
GAT	1:10 + SMOTE	Focal loss	0,233	0,121	0,820	0	19,0	0,935 / 0,994	0,646
GAT	1:10 + SMOTE	Sin ajuste	0,297	0,201	0,865	3	16,0	0,934 / 0,994	0,625
GAT	1:20	Pesos por clase	0,185	0,123	0,765	0	18,3	0,918 / 0,993	0,648
GAT	1:20	Focal loss	0,304	0,182	0,852	2	16,0	0,925 / 0,994	0,644
GAT	1:20	Sin ajuste	0,357	0,262	0,883	3	12,3	0,928 / 0,992	0,694
GAT	Nativo	Pesos por clase	0,187	0,124	0,765	0	18,3	0,918 / 0,993	0,650
GAT	Nativo	Focal loss	0,268	0,162	0,833	2	13,7	0,924 / 0,994	0,636
GAT	Nativo	Sin ajuste	0,282	0,172	0,848	2	7,3	0,921 / 0,990	0,642
GAT	1:100	Pesos por clase	0,179	0,112	0,760	0	18,0	0,918 / 0,993	0,647
GAT	1:100	Focal loss	0,302	0,218	0,862	3	10,3	0,910 / 0,994	0,636
GAT	1:100	Sin ajuste	0,154	0,094	0,753	0	5,7	0,899 / 0,979	0,528
GAT	1:200	Pesos por clase	0,211	0,128	0,764	0	19,0	0,921 / 0,992	0,648
GAT	1:200	Focal loss	0,229	0,126	0,801	1	7,0	0,910 / 0,992	0,639
GAT	1:200	Sin ajuste	0,102	0,054	0,701	0	5,7	0,883 / 0,970	0,452
TAGCN	1:10	Pesos por clase	0,389	0,166	0,822	2	20,0	0,940 / 0,989	0,602
TAGCN	1:10	Focal loss	0,337	0,149	0,809	3	16,3	0,908 / 0,993	0,486

Arq.	Escenario	Pérdida	Val.	Prueba	ROC	F	VP	GE / SH	J
TAGCN	1:10	Sin ajuste	0,594	0,184	0,859	3	20,7	0,954 / 0,992	0,703
TAGCN	1:10 + SMOTE	Pesos por clase	0,374	0,135	0,809	2	17,7	0,943 / 0,988	0,617
TAGCN	1:10 + SMOTE	Focal loss	0,323	0,138	0,804	3	16,3	0,907 / 0,993	0,479
TAGCN	1:10 + SMOTE	Sin ajuste	0,575	0,213	0,876	3	19,7	0,944 / 0,994	0,680
TAGCN	1:20	Pesos por clase	0,214	0,113	0,754	0	14,7	0,917 / 0,993	0,526
TAGCN	1:20	Focal loss	0,512	0,207	0,867	3	21,3	0,929 / 0,994	0,655
TAGCN	1:20	Sin ajuste	0,307	0,137	0,795	1	9,7	0,926 / 0,994	0,619
TAGCN	Nativo	Pesos por clase	0,212	0,113	0,754	0	14,7	0,917 / 0,992	0,526
TAGCN	Nativo	Focal loss	0,529	0,244	0,875	3	17,7	0,924 / 0,995	0,629
TAGCN	Nativo	Sin ajuste	0,149	0,052	0,732	0	2,0	0,917 / 0,995	0,625
TAGCN	1:100	Pesos por clase	0,357	0,139	0,798	2	17,0	0,943 / 0,988	0,605
TAGCN	1:100	Focal loss	0,282	0,078	0,751	1	9,0	0,915 / 0,994	0,551
TAGCN	1:100	Sin ajuste	0,046	0,030	0,611	0	0,0	0,907 / 0,992	0,550
TAGCN	1:200	Pesos por clase	0,307	0,109	0,779	2	15,0	0,938 / 0,987	0,552
TAGCN	1:200	Focal loss	0,054	0,029	0,609	0	1,3	0,910 / 0,993	0,539
TAGCN	1:200	Sin ajuste	0,046	0,030	0,611	0	0,0	0,907 / 0,992	0,549

Apéndice E

Estabilidad detallada y transacciones explicadas

E.1. Estabilidad por escenario y arquitectura

La Tabla E.1 desglosa por arquitectura la estabilidad por escenario de la Tabla 6.4, con pesos por clase. Cada celda es la media de tres semillas. Incluye GCN, que no supera el filtro de calidad con esta pérdida. Ninguna arquitectura muestra una caída sistemática hacia 1:100 y 1:200.

E.2. Las transacciones explicadas

La Tabla E.2 describe las 30 transacciones ilícitas de validación sobre las que se midió la estabilidad. Todas pertenecen a los pasos de tiempo 30 a 35, tienen pocas aristas y entre la cuarta parte y la mitad de sus variables valen cero. En promedio, cada una es clasificada correctamente por el 64,5 % de los 54 modelos con pesos por clase de las tres arquitecturas que aprenden, de modo que el conjunto incluye transacciones que casi todos los modelos detectan y otras que casi ninguno detecta.

Tabla E.2: Las 30 transacciones ilícitas de validación explicadas. Nodo: índice en el grafo; Paso: paso de tiempo; Grado: número de aristas; Nulas: fracción de variables que valen 0 tras el escalado; Acierto: fracción de los 54 modelos con pesos por clase (GraphSAGE, GAT y TAGCN; 6 escenarios; 3 semillas) que la clasifican como ilícita con umbral 0,5.

Nodo	Paso	Grado	Nulas	Acierto
123122	30	1	0,50	0,69
123491	31	2	0,23	0,41

Nodo	Paso	Grado	Nulas	Acierto
123935	31	2	0,48	1,00
124436	31	2	0,42	0,26
125970	31	1	0,46	0,19
126209	32	2	0,31	0,56
126246	32	2	0,50	0,70
126320	32	2	0,39	0,52
126853	32	2	0,39	0,50
126854	32	2	0,42	0,52
127435	32	1	0,39	0,54
128100	32	2	0,42	0,41
128946	32	1	0,48	0,43
128985	32	1	0,47	0,56
129303	32	2	0,45	0,89
129889	32	2	0,42	0,52
130250	32	1	0,45	0,98
130493	32	2	0,48	1,00
136391	35	2	0,47	0,74
136551	35	1	0,47	0,65
136736	35	2	0,46	0,65
139218	35	2	0,42	1,00
139219	35	2	0,48	0,72
139715	35	1	0,50	0,31
139737	35	1	0,50	0,89
140788	35	1	0,42	0,44
140789	35	1	0,47	0,98
140830	35	1	0,47	0,65
140890	35	1	0,48	0,98
140978	35	1	0,46	0,69

Tabla E.1: Estabilidad entre réplicas por escenario y arquitectura (pesos por clase, media de 3 semillas).

Medida	Explicador	Arquitectura	1:10	1:10 + SMOTE	1:20	Nativo	1:100	1:200
Spearman global	GNNExplainer	GCN	0,936	0,937	0,937	0,937	0,936	0,942
		GraphSAGE	0,929	0,926	0,953	0,916	0,957	0,937
		GAT	0,937	0,919	0,918	0,918	0,918	0,921
		TAGCN	0,940	0,943	0,917	0,917	0,943	0,938
	Shapley	GCN	0,996	0,996	0,996	0,996	0,996	0,996
		GraphSAGE	0,994	0,994	0,992	0,995	0,992	0,992
		GAT	0,994	0,993	0,993	0,993	0,993	0,992
		TAGCN	0,989	0,988	0,993	0,992	0,988	0,987
	Expected Gradients	GCN	0,923	0,922	0,923	0,923	0,918	0,914
		GraphSAGE	0,941	0,946	0,906	0,948	0,915	0,945
		GAT	0,939	0,930	0,929	0,929	0,929	0,924
		TAGCN	0,912	0,915	0,926	0,926	0,911	0,913
Spearman $\neq 0$	GNNExplainer	GCN	0,689	0,695	0,690	0,690	0,688	0,720
		GraphSAGE	0,660	0,651	0,781	0,596	0,800	0,701
		GAT	0,692	0,616	0,616	0,615	0,615	0,626
		TAGCN	0,688	0,701	0,565	0,566	0,706	0,674
	Shapley	GCN	0,993	0,994	0,994	0,993	0,993	0,993
		GraphSAGE	0,992	0,991	0,987	0,993	0,989	0,989
		GAT	0,992	0,990	0,989	0,989	0,989	0,987
		TAGCN	0,980	0,980	0,990	0,989	0,978	0,978
	Expected Gradients	GCN	0,958	0,958	0,958	0,957	0,955	0,952
		GraphSAGE	0,968	0,973	0,943	0,975	0,951	0,960
		GAT	0,965	0,959	0,959	0,958	0,959	0,957
		TAGCN	0,944	0,943	0,961	0,961	0,946	0,945
Jaccard top-10	GNNExplainer	GCN	0,703	0,709	0,709	0,710	0,713	0,699
		GraphSAGE	0,604	0,605	0,669	0,611	0,660	0,600
		GAT	0,656	0,645	0,648	0,650	0,647	0,648
		TAGCN	0,602	0,617	0,526	0,526	0,605	0,552
	Shapley	GCN	0,946	0,949	0,948	0,950	0,940	0,950
		GraphSAGE	0,936	0,921	0,913	0,944	0,901	0,900
		GAT	0,943	0,915	0,924	0,924	0,926	0,911
		TAGCN	0,873	0,860	0,925	0,925	0,843	0,847
	Expected Gradients	GCN	0,797	0,803	0,800	0,792	0,796	0,794
		GraphSAGE	0,808	0,809	0,776	0,805	0,777	0,830
		GAT	0,828	0,818	0,807	0,812	0,812	0,813
		TAGCN	0,813	0,815	0,800	0,794	0,791	0,824

Apéndice F

Glosario

Arista. Conexión entre dos nodos de un grafo. En Elliptic, un flujo de fondos de una transacción a otra.

Campo receptivo. Conjunto de nodos cuya información usa una GNN para predecir un nodo; con L capas, los vecinos a L saltos o menos.

Cierre del mercado. Cierre de un mercado ilegal de la red oscura en el paso de tiempo 43 de Elliptic, después del cual la actividad ilícita cambia y los modelos dejan de detectarla.

Control de pesos aleatorios. Prueba que compara la explicación de un modelo entrenado con la de la misma arquitectura sin entrenar; si se parecen, la explicación no depende de lo aprendido.

Desbalance de clases. Situación en la que una clase tiene muchos menos ejemplos que la otra. Se expresa como una razón, por ejemplo 1:41 (una ilícita por cada 41 negativas).

Escenario. Nivel de desbalance con el que se entrena un modelo, obtenido submuestreando negativas, sobremuestreando ilícitas o, en los escenarios de estrés, quitando ilícitas.

Estabilidad. Grado en que un explicador produce la misma explicación al repetirse sobre la misma transacción y el mismo modelo.

Explicador. Método que asigna a cada variable, o a cada arista, una importancia en una predicción concreta.

Filtro de calidad. Criterio para considerar que un modelo aprendió: F1 de al menos 0,30 y MCC de al menos 0,15 en validación, con umbral 0,5.

***Focal loss*.** Función de pérdida que reduce el peso de los ejemplos que el modelo ya clasifica bien y pondera la clase rara.

Grafo. Estructura formada por nodos y aristas. En Elliptic, los nodos son transacciones de Bitcoin.

Heterofilia. Tendencia de los nodos de clases distintas a conectarse entre sí; lo opuesto es la homofilia.

Jaccard top-10. Fracción de variables compartidas entre las 10 más importantes de dos explicaciones, sobre el total de variables distintas que aparecen en ambas listas.

Modo C. Definición de la clase negativa como lícitas más transacciones sin etiqueta.

Partición temporal. División de los datos en entrenamiento, validación y prueba según el paso de tiempo, de modo que el modelo nunca ve el futuro.

Pesos por clase. Función de pérdida en la que cada clase aporta la mitad del total, sin importar cuántos ejemplos tenga.

PR-AUC. Área bajo la curva de precisión y exhaustividad. Su valor al azar es la proporción de ilícitas.

Pseudorreplicación. Tratar como independientes observaciones que no lo son, lo que infla la significancia estadística.

Réplica. Cada una de las cinco ejecuciones de un explicador sobre la misma transacción y el mismo modelo, con semillas distintas.

Semilla. Número que fija la aleatoriedad del entrenamiento o de un explicador. Dos modelos que solo difieren en la semilla parten de pesos iniciales distintos.

SMOTE. Técnica que crea ejemplos sintéticos de la clase rara interpolando entre ejemplos reales.

Spearman global. Correlación de rangos entre los órdenes completos de importancia de las 165 variables de dos explicaciones.

Variable nula. Variable que vale exactamente cero tras el escalado, por ser igual a la mediana del entrenamiento.